



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA

Trabajo fin de Grado

Grado en Ingeniería Informática — Ingeniería del Software

Sistema de Gestión de Entregables Académicos

**Realizado por
Manuel María Calderón Rodríguez
José Manuel Márquez Gutiérrez**

**Dirigido por
Antonio Manuel Gutiérrez Fernández**

**Departamento
Escuela Técnica Superior de Ingeniería Informática**

Sevilla, Mayo de 2026

Resumen

Contexto y problema: En la operativa académica habitual, los flujos de trabajo de entregas académicas suelen estar fragmentados entre herramientas de propósito general, lo que genera fricciones en trazabilidad, control de versiones de entregas y consistencia de calificaciones. Esta memoria aborda la necesidad de disponer de un flujo integral de extremo a extremo, desde la definición del entregable hasta la publicación de retroalimentación y calificaciones, adaptado a administradores, docentes y estudiantes.

Relevancia: La necesidad es especialmente significativa por su impacto directo en la carga de trabajo docente, la experiencia del estudiante y la mantenibilidad a largo plazo a medida que aumenta el número de cursos y grupos académicos. Además, aborda la mitigación del error humano mediante la implementación de un motor de validación semántica y estructural de entregables. Se busca garantizar la integridad y conformidad de los entregables mediante la definición dinámica de esquemas de validación.

Solución propuesta y método: Se diseña e implementa una plataforma web con arquitectura cliente-servidor desacoplada, con frontend en React + TypeScript y backend en Spring Boot sobre Java 21, persistencia relacional y migraciones versionadas con Flyway. La seguridad combina autenticación JWT con renovación de sesión, autorización por rol y validaciones por contexto académico. Además, el sistema se integra con OneDrive para la gestión centralizada de los archivos asociados a las entregas. El desarrollo sigue incrementos iterativos con trazabilidad explícita entre requisitos, backlog, implementación y pruebas.

Resultados y por qué la solución es adecuada: La plataforma resultante cubre el ciclo funcional completo (cursos, actividades, entregables, entregas, calificación y retroalimentación), aplicando una estrategia de calidad en capas (pruebas unitarias, de integración, de extremo a extremo y de mutación), además de automatización CI/CD y despliegue en cloud reproducible. El resultado es una solución funcional, mantenible y evolutiva, con menor riesgo de regresiones y mayor trazabilidad técnica.

Agradecimientos

Queremos expresar nuestro agradecimiento al tutor del Trabajo Fin de Grado, por la orientación técnica y metodológica aportada durante el desarrollo del proyecto, y por su seguimiento continuo en las distintas fases de análisis, implementación y validación.

Asimismo, agradecemos a la Escuela Técnica Superior de Ingeniería Informática de la Universidad de Sevilla el marco formativo y los recursos que han hecho posible la realización de este trabajo.

Finalmente, reconocemos el apoyo de nuestras familias y compañeros, cuya ayuda ha sido clave para completar este proyecto con el nivel de exigencia requerido.

Índice general

Índice general	III
Índice de tablas	IX
Índice de figuras	XI
1 Introducción y antecedentes	1
1.1 Contexto y problema a resolver	1
1.2 Análisis de antecedentes y situación inicial	1
1.2.1 Actores y procesos principales	2
1.2.2 Fortalezas del entorno de partida	2
1.2.3 Debilidades estructurales detectadas	2
1.2.4 Brechas entre escenario actual y necesidades objetivo	2
1.2.5 Entorno tecnológico de partida	3
1.2.6 Análisis de competidores y brechas sectoriales	3
1.2.7 Aportación realizada (escenario <i>objetivo</i>)	4
1.2.8 Matriz de mapeo debilidades-aporaciones	4
1.2.9 Comparativa sintética	5
1.2.10 Alineación con objetivos de negocio	5
1.2.11 Limitaciones residuales y continuidad	6
1.3 Aportación del proyecto y justificación	6
1.4 Objetivos del proyecto	6
1.4.1 Metodología para definir los objetivos	7
1.4.2 Priorización de objetivos (criterio MoSCoW)	7
1.4.3 Alcance funcional del trabajo	8
1.4.4 Delimitación explícita del alcance	8
1.4.5 Criterios de cumplimiento y evidencia de logro	9
1.4.6 Trazabilidad objetivos-requisitos-entregables	9
1.4.7 Riesgos estratégicos asociados a los objetivos	10
1.5 Alcance funcional y riesgos estratégicos	10
1.6 Ecosistema de desarrollo y operación	10
2 Análisis temporal y costes de desarrollo	12
2.1 Análisis temporal	12
2.1.1 Metodología aplicada y roles del proyecto	12
2.1.2 Herramientas de gestión y seguimiento	12
2.1.3 Marco de planificación	12
2.1.4 Hipótesis de capacidad y calendario base	13
2.1.5 Planificación macro por sprints	13
2.1.6 Planificación micro y distribución del esfuerzo	13

2.1.7	Hitos de control del proyecto	14
2.1.8	Seguimiento del avance y métricas de control	14
2.1.9	Control de desviaciones con valor ganado simplificado	14
2.1.10	Desviaciones y replanificación	15
2.1.11	Lecciones aprendidas de planificación	15
2.1.12	Riesgos temporales y mitigación	15
2.1.13	Conclusión temporal	15
2.2	Costes de desarrollo	15
2.2.1	Criterio de estimación económica	15
2.2.2	Coste de personal	16
2.2.3	Coste de personal por iteración	16
2.2.4	Coste asociado a aseguramiento de calidad	16
2.2.5	Costes de infraestructura y operación	17
2.2.6	Coste total del proyecto y escenarios de sensibilidad	17
2.2.7	Interpretación económica y sostenibilidad	18
2.2.8	Conclusión del análisis de costes	18
3	Análisis y requisitos	19
3.1	El proceso de análisis en la Ingeniería del Software	19
3.1.1	Situación actual y necesidades de negocio	19
3.2	Especificación de Requisitos del Sistema (ERS)	19
3.2.1	Fundamentación teórica del ERS	19
3.2.2	Fuentes de requisitos	19
3.2.3	Estructura del catálogo	20
3.2.4	Requisitos funcionales	20
3.2.5	Reglas de negocio	20
3.2.6	Requisitos no funcionales	20
3.2.7	Priorización y alcance	21
3.2.8	Trazabilidad requisito-backlog-código	21
3.2.9	Contexto documental de partida (ERS y DAS)	22
3.2.10	Descomposición en subsistemas (ERS)	23
3.2.11	Catálogo exhaustivo de requisitos del ERS	23
3.3	Documento de Arquitectura de Software (DAS)	26
3.3.1	Fundamentación teórica del DAS	26
3.3.2	Reutilización de diagramas y mockups del DAS	26
3.3.3	Lectura técnica de los diagramas estructurales del DAS	32
3.3.4	Flujos operativos del DAS y su encaje en la solución	33
3.3.5	Catálogo de casos de uso y matriz de trazabilidad ERS-DAS-implementación	37
3.3.6	Cobertura documental alcanzada	41
4	Diseño y Arquitectura	42
4.1	Diseño del sistema	42
4.1.1	Arquitectura de alto nivel	42
4.1.2	Aportación arquitectónica y tecnológica	42
4.1.3	Vista de despliegue	42
4.1.4	Diseño de dominio y datos	43
4.1.5	Diseño de API	43
4.1.6	Diseño de seguridad	43
4.1.7	Diseño del frontend	43
4.2	Alternativas de arquitectura y stack	44
4.2.1	Arquitectura de aplicación	44
4.2.2	Backend framework	44

4.2.3	Frontend framework	44
4.3	Alternativas de persistencia y almacenamiento	45
4.3.1	Base de datos relacional	45
4.3.2	Estrategia de esquema	45
4.3.3	Almacenamiento de archivos	45
4.4	Alternativas de seguridad y sesión	45
4.4.1	Modelo de autenticación	45
4.4.2	Extensiones de seguridad	46
4.5	Alternativas investigadas y descartadas	46
4.5.1	SharePoint para OneDrive corporativo	46
4.5.2	Registro de usuarios con OAuth	46
4.6	Alternativas de despliegue y operación	46
4.6.1	Estrategia de despliegue	46
4.6.2	CI/CD y calidad	46
4.7	Comparativa multicriterio	47
4.8	Decisiones adoptadas y justificación final	47
4.9	Conclusiones	47
5	Implementación y Desarrollo	48
5.1	Implementación	48
5.1.1	Implementación backend	48
5.1.2	Implementación de flujos críticos	48
5.1.3	Implementación frontend	48
5.1.4	Persistencia, migraciones e integraciones	48
5.1.5	Verificación de implementación	49
5.2	Discusión técnica	49
5.2.1	Decisiones y compromisos	49
5.2.2	Limitaciones y trabajo futuro	50
5.3	Conclusiones	51
6	Justificación detallada de decisiones	52
6.1	Introducción	52
6.2	Marco de toma de decisiones	52
6.2.1	Criterios utilizados	52
6.2.2	Ponderación de criterios	52
6.3	Decisiones de alcance funcional	53
6.3.1	Decisión D1: definir un MVP centrado en flujo académico nuclear	53
6.3.2	Decisión D2: diseñar experiencia por rol desde el inicio	53
6.4	Decisiones de arquitectura	53
6.4.1	Decisión D3: arquitectura SPA + API desacoplada	53
6.4.2	Decisión D4: arquitectura backend por capas	53
6.4.3	Decisión D5: uso de DTOs y mapper dedicado	54
6.5	Decisiones de stack tecnológico	54
6.5.1	Decisión D6: backend en Spring Boot + Java 21	54
6.5.2	Decisión D7: frontend en React + TypeScript + Vite	54
6.6	Decisiones de seguridad	54
6.6.1	Decisión D8: autenticación JWT con refresh token	54
6.6.2	Decisión D9: autorización por rol + contexto académico	54
6.7	Decisiones de datos e integraciones	55
6.7.1	Decisión D10: PostgreSQL como base de datos principal	55
6.7.2	Decisión D11: Flyway como autoridad de esquema	55
6.7.3	Decisión D12: estrategia híbrida de almacenamiento de archivos	55

6.8	Decisiones de calidad y verificación	55
6.8.1	Decisión D13: testing en capas	55
6.8.2	Decisión D14: incluir mutación (PIT/Stryker)	55
6.9	Decisiones de operación y despliegue	55
6.9.1	Decisión D15: GitHub Actions como columna vertebral CI/CD	55
6.9.2	Decisión D16: despliegue en PaaS gestionado	55
6.10	Matriz consolidada de decisiones	56
6.11	Análisis coste-beneficio por bloques de decisiones	56
6.11.1	Bloque de arquitectura	56
6.11.2	Bloque de stack tecnológico	57
6.11.3	Bloque de seguridad	57
6.11.4	Bloque de calidad y pruebas	57
6.11.5	Bloque de despliegue y operación	57
6.12	Riesgos asociados a decisiones y medidas de contingencia	57
6.13	Trazabilidad de decisiones con requisitos y evidencia	58
6.14	Decisiones descartadas y por qué no se adoptaron en esta fase	58
6.14.1	Autenticación federada completa en MVP	58
6.14.2	2FA integral desde primera versión	59
6.14.3	Orquestación avanzada de infraestructura	59
6.15	Conclusiones	59
7	Pruebas	60
7.1	Introducción	60
7.2	Objetivos de verificación	60
7.3	Estrategia de pruebas en capas	60
7.4	Inventario cuantitativo de pruebas realizadas	61
7.5	Pruebas en frontend	61
7.5.1	Pruebas unitarias y cobertura	61
7.5.2	Módulos testeados en frontend	61
7.5.3	Pruebas E2E	62
7.5.4	Mutación en frontend	62
7.6	Pruebas en backend	62
7.6.1	Pruebas unitarias e integración	62
7.6.2	Módulos testeados en backend	63
7.6.3	Cobertura backend	63
7.6.4	Mutación en backend	63
7.7	Automatización en CI/CD	64
7.8	Resultados verificados	64
7.9	Criterios de aceptación	64
7.10	Limitaciones y riesgos	64
7.11	Conclusiones	65
8	Manual de usuario	66
8.1	Introducción	66
8.2	Acceso al sistema	66
8.2.1	Requisitos previos de uso	66
8.2.2	Pantallas de acceso	66
8.2.3	Ciclo de sesión	66
8.2.4	Recuperación de contraseña	67
8.3	Perfiles y permisos	67
8.4	Vistas comunes a todos los perfiles	67
8.5	Guía de uso para profesorado	68

8.5.1	Vistas principales	68
8.5.2	Crear y configurar una actividad	68
8.5.3	Crear y editar entregables	71
8.5.4	Revisar entregas y evaluar	74
8.5.5	Flujo docente recomendado de extremo a extremo	76
8.6	Guía de uso para estudiantes	77
8.6.1	Vistas principales	77
8.6.2	Consultar actividades y entregables	77
8.6.3	Realizar una entrega	78
8.6.4	Recomendaciones para entregas sin incidencias	79
8.6.5	Consultar historial y feedback	79
8.7	Guía de uso para administración	80
8.7.1	Administración de usuarios	80
8.7.2	Administración de cursos	81
8.7.3	Administración de grupos	84
8.7.4	Panel de administración	87
8.7.5	Operaciones destacadas	87
8.7.6	Políticas recomendadas de gobierno de usuarios	87
8.8	Mensajes de error y resolución de incidencias	87
8.8.1	Matriz de incidencias operativas	87
8.9	Seguridad y protección de la información en el uso diario	88
8.10	Checklist operativo por perfil	88
8.11	Buenas prácticas de uso	88
8.12	Alternativa a la enseñanza virtual y aspectos novedosos	89
8.13	Desafíos técnicos encontrados en el proyecto	89
8.14	Análisis de la evolución del proyecto	89
8.15	Conclusiones	89
9	Conclusiones y desarrollos futuros	90
9.1	Síntesis final del trabajo	90
9.2	Grado de cumplimiento de objetivos	90
9.3	Aportaciones principales	90
9.4	Resultados y evidencia	91
9.5	Evaluación del impacto por tipo de usuario	91
9.6	Validación del enfoque de ingeniería adoptado	91
9.7	Análisis de desviaciones en la planificación temporal	91
9.8	Limitaciones del trabajo	92
9.9	Lecciones aprendidas	92
9.10	Riesgos residuales y deuda técnica controlada	92
9.11	Desarrollos futuros	92
9.11.1	Corto plazo	92
9.11.2	Medio plazo	93
9.11.3	Largo plazo	93
9.12	Hoja de ruta priorizada de continuidad	93
9.13	Reflexión final	93
9.14	Cierre	93
A	Manual de instalación	95
A.1	Requisitos previos	95
A.2	Preparación del entorno	95
A.3	Base de datos	95
A.4	Arranque del backend	95

A.5	Arranque del frontend	95
A.6	Ejecución con contenedores	96
A.7	Build de producción	96
B	Configuración por entornos	97
B.1	Variables críticas de backend	97
B.2	Variables críticas de frontend	97
C	Operación y verificación	98
C.1	Comprobaciones de salud	98
C.2	Validación post-despliegue	98
D	Declaración de uso de IA generativa	99
D.1	Alcance y criterios de uso	99
D.2	Proceso de control y verificación	99
D.3	Ejemplos concretos de consultas y respuestas	99
E	Documentación complementaria disponible	101
F	Caso de uso de ejemplo	102
F.1	1. Administrador: Configuración inicial	102
F.2	2. Profesor: Creación de la actividad y el entregable	102
F.3	3. Estudiante: Revisión y entrega	102
F.4	4. Profesor: Corrección y retroalimentación	102
F.5	5. Estudiante: Consulta de resultados	103
	Referencias	104

Índice de tablas

1.1	Brechas identificadas entre el escenario de partida y el objetivo	2
1.2	Relación entre debilidades detectadas y aportación implementada	5
1.3	Comparativa entre escenario de partida y propuesta desarrollada	5
1.4	Priorización de objetivos conforme a estrategia MoSCoW	8
1.5	Criterios de evaluación del cumplimiento de objetivos	9
1.6	Trazabilidad entre objetivos, requisitos y resultados técnicos	10
2.1	Planificación temporal macro del proyecto	13
2.2	Seguimiento temporal con valor ganado simplificado	14
2.3	Riesgos temporales y acciones de mitigación	15
2.4	Coste de personal para 660 horas de proyecto	16
2.5	Coste por sprint según escenario de tarifa	16
2.6	Coste estimado del aseguramiento de calidad	17
2.7	Comparativa entre coste real y coste operativo orientativo	17
2.8	Coste total estimado del proyecto según escenario de tarifa	18
3.1	Requisitos no funcionales representativos	21
3.2	Ejemplo de trazabilidad entre requisitos, backlog e implementación	22
3.3	Subsistemas definidos en ERS y reutilizados en el diseño del TFG	23
3.4	Inventario de requisitos generales (RQG) en ERS	23
3.5	Inventario de requisitos de información (RQI) en ERS	24
3.6	Inventario de requisitos funcionales (RQF-001 a RQF-012)	24
3.7	Inventario de requisitos funcionales (RQF-013 a RQF-025)	25
3.8	Inventario de reglas de negocio (REGN) en ERS	25
3.9	Inventario de requisitos no funcionales (RQNF) en ERS	26
3.10	Diagramas principales reutilizados desde DAS	26
3.11	Inventario de mockups de interfaz reutilizados desde DAS	32
3.12	Interpretación técnica de los diagramas estructurales del DAS	33
3.13	Cobertura operativa por rol con evidencia de DAS y requisitos de ERS	37
3.14	Cobertura por familias de casos de uso del DAS dentro del alcance del MVP	40
3.15	Trazabilidad ampliada entre ERS, DAS e implementación efectiva	40
4.1	Vista de despliegue simplificada del sistema	43
4.2	Comparativa global (escala 1-5, mayor es mejor)	47
6.1	Ponderación orientativa para selección de decisiones	53
6.2	Resumen de decisiones clave y justificación dominante	56
6.3	Riesgos derivados de decisiones y medidas de contingencia	58
6.4	Trazabilidad entre decisiones, requisitos y evidencia	58
7.1	Matriz de niveles y herramientas de prueba	61

7.2	Distribución de casos unitarios por módulo en frontend	62
7.3	Distribución de casos de prueba por módulo en backend	63
7.4	Resumen de resultados de verificación	64
8.1	Capacidades por perfil de usuario	67
8.2	Matriz de incidencias frecuentes y resolución recomendada	87
8.3	Checklist operativo por perfil	88
9.1	Hoja de ruta priorizada para evolución del sistema	93

Índice de figuras

3.1	Arquitectura lógica del sistema (DAS)	27
3.2	Modelo de clases del sistema (DAS)	27
3.3	Diagrama de estado de actividades (DAS)	28
3.4	Navegabilidad principal de la interfaz (DAS)	28
3.5	Secuencia de autenticación y acceso (DAS)	29
3.6	Secuencia de realización de entrega (DAS)	29
3.7	Mockup de panel principal de estudiante (DAS)	30
3.8	Mockup de panel principal de profesor (DAS)	31
3.9	Secuencia de creación de actividad en backend (DAS)	34
3.10	Secuencia de evaluación y feedback (DAS)	34
3.11	Mockup operativo: estudiante dentro de una asignatura (DAS)	35
3.12	Mockup operativo: detalle de calificación como profesor (DAS)	36
3.13	Mockup de entrega con archivo adjunto (DAS)	37
3.14	Caso de uso representativo CU-1 (DAS)	38
3.15	Caso de uso representativo CU-2 (DAS)	38
3.16	Caso de uso representativo CU-3 (DAS)	39
3.17	Caso de uso representativo CU-4 (DAS)	39
8.1	Pantalla de inicio de sesión. Vista compartida por todos los perfiles.	67
8.2	Sección de perfil (1/2). Vista compartida por todos los perfiles.	68
8.3	Sección de perfil (2/2). Vista compartida por todos los perfiles.	68
8.4	Dashboard docente. Secciones visibles: dashboard, actividades, perfil y evaluaciones.	69
8.5	Sección de actividades por curso.	69
8.6	Detalle de asignatura con profesorado asignado.	70
8.7	Creación de actividad (1/2).	70
8.8	Creación de actividad (2/2).	71
8.9	Eliminación de actividad desde la asignatura.	71
8.10	Detalle de actividad y entregables asociados.	72
8.11	Creación de entregables (1/2).	72
8.12	Creación de entregables (2/2).	73
8.13	Edición de actividad (1/3).	73
8.14	Edición de actividad (2/3).	74
8.15	Edición de actividad (3/3).	74
8.16	Detalle de entregable para revisiones.	75
8.17	Bandeja de evaluaciones (1/2).	75
8.18	Bandeja de evaluaciones (2/2).	76
8.19	Evaluación de la actividad de un alumno.	76
8.20	Dashboard de estudiante. Secciones visibles: panel principal, actividades, perfil y calificaciones.	77
8.21	Detalle de entregable desde la visión de estudiante.	77

8.22	Formulario de entrega (1/2).	78
8.23	Formulario de entrega (2/2).	78
8.24	Entrega registrada correctamente.	79
8.25	Consulta de calificación por estudiante.	79
8.26	Panel de administración. Secciones visibles: usuarios, cursos y grupos.	80
8.27	Creación de usuarios.	80
8.28	Modificación de usuarios.	81
8.29	Eliminación de usuarios.	81
8.30	Gestión de cursos.	82
8.31	Creación de curso (1/2).	82
8.32	Creación de curso (2/2).	83
8.33	Edición de curso.	83
8.34	Eliminación de curso.	84
8.35	Asignación de profesorado a cursos.	84
8.36	Gestión de grupos.	85
8.37	Creación de grupo.	85
8.38	Edición de grupo.	86
8.39	Eliminación de grupo.	86
8.40	Asignación de usuarios en grupo.	86

Introducción y antecedentes

1.1– Contexto y problema a resolver

El presente Trabajo Fin de Grado se desarrolla en el *ámbito* de la gestión académica universitaria, centrado en el ciclo de vida completo de las actividades evaluables: definición de actividades, publicación de entregables, realización de entregas, evaluación y retroalimentación.

La motivación principal se fundamenta en un problema habitual detectado durante el flujo de entrega: el uso de la plataforma institucional requiere tareas manuales de clasificación y validación de archivos. Esta situación introduce ineficiencias críticas: aumenta el esfuerzo administrativo del profesorado, reduce la precisión en la guía al estudiante y debilita la trazabilidad entre las entregas, sus versiones y el *feedback* asociado.

Desde la perspectiva de la ingeniería del software, el problema a resolver consiste en diseñar un sistema especializado que, con recursos acotados, maximice la cobertura funcional del ciclo de entregables, garantizando seguridad por rol, una interfaz de usuario enriquecida que reduzca los errores involuntarios y capacidad de evolución, sin depender de tareas manuales externas a la plataforma.

Para el diseño del proyecto nos apoyamos en metodologías de la Ingeniería del Software: (i) la elicitación de requisitos y el análisis y diseño recogidos en los artefactos ERS y DAS; y (ii) la planificación y seguimiento mediante backlog y plan de sprints (SCRUM). Estos artefactos garantizan trazabilidad entre requisitos, implementación y pruebas.

1.2– Análisis de antecedentes y situación inicial

En el entorno académico actual, intervienen tres actores fundamentales: el **profesor**, que gestiona la recogida y corrección; el **estudiante**, que realiza las entregas; y el **administrador**, que mantiene la infraestructura. Aunque los sistemas de gestión del aprendizaje (LMS) generalistas ofrecen ventajas como la autenticación centralizada y el acceso estandarizado, carecen de especialización en el flujo de entregables.

Las plataformas LMS tradicionales tienden a tratar las entregas como simples cargas de archivos, sin validaciones profundas sobre la estructura del contenido (p. ej., archivos requeridos dentro de un paquete ZIP). Las herramientas especializadas en corrección técnica son potentes para código pero rígidas para entregas heterogéneas. Asimismo, el uso de suites colaborativas en la nube fragmenta la trazabilidad y desvincula la evaluación de la entrega real. Esta dependencia de herramientas no adaptadas traslada gran parte del esfuerzo de validación y organización al puesto local del docente, incrementando la carga administrativa y el riesgo de inconsistencias.

1.2.1. Actores y procesos principales

En la situación de partida intervienen tres actores de negocio: el **profesor**, que crea actividades y gestiona la recogida y corrección de entregas; el **estudiante**, que realiza la entrega de ficheros en la plataforma docente; y el **administrador de plataforma**, responsable de mantener servicios base e infraestructura institucional. Esta distribución delimita responsabilidades y explica por qué ciertos problemas se concentran en la realización y evaluación de entregables.

Los procesos de referencia en ese escenario son la creación y publicación de tareas, la entrega de prácticas por el alumnado y la descarga masiva y organización manual de entregas por el profesorado. La presencia de un proceso dedicado a organización manual evidencia la dificultad de gestionar el flujo de entregables de manera eficiente.

1.2.2. Fortalezas del entorno de partida

El análisis del contexto inicial evidencia fortalezas relevantes. En particular, se aporta centralización de usuarios mediante identidad institucional, se garantiza disponibilidad generalizada y acceso web estandarizado, y se ofrece un marco de uso conocido por estudiantes y docentes. Estas ventajas justifican una estrategia de complementariedad en lugar de sustitución total.

Estas fortalezas justifican que la propuesta no persiga sustituir por completo el ecosistema académico existente, sino complementarlo en el punto donde se concentra la fricción operativa.

1.2.3. Debilidades estructurales detectadas

Junto a las fortalezas, se detectan debilidades que afectan al rendimiento del proceso. La **DEB-001** se manifiesta en rigidez de gestión de archivos y clasificación manual reiterada, mientras que la **DEB-002** limita la evolución funcional al depender de arquitecturas cerradas. A esto se suma la **DEB-003**, que reduce la flexibilidad del flujo de entrega para el estudiante, y la **DEB-004**, que refleja una integración no nativa con almacenamiento cloud moderno. En conjunto, estas debilidades concentran el coste operativo en fases críticas de entrega y corrección.

1.2.4. Brechas entre escenario actual y necesidades objetivo

Para justificar la intervención, se identifican brechas concretas entre la situación *actual* y el resultado esperado en términos de ingeniería.

La Tabla 1.1 sintetiza las principales brechas detectadas y su impacto sobre las necesidades objetivo.

Brecha	Situación inicial	Necesidad objetivo
Operativa docente	Descarga y organización manual recurrente	Flujo integrado de corrección con menor carga administrativa
Experiencia de entrega	Interacción poco guiada según contexto	Flujo orientado por rol y estado de entregable
Trazabilidad académica	Datos dispersos entre herramientas y estados	Cadena unificada entrega-version-evaluación-feedback
Evolución funcional	Alta dependencia de plataforma generalista	Arquitectura modular y extensible por componentes
Calidad y despliegue	Validación heterogénea por tarea	Verificación automatizada y entrega continua

Tabla 1.1: Brechas identificadas entre el escenario de partida y el objetivo

1.2.5. Entorno tecnológico de partida

El contexto previo se caracteriza por una plataforma institucional LMS con autenticación centralizada, acceso multiplataforma desde navegador y dependencia de herramientas locales para el tratamiento de paquetes de entrega. Esta dependencia traslada parte del esfuerzo al puesto de trabajo del docente y amplifica diferencias de criterio en la gestión de materiales.

La consecuencia directa es que parte del trabajo crítico de evaluación se desplaza al puesto local del docente, con impacto en tiempo de corrección, estandarización de criterios y riesgo de inconsistencias en trazabilidad.

1.2.6. Análisis de competidores y brechas sectoriales

Para dimensionar la aportación real del proyecto en el sector, se revisó el comportamiento de soluciones ampliamente usadas en docencia, así como herramientas más específicas de recepción de entregas. El objetivo no es enumerar marcas, sino identificar patrones funcionales y brechas que condicionan la operativa docente.

Tipologías de competidores analizados

El análisis se ha estructurado en cuatro familias. Se consideran **LMS generalistas** (como Moodle, Canvas, Blackboard o Google Classroom) que priorizan gestión de cursos, contenidos, evaluaciones y comunicación, **herramientas de evaluación técnica** (como GitHub Classroom, Gradescope o CodeGrade) orientadas a entregas de programación o autoevaluación, **suites colaborativas y almacenamiento cloud** (como Microsoft Teams, Google Drive o Dropbox) centradas en compartir archivos y carpetas sin cubrir el flujo completo de entrega, y **gestores de entrega simples** (formularios web o repositorios públicos) que ofrecen subida de ficheros con mínimo control posterior. Esta segmentación permite comparar la propuesta con alternativas reales sin reducir el análisis a marcas concretas.

LMS generalistas

Las plataformas LMS de uso masivo (Moodle, Blackboard, Canvas) ofrecen una cobertura funcional amplia y estable, pero su diseño generalista tiende a tratar las entregas como un módulo más de un ecosistema mayor. Esto provoca que la validación automática de ficheros sea habitualmente limitada a comprobar la extensión y el tamaño, y que el control detallado de la estructura interna de paquetes (como los ficheros .zip) recaiga en instrucciones manuales hacia el alumnado o en una ardua revisión a posteriori por parte del docente.

Herramientas de evaluación técnica

Las plataformas especializadas en corrección técnica priorizan integraciones con repositorios o sistemas de autoevaluación. Son muy potentes cuando la entrega es estrictamente código, pero pierden flexibilidad cuando el entregable es heterogéneo (documentos, material multimedia, informes mixtos o combinaciones).

Suites colaborativas y almacenamiento cloud

Las suites colaborativas permiten compartir archivos y carpetas de forma flexible, pero no sustituyen un flujo académico completo. En la práctica, la trazabilidad de versiones, los estados de entrega y la publicación de notas suelen quedar fuera del sistema, dependiendo de procesos manuales o documentos externos.

Gestores de entrega simples

Las soluciones más ligeras se centran en el acto de subir archivos, pero no ofrecen trazabilidad de versiones, validación avanzada ni mecanismos de control contextual por curso y grupo. En estos escenarios el docente debe reconstruir manualmente qué se entregó, cuándo y bajo qué reglas, lo que incrementa errores y retrabajo.

Brechas sectoriales detectadas

De la revisión anterior se desprenden brechas recurrentes. Falta validación previa de estructura y nombre en paquetes ZIP, la trazabilidad de versiones y reenvíos es limitada dentro del ciclo de entrega, y las validaciones por tipo de entrega resultan insuficientes para escenarios heterogéneos. Además, el control contextual por curso y grupo es poco explícito en herramientas generalistas, y la operativa docente sigue dependiendo de descarga y comprobación manual de lotes de entrega. Estas carencias determinan el foco de la propuesta.

Implicaciones para el diseño del TFG

Estas brechas guiaron la definición del alcance funcional y de las validaciones del sistema. La aportación se orientó a reducir carga manual, mejorar la trazabilidad y ofrecer verificaciones técnicas que no suelen estar disponibles en soluciones generalistas.

1.2.7. Aportación realizada (escenario *objetivo*)

La propuesta implementada aporta una mejora estructural, orientada a reducir fricción operativa, aumentar trazabilidad y consolidar calidad de producto.

Aportación funcional

El sistema desarrollado aborda directamente la fricción operativa y las deficiencias detectadas en las soluciones de mercado. En concreto, se implementaron capacidades de gestión integral de cursos, grupos, actividades y entregables con visibilidad por rol, así como validación avanzada de ZIP (estructura, extensiones y nombre esperado). Además, se incorporó un modo estricto o mínimo requerido en entregas ZIP según criterio docente, control de tipos de entrega (archivo, enlace o solo texto) con reglas específicas, y versionado con reenvío y estado de entrega trazables. Como soporte operativo, se añadió descarga masiva, previsualización de contenido para optimizar revisión y un borrador automático en cliente con revalidación previa al envío.

Estas capacidades se alinean con los procesos objetivo del análisis: **PRON-004** (creación/publicación de actividad), **PRON-005** (realización de entrega con validación) y **PRON-006** (evaluación y feedback).

Aportación en calidad y operación

Además de funcionalidad, el proyecto incorpora capacidades de calidad y operación mediante una estrategia de testing en capas (unitarias, integración, E2E y mutación), pipelines CI/CD para validación continua y despliegue reproducible en entorno PaaS con configuración por entorno. Estas decisiones garantizan que el producto sea verificable y operable en condiciones reales.

La aportación, por tanto, no se limita a “que funcione”, sino a que el sistema sea verificable, mantenible y operable.

1.2.8. Matriz de mapeo debilidades-aportaciones

La Tabla 1.2 sintetiza la relación entre debilidades detectadas y la respuesta concreta implementada en el proyecto.

Debilidad	Impacto inicial	Aportación aplicada
DEB-001	Sobrecarga manual en corrección y organización de ficheros	Flujo estructurado de entregas y consultas por contexto
DEB-002	Dificultad de evolución sobre stack no modular	Arquitectura desacoplada con API y capas de servicio
DEB-003	Fricción para el estudiante durante la entrega	Formularios guiados, validaciones y seguimiento de estado
DEB-004	Dependencia de almacenamiento local/heterogéneo	Integraciones configurables y estrategia de almacenamiento por entorno

Tabla 1.2: Relación entre debilidades detectadas y aportación implementada

1.2.9. Comparativa sintética

La Tabla 1.3 resume el salto entre escenario de partida y la aportación del TFG en términos de operación y trazabilidad.

Aspecto	Situación inicial	Aportación del TFG
Gestión de entregas	Descarga masiva y organización manual en local	Flujo de entrega estructurado con trazabilidad
Evaluación	Proceso parcialmente desacoplado de la entrega	Evaluación y feedback integrados en el mismo ciclo
Evolución técnica	Dependencia de stack monolítico/generalista	Arquitectura modular con API REST
Control de acceso	Autenticación institucional de base	Modelo de roles y validación contextual por operación
Calidad del desarrollo	Validación heterogénea según tarea	Estrategia de testing y automatización CI/CD

Tabla 1.3: Comparativa entre escenario de partida y propuesta desarrollada

1.2.10. Alineación con objetivos de negocio

La aportación realizada materializa los objetivos de negocio definidos en la fase de análisis de la siguiente forma:

- Se consigue el **OBJN-001 (agilizar gestión docente)** mediante la capacidad de definir reglas automáticas de validación de formatos y estructuras internas (ficheros requeridos dentro de un .zip), lo cual reduce sustancialmente el número de entregas inválidas que un profesor debe revisar y descartar manualmente.
- Se satisface el **OBJN-002 (mejorar la accesibilidad y usabilidad del estudiante)** implementando un flujo de subida de archivos claro, con prevalidaciones en tiempo real en el lado del cliente (borrador local) y confirmaciones explícitas de fecha y versión de la entrega.
- Se refuerza el **OBJN-003 (trazabilidad de la evaluación)** al ofrecer una vista dedicada donde la entrega exacta, los comentarios de corrección (feedback) y la calificación cuantitativa conviven en un *único* contexto cerrado, evitando el envío de correos masivos para comunicar notas o incidencias.
- Finalmente, el **OBJN-004 (capacidad de evolución e integración)** se asegura mediante una arquitectura que expone todo su catálogo de operaciones a través de una API documen-

tada, posibilitando enganchar el sistema a módulos externos de autoevaluación de código o herramientas de prevención de plagio en el futuro.

1.2.11. Limitaciones residuales y continuidad

Aun cuando la mejora es significativa, se mantienen líneas de continuidad planificadas: ampliación de funcionalidades avanzadas fuera del MVP, extensión de integraciones externas según contexto institucional y refuerzo de observabilidad y métricas operativas de producción. Esta continuidad permite evolucionar sin comprometer la estabilidad alcanzada.

Estas limitaciones no invalidan la aportación obtenida, sino que delimitan una hoja de ruta realista para evolución posterior.

1.3– Aportación del proyecto y justificación

Para solventar las brechas detectadas, este proyecto aporta un sistema estructurado y específico que transforma un proceso manual y disperso en un flujo digital unificado.

La principal aportación funcional es la gestión integral de actividades y entregables con validaciones avanzadas automáticas (control de formatos, previsualización, y chequeo estricto de estructura), además de un sistema de versionado y trazabilidad completa de cada entrega. Esto permite integrar la evaluación y la retroalimentación en el mismo ciclo, reduciendo drásticamente la sobrecarga manual de clasificación.

A nivel arquitectónico, la solución adopta un diseño desacoplado mediante una API REST, separando la capa de experiencia de usuario de la lógica de negocio y persistencia, lo que facilita el mantenimiento evolutivo. Además, se aplican políticas de calidad y operación modernas (testing en capas, CI/CD) que garantizan que el producto sea verificable y estable en un entorno de despliegue continuo.

1.4– Objetivos del proyecto

El **objetivo general** del trabajo es diseñar, implementar y validar un sistema web para la gestión integral de entregables académicos, que permita a profesores y estudiantes operar de forma centralizada, segura y trazable, reduciendo tareas manuales y mejorando la eficiencia del proceso de evaluación continua.

Para alcanzar este fin, se han definido los siguientes objetivos específicos. En lugar de un listado telegráfico, a continuación se describen de manera extensiva y verificable, indicando su propósito y la evidencia esperada.

Objetivo 1 — Centralizar la estructura académica. Modelar de forma integral la jerarquía académica (cursos, asignaturas, grupos, actividades y entregables) para que toda información relevante esté vinculada por contexto. La centralización reduce ambigüedades operativas, facilita consultas por contexto y permite la trazabilidad entre actividad, entrega y evaluación. La evidencia de cumplimiento se mide por la existencia de los módulos de administración y vistas por rol que permitan navegar y operar sobre esa estructura.

Objetivo 2 — Proporcionar flexibilidad de configuración. Dotar al profesorado de mecanismos para definir plazos, reglas de visibilidad y criterios de validación automática adaptables por actividad. Esto incluye la capacidad de configurar tipos de entrega (archivo, enlace, texto), reglas sobre la estructura de paquetes (p. ej., contenido requerido en ZIP) y niveles de tolerancia en validaciones. La meta es posibilitar que las políticas docentes se expresen sin necesidad de cambios de código, y la evidencia incluye formularios de configuración y pruebas que demuestren su aplicación en distintos contextos.

Objetivo 3 — Optimizar el proceso de entrega. Ofrecer al alumnado un flujo guiado que minimice errores involuntarios: prevalidación en cliente, borradores locales, confirmación de versión y control de versiones de entregables. Este objetivo busca reducir incidencias por entregas

mal formadas y agilizar la revisión docente, medido por la disminución de entregas inválidas en escenarios de prueba y por mejoras en tiempos de corrección en estudios de caso.

Objetivo 4 — Mejorar la evaluación y la retroalimentación. Integrar herramientas de corrección y anotación directamente sobre la entrega, permitiendo a los docentes dejar comentarios vinculados a la versión exacta entregada y gestionar escalas/rubricas. El objetivo es que el feedback sea recuperable, versionado y asociable a la entrega concreta, con evidencia en pantallas, endpoints y exportes que muestren la correspondencia entre entrega, comentario y calificación.

Objetivo 5 — Garantizar seguridad y control de acceso. Diseñar y ejecutar un sistema de seguridad basado en el paradigma RBAC (Role-Based Access Control), que garantice la integridad de los datos y la privacidad de los usuarios mediante una separación estricta de privilegios. El objetivo es asegurar que las operaciones de administración, gestión docente y participación del alumnado estén protegidas por capas de autenticación y autorización consistentes. Adicionalmente, el sistema debe ser capaz de evaluar el acceso no solo por el rol del usuario, sino por el contexto académico (por ejemplo, vinculación a un grupo, pertenencia a una asignatura concreta o vigencia de plazos), mitigando riesgos de acceso no autorizado a información sensible o a acciones administrativas críticas. La evidencia incluye esquemas de autorización, casos de prueba de control de acceso y políticas aplicadas en diferentes contextos.

Objetivo 6 — Diseñar una arquitectura modular. Mantener una separación clara entre frontend y backend mediante APIs bien definidas y contratos (DTOs), de manera que la evolución de componentes sea independiente y la integración con servicios externos resulte sencilla. La modularidad facilita pruebas aisladas, despliegues parciales y la incorporación de nuevos módulos en el futuro, medido por la existencia de interfaces documentadas y pruebas de integración.

Objetivo 7 — Asegurar la calidad técnica. Garantizar la verificación continua mediante pruebas automáticas (unitarias, integración, E2E) y mutación, junto a pipelines CI/CD que validen cada candidato. Este objetivo asegura que las decisiones de diseño se traduzcan en resultados medibles de calidad; la evidencia se compone de informes de cobertura, ejecución de pruebas en CI y resultados de mutación.

Los cinco primeros objetivos constituyen los requisitos indispensables (*Must Have*) para disponer de un MVP plenamente operativo. Los dos últimos se orientan a garantizar mantenibilidad y reproducibilidad técnica en el medio plazo.

1.4.1. Metodología para definir los objetivos

La definición de objetivos no se ha realizado de forma declarativa aislada, sino mediante un proceso iterativo de convergencia entre objetivos de negocio (OBJN) definidos en el análisis inicial, requisitos funcionales y no funcionales del ERS, decisiones arquitectónicas del DAS, prioridades MoSCoW del Product Backlog y la capacidad temporal real (660 horas en 6 iteraciones). Esta convergencia asegura que cada objetivo no solo responda a una necesidad, sino que sea viable dentro del calendario y del alcance del TFG.

Este enfoque permite que cada objetivo sea evaluable, relacionable con entregables técnicos concretos y verificable mediante evidencia objetiva de implementación y pruebas.

1.4.2. Priorización de objetivos (criterio MoSCoW)

Para alinear expectativas y alcance, los objetivos se priorizan con criterio MoSCoW, consistente con el backlog del proyecto.

Prioridad	Objetivos asociados	Justificación
Must Have	O1, O2, O3, O4, O5	Sin estos objetivos no existe flujo académico completo de entrega y evaluación.
Should Have	O6, O7	Aumentan mantenibilidad y calidad, y reducen riesgo de regresión.
Could Have	Extensiones analíticas y observabilidad avanzada	Aportan valor, pero no condicionan la validez del MVP.
Won't Have (MVP)	inicio de sesión federado (OAuth2/OIDC), 2FA integral, notificaciones avanzadas	Se reservan para evolución por coste de implementación y validación.

Tabla 1.4: Priorización de objetivos conforme a estrategia MoSCoW

Referencias conceptuales: Criterio MoSCoW (Must/Should/Could/Won't) (Consortium, 2026) y su aplicación al backlog del proyecto. El método MoSCoW es una técnica de priorización ágil que clasifica los requisitos en Must Have (críticos para el MVP), Should Have (importantes pero no vitales), Could Have (deseables si hay tiempo) y Won't Have (excluidos del alcance actual), asegurando el foco en la entrega de valor temprano.

1.4.3. Alcance funcional del trabajo

El sistema desarrollado se articula en torno a tres pilares operativos que definen su alcance funcional. En primer lugar, se implementa un módulo de gestión de identidad, que permite la administración de usuarios mediante un control de acceso basado en roles (RBAC), garantizando la seguridad de la jerarquía académica (cursos, grupos y actividades). En segundo lugar, el núcleo del sistema se centra en la orquestación de entregables: configuración dinámica de plazos y visibilidad por parte del docente, ciclo de vida completo para la entrega del estudiante (subida, control de versiones y feedback evaluativo). Finalmente, el alcance se extiende hacia la automatización de la infraestructura, integrando procesos de validación y despliegue continuo (CI/CD) que aseguran la integridad técnica de cada entrega en entornos de producción.

El alcance funcional cubierto en este TFG incluye la gestión de usuarios y el control de acceso por rol, la gestión de cursos, grupos y actividades académicas, y la administración de entregables con configuración de plazos y visibilidad. Además, se cubren la realización de entregas con historial de versiones y descarga, la evaluación con retroalimentación docente y la automatización de validación y despliegue en entorno CI/CD para asegurar trazabilidad técnica.

1.4.4. Delimitación explícita del alcance

Para mantener viabilidad temporal y calidad de salida, se delimita de forma explícita lo que queda fuera del MVP, aunque exista su consideración en el análisis inicial. En concreto, se excluyen el inicio de sesión federado (OAuth2/OIDC institucional), el doble factor de autenticación integral (2FA), las notificaciones avanzadas desacopladas y las funcionalidades avanzadas de analítica y monitorización, por su coste de implementación y validación en el horizonte temporal disponible.

Estas exclusiones se refieren al inicio de sesión federado; la autorización OAuth2 necesaria para conectar OneDrive sí está implementada.

Esta delimitación no representa una carencia metodológica, sino una decisión de ingeniería para proteger el objetivo principal: entregar un sistema funcional, verificable y defendible en plazo.

1.4.5. Criterios de cumplimiento y evidencia de logro

Para garantizar una evaluación objetiva del éxito del proyecto, se han definido criterios de aceptación cuantificables para cada objetivo propuesto. En la Tabla 1.5 se detallan los indicadores principales y la evidencia técnica que sustenta el logro de cada hito, priorizando la verificabilidad mediante métricas de calidad.

El cumplimiento de objetivos se evalúa mediante evidencia verificable, evitando formulaciones subjetivas.

Obj.	Indicador principal	Meta de cumplimiento	Evidencia
O1–O4	Cobertura del flujo funcional extremo a extremo	Flujo completo operativo por rol	Endpoints y pantallas de curso/actividad/entrega/evaluación
O5	Restricciones de acceso aplicadas	Operaciones protegidas por rol/contexto	Configuración de seguridad, filtros y tests de autorización
O6	Separación modular de componentes	Bajo acoplamiento entre capas	Arquitectura frontend-backend + DTOs + servicios
O7	Calidad técnica cuantificable	Suites en verde y métricas altas	Cobertura, E2E y mutación reportadas
O8	Reproducibilidad de entrega	Pipeline estable para build/test/deploy	Workflows CI/CD y despliegue trazable

Tabla 1.5: Criterios de evaluación del cumplimiento de objetivos

1.4.6. Trazabilidad objetivos-requisitos-entregables

El rigor metodológico del trabajo se apoya en una matriz de trazabilidad que vincula los objetivos estratégicos con los requisitos funcionales (RQF) y no funcionales (RQNF) definidos en la especificación de requisitos. Esta estructura asegura que cada incremento desarrollado durante los Sprints (EP) responda directamente a las metas de ingeniería del proyecto, garantizando una evolución del software coherente y auditable.

La trazabilidad constituye un criterio central de rigor académico. Cada objetivo se conecta con requisitos del ERS y con entregables técnicos ejecutados durante los sprints. Los requisitos funcionales y no funcionales detallados de forma exhaustiva pueden consultarse en el Capítulo 3.

La Tabla 1.6 muestra la trazabilidad establecida entre objetivos, requisitos y entregables técnicos.

Objetivo	Requisitos relacionados	Backlog/Sprint	Entregable técnico
O1	RQF-005 a RQF-017	EP-04, EP-05 (S1)	Módulos de cursos, grupos y actividades
O2–O3	RQF-018 a RQF-026	EP-06, EP-07 (S1-S2)	Módulos de entregables y entregas
O4	RQF-027 a RQF-029	EP-09 (S1-S3)	Módulo de evaluación y feedback
O5	RQNF-009, REGN-001	EP-12 (S1)	JWT, control de roles y validación contextual
O7	RQNF-001, RQNF-004	EP-13 (S1-S4)	Testing en capas + mutación
O8	RQNF-008, RQNF-004	EP-14 (S1)	CI/CD y despliegue automatizado

Tabla 1.6: Trazabilidad entre objetivos, requisitos y resultados técnicos

1.4.7. Riesgos estratégicos asociados a los objetivos

Al definir objetivos de forma ambiciosa, es necesario explicitar riesgos y medidas de mitigación. Existe un riesgo de sobrealcance funcional si se incorporan más funcionalidades de las asumibles en 660 horas, por lo que la mitigación se apoya en priorización MoSCoW y control por hitos. También hay riesgo de deuda de calidad si se pospone el testing a la fase final, lo que se corrige con una estrategia de pruebas integrada en cada sprint. Por último, el riesgo de incoherencia documental entre memoria y estado real del producto se reduce mediante redacción iterativa y trazabilidad objetiva.

1.5– Alcance funcional y riesgos estratégicos

El alcance de este TFG cubre el flujo de trabajo *core* de la evaluación continua: desde la administración de la estructura académica, hasta la realización de entregas, la corrección y la descarga de evidencias.

Para mantener la viabilidad temporal del proyecto y asegurar la calidad, se excluyen deliberadamente algunas características avanzadas contempladas en el análisis teórico, como la implementación de inicio de sesión federado institucional (OAuth2/OIDC pleno), un doble factor de autenticación integral (2FA) nativo y las notificaciones desacopladas en tiempo real.

La ejecución de este alcance se gestionó asumiendo riesgos clave, como la complejidad de integración y el límite de tiempo de desarrollo, que se mitigaron mediante iteraciones ágiles, validación continua y una estrecha alineación entre los requisitos formales (ERS) y la implementación.

1.6– Ecosistema de desarrollo y operación

Para asegurar la coherencia entre el diseño, la validación y el despliegue del sistema, se ha seleccionado un conjunto de tecnologías y herramientas alineado con estándares actuales de la industria:

- **Backend:** Se emplea **Java 21** como lenguaje principal con el framework **Spring Boot**. La seguridad se gestiona con **Spring Security**, el acceso a base de datos mediante **JPA/Hibernate**, el control de versiones del esquema de base de datos a través de **Flyway** y la construcción con **Maven**, manteniendo el backend organizado por capas para facilitar validación y trazabilidad.

- **Frontend:** Interfaz desarrollada como *Single Page Application* (SPA) con **React** y **TypeScript**. Se usa **Vite** como empaquetador y entorno de desarrollo. Para el enrutamiento se utiliza **React Router** y para las peticiones HTTP, **Axios**. La validación de componentes se realiza con **Vitest** y pruebas de mutación mediante **Stryker**, reforzando el control de calidad sobre componentes, contextos y servicios de cliente.
- **Integración Continua y Despliegue (CI/CD):** El código fuente se versiona en GitHub, empleando *workflows* de GitHub Actions para ejecución automatizada de compilación, *linting* y pruebas. Las versiones candidatas se despliegan automáticamente en la plataforma como servicio (PaaS) **Render**, utilizando contenedores Docker para el backend y sitios estáticos para el frontend, junto a una base de datos gestionada PostgreSQL. Este flujo facilita una entrega reproducible, auditable y alineada con los sprints del proyecto.
- **Documentación y Modelado:** La especificación de requisitos y análisis se realizó con la herramienta **Proteus**. Esta memoria y la documentación técnica han sido elaboradas en **L^AT_EX**. Adicionalmente, el proyecto hace un uso transparente y guiado de asistentes de Inteligencia Artificial para agilizar tareas de redacción y refinamiento de código base, manteniendo en todo momento el criterio y autoría del equipo de ingeniería.

Análisis temporal y costes de desarrollo

2.1– Análisis temporal

2.1.1. Metodología aplicada y roles del proyecto

La ejecución del TFG se ha organizado con un enfoque **ágil adaptado a contexto académico**, tomando Scrum como marco de referencia ligero. No se replica un proceso empresarial completo, pero sí se mantienen los elementos que aportan mayor valor de control: iteraciones cortas, objetivos por sprint, revisión frecuente y replanificación basada en evidencia.

Los roles de trabajo utilizados han sido el **equipo de desarrollo**, formado por dos autores con responsabilidad compartida en backend, frontend, pruebas y documentación técnica; el **tutor del TFG**, con función de supervisión funcional y técnica para validar hitos, alcance y calidad de entregables; y los **stakeholders académicos**, como referencia de necesidades operativas reales en docencia, entrega y evaluación. Esta combinación permite equilibrar rigor técnico con alineación a uso docente.

La duración de iteración se fijó en dos semanas, con entregables funcionales incrementales y una revisión de cierre por sprint.

2.1.2. Herramientas de gestión y seguimiento

La gestión del trabajo se ha apoyado en artefactos versionados del propio repositorio y en herramientas de planificación integradas con el flujo de desarrollo. En concreto, se utilizaron **Product Backlog** y **Sprint Backlog** para priorización y control de alcance, **GitHub Issues/Projects** para trazabilidad de tareas y decisiones de ejecución, y un **registro horario por sprint** que contrasta capacidad prevista con dedicación real. Esta triangulación permite justificar desviaciones de forma auditable.

Este esquema permite justificar desviaciones de forma auditable, conectando horas, funcionalidades y evidencia de validación.

2.1.3. Marco de planificación

La planificación temporal del proyecto se ha definido siguiendo un enfoque incremental basado en *sprints* quincenales, con revisiones periódicas del alcance. El objetivo de este enfoque no es únicamente repartir tareas, sino reducir riesgo técnico de forma temprana y asegurar que cada incremento entregue valor funcional verificable.

El marco de trabajo adoptado combina planificación por objetivos (cada sprint se orienta a un bloque funcional medible), control continuo con seguimiento semanal de avance, calidad y

bloqueos, y replanificación explícita cuando cambian supuestos. Esta mezcla aporta disciplina sin perder capacidad de ajuste.

2.1.4. Hipótesis de capacidad y calendario base

Los supuestos de capacidad empleados para el plan temporal fueron un equipo de desarrollo de **2 personas**, con dedicación estimada de **30 horas por persona y semana**, lo que fija una capacidad total semanal de **60 horas**. Se definieron sprints de **2 semanas**, con una capacidad por sprint de **120 horas**. Estos supuestos hacen explícita la base del calendario y permiten medir desviaciones.

Sobre esa base, el proyecto se estructuró en **6 iteraciones** (Sprint 0 + Sprint 1 a Sprint 5), con un esfuerzo total planificado de **660 horas**. Se añadió un Sprint 5 de cierre, con menor carga (60 h) y calendario extendido hasta el 12 de mayo para revisión final, corrección de incidencias y ajuste documental.

2.1.5. Planificación macro por sprints

La Tabla 2.1 resume la planificación temporal de alto nivel.

Sprint	Periodo	Horas	Objetivo principal
S0	Sep 2025 - 22 Feb 2026	120	Documentación y análisis inicial: ERS, DAS, casos de uso, modelo de datos y backlog.
S1	17 Feb - 2 Mar 2026	120	Infraestructura base, arquitectura backend/frontend, autenticación, módulos núcleo y despliegue inicial.
S2	3 Mar - 16 Mar 2026	120	Funcionalidad de entregables y entregas, mejora del flujo de estudiante/-profesor e integraciones previstas.
S3	17 Mar - 30 Mar 2026	120	Evaluación y feedback, mejoras de usabilidad para profesorado, refuerzo de pruebas e inicio formal de memoria.
S4	31 Mar - 13 Abr 2026	120	Cierre técnico, pruebas finales, documentación completa, pulido del producto y preparación de defensa.
S5	14 Abr - 12 May 2026	60	Revisión final, corrección de incidencias, ajustes de memoria y cierre documental.

Tabla 2.1: Planificación temporal macro del proyecto

2.1.6. Planificación micro y distribución del esfuerzo

Además de la visión macro, cada sprint se descompuso en líneas de trabajo concretas con asignación horaria por responsable. El criterio aplicado fue identificar primero las tareas críticas de MVP, reservar capacidad para pruebas en cada iteración y mantener un *buffer* explícito para incidencias. Esta pauta reduce el riesgo de concentrar la validación en la fase final.

En términos acumulados, la distribución de esfuerzo se ha orientado a construir la base arquitectónica y de seguridad en fases tempranas, desarrollar funcionalidad de negocio en fases intermedias y concentrar validación de calidad, documentación y cierre en la fase final. Esta secuencia protege la estabilidad del sistema a medida que crece el alcance.

Esta distribución evita concentrar el testing al final y reduce el riesgo de deuda técnica de cierre, un problema habitual en proyectos con calendario académico estricto.

2.1.7. Hitos de control del proyecto

Para asegurar gobernanza temporal se definieron hitos de control: **H1** (cierre de requisitos y arquitectura con ERS/DAS validados), **H2** (disponibilidad de la base técnica del producto en backend, frontend y seguridad), **H3** (primer flujo operativo extremo a extremo actividad-entrega), **H4** (despliegue operativo y pipelines CI/CD estables), **H5** (incorporación del módulo de evaluación y feedback), **H6** (estabilización de calidad con pruebas automáticas y mutación), **H7** (cierre de memoria técnica y validación documental) y **H8** (preparación de defensa y cierre final del proyecto). Esta secuencia introduce puntos de control verificables en momentos críticos.

2.1.8. Seguimiento del avance y métricas de control

El seguimiento temporal se realizó con métricas sencillas: horas planificadas frente a horas completadas por sprint, porcentaje acumulado de ejecución respecto a las 660 horas totales, velocidad de entrega por iteración (PBIs o tareas completadas) e indicadores de calidad como condición de cierre (tests en verde, cobertura y estabilidad de pipelines). Estas métricas conectan avance, alcance y calidad en una misma lectura.

Como referencia de control acumulado, tras S0 se ejecutaron 120 h (18 %), tras S1 se alcanzaron 240 h (36 %), el objetivo tras S2 fue 360 h (55 %), tras S3 480 h (73 %), tras S4 600 h (91 %) y tras S5 660 h (100 %). Esta trayectoria permite comprobar si la ejecución se mantiene alineada con el plan.

2.1.9. Control de desviaciones con valor ganado simplificado

Para evaluar el progreso del proyecto de forma objetiva, se ha utilizado una adaptación del método de análisis de valor ganado (EVM) acorde al alcance de un trabajo académico.

Se consideran tres magnitudes: **PV** (*Planned Value*), como horas planificadas acumuladas; **EV** (*Earned Value*), como horas equivalentes al trabajo realmente completado; y **AC** (*Actual Cost*), como horas reales consumidas. Esta base permite interpretar desviaciones de manera objetiva.

Los indicadores de interpretación utilizados son:

$$SPI = \frac{EV}{PV} \quad CPI = \frac{EV}{AC}$$

donde un valor cercano a 1 indica equilibrio entre plan y ejecución.

Corte	PV (h)	EV (h)	AC (h)	SPI	CPI
Fin S1	240	236	242	0.98	0.98
Fin S2	360	354	366	0.98	0.97
Fin S3	480	474	486	0.99	0.98
Fin S4 (objetivo)	600	600	600	1.00	1.00
Fin S5 (objetivo)	660	660	660	1.00	1.00

Tabla 2.2: Seguimiento temporal con valor ganado simplificado

La lectura de estos valores indica desviaciones controladas, sin deriva grave de calendario, lo que valida la eficacia de la replanificación incremental.

2.1.10. Desviaciones y replanificación

El plan no permaneció estático durante todo el proyecto. Se registraron revisiones de planificación con trazabilidad de versiones, destacando el ajuste del modelo de capacidad (de una planificación inicial más fragmentada a una estructura de sprints de 120 h), la recalibración de fechas de S1-S4 para alinear ejecución técnica y calendario académico, y la incorporación de tareas documentales en S3 para evitar cuello de botella en S4. Estas revisiones preservan el objetivo global sin perder consistencia temporal.

La replanificación se adoptó con criterio de continuidad: no alteró el objetivo global de 660 h, pero mejoró la probabilidad de cumplimiento de hitos y redujo el riesgo de cierre tardío.

2.1.11. Lecciones aprendidas de planificación

Del seguimiento temporal se extraen tres lecciones principales. Iniciar testing temprano reduce el trabajo repetido porque distribuir pruebas por sprint evita acumulación de defectos al final. Documentar en paralelo reduce riesgo de cierre, ya que incorporar memoria desde S3 disminuye el cuello de botella de la fase final. Planificar un *buffer* explícito mejora estabilidad al absorber incidencias sin romper hitos críticos.

2.1.12. Riesgos temporales y mitigación

Durante la planificación se identificaron riesgos de calendario relevantes. La Tabla 2.3 resume los principales.

Riesgo	Prob.	Impacto	Mitigación
Complejidad UX en panel docente de evaluación	Media	Alto	Prototipado temprano y validación incremental por sprint.
Sobrecarga de redacción de memoria en fase final	Alta	Alto	Inicio de memoria en S3 y redacción paralela al desarrollo.
Incidencias de integración frontend-backend	Media	Medio	Pruebas de integración continuas y criterios de cierre por calidad.
Variabilidad de disponibilidad semanal del equipo	Media	Alto	Reserva de buffer de contingencia por iteración.

Tabla 2.3: Riesgos temporales y acciones de mitigación

2.1.13. Conclusión temporal

Desde el punto de vista temporal, la estrategia adoptada combina rigor de planificación y adaptabilidad operativa. La descomposición por sprints, el seguimiento con métricas de avance y la replanificación controlada permiten sostener una ejecución realista, alineada con los requisitos del TFG y con las restricciones del calendario académico.

2.2– Costes de desarrollo

2.2.1. Criterio de estimación económica

El análisis de costes se ha realizado con enfoque de **coste de esfuerzo**. La unidad base de cálculo es la hora de trabajo invertida por el equipo.

Se distinguen dos escenarios para evitar una lectura sesgada: el **Escenario A** (referencia académica) con tarifa de 15 EUR/h y el **Escenario B** (referencia profesional junior) con tarifa de 25 EUR/h. Esta doble referencia permite contextualizar el coste según perfiles de mercado diferentes.

La duración total considerada es de 660 horas.

2.2.2. Coste de personal

El coste principal del proyecto es el esfuerzo humano. Se calcula como:

$$C_{personal} = H_{total} \times T_{hora}$$

donde $H_{total} = 660$ horas y T_{hora} es la tarifa horaria del escenario.

Escenario	Tarifa	Coste personal total
Escenario A (académico)	15 EUR/h	9 900 EUR
Escenario B (junior mercado)	25 EUR/h	16 500 EUR

Tabla 2.4: Coste de personal para 660 horas de proyecto

2.2.3. Coste de personal por iteración

Dado que los sprints regulares están planificados en 120 horas y el sprint de cierre en 60 horas, el coste por iteración regular es:

$$C_{sprint} = 120 \times T_{hora}$$

Para el sprint de cierre:

$$C_{sprint_cierre} = 60 \times T_{hora}$$

Sprint	Horas	Coste A (15 EUR/h)	Coste B (25 EUR/h)
S0	120	1 800 EUR	3 000 EUR
S1	120	1 800 EUR	3 000 EUR
S2	120	1 800 EUR	3 000 EUR
S3	120	1 800 EUR	3 000 EUR
S4	120	1 800 EUR	3 000 EUR
S5	60	900 EUR	1 500 EUR
Total	660	9 900 EUR	16 500 EUR

Tabla 2.5: Coste por sprint según escenario de tarifa

2.2.4. Coste asociado a aseguramiento de calidad

La planificación incorpora tareas de testing en varias iteraciones (unitarias, integración, E2E y mutación). Tomando como referencia las horas de testing explícitamente planificadas en S1-S4, el esfuerzo de calidad asciende a:

$$H_{QA} = 10 + 26 + 14 + 24 = 74 \text{ h}$$

Esto representa un peso relativo de:

$$\frac{74}{660} \times 100 = 11,21 \%$$

del esfuerzo total del proyecto, porcentaje coherente con un producto que prioriza la estabilidad y calidad del software.

Concepto	Horas	Coste A	Coste B
Esfuerzo QA acumulado (S1-S4)	74	1 110 EUR	1 850 EUR

Tabla 2.6: Coste estimado del aseguramiento de calidad

2.2.5. Costes de infraestructura y operación

Durante el desarrollo académico se han utilizado, de forma predominante, planes gratuitos o créditos de estudiante para servicios de integración, despliegue y repositorio. Por tanto, el coste operativo **real observado** ha sido próximo a 0 EUR en la etapa de desarrollo.

No obstante, para ofrecer una visión realista, se incluye una estimación orientativa de operación mínima en un entorno no gratuito durante 4 meses de proyecto (basada en las tarifas del proveedor PaaS Render):

Concepto	Coste real (proyecto)	Coste orientativo (4 meses)
Hosting backend	0 EUR	28 EUR
Base de datos gestionada	0 EUR	80 EUR
Servicios auxiliares de almacenamiento/transferencia	0 EUR	20 EUR
Dominio y gestión básica	0 EUR	4 EUR
Total operación	0 EUR	132 EUR

Tabla 2.7: Comparativa entre coste real y coste operativo orientativo

2.2.6. Coste total del proyecto y escenarios de sensibilidad

El coste total se modela como la suma del coste de personal y el coste operativo:

$$C_{total} = C_{personal} + C_{operacion}$$

Para ofrecer una visión realista de la viabilidad del proyecto en distintos contextos, se plantea un análisis de sensibilidad con tres escenarios de tarifas, manteniendo las 660 horas como constante de alcance. Se compara el coste asumiendo que no hay costes de operación (como ha ocurrido durante el desarrollo) frente a un escenario con operación orientativa de pago.

Escenario (Tarifa)	Coste personal	Total (Sin costes de operación)	Total (Con operación orientativa)
Académico (15 EUR/h)	9 900 EUR	9 900 EUR	10 032 EUR
Junior mercado (25 EUR/h)	16 500 EUR	16 500 EUR	16 632 EUR
Perfil intermedio (35 EUR/h)	23 100 EUR	23 100 EUR	23 232 EUR
Perfil senior (45 EUR/h)	29 700 EUR	29 700 EUR	29 832 EUR

Tabla 2.8: Coste total estimado del proyecto según escenario de tarifa

Este análisis confirma que, incluso con incrementos de tarifa, la estructura modular y la automatización de calidad mantienen la viabilidad económica relativa del sistema frente al valor funcional entregado.

2.2.7. Interpretación económica y sostenibilidad

El análisis muestra que el coste está dominado por el esfuerzo de desarrollo, mientras que el coste operativo es bajo en comparación. Esto es coherente con la naturaleza de un producto software de tamaño medio, donde la inversión principal está en diseño, implementación, validación y documentación.

Además, la orientación modular del sistema reduce coste futuro de evolución, ya que permite incorporar nuevas funcionalidades sin rehacer componentes nucleares.

Desde la perspectiva de sostenibilidad del producto, la combinación de arquitectura desacoplada, testing automatizado y despliegue continuo reduce también el coste de mantenimiento correctivo, que suele representar una parte significativa del coste total de ciclo de vida.

2.2.8. Conclusión del análisis de costes

El coste global del proyecto resulta razonable para el alcance funcional conseguido y para el nivel de calidad exigido en un TFG con vocación de producto real. La estrategia de planificación por sprints, junto con el control de calidad continuo, ha permitido mantener equilibrio entre plazo, alcance y robustez técnica.

Análisis y requisitos

3.1– El proceso de análisis en la Ingeniería del Software

El análisis de requisitos es una etapa fundamental dentro del ciclo de vida de la Ingeniería del Software. Su propósito es definir con precisión las necesidades del dominio del problema antes de plantear una solución tecnológica. Un análisis exhaustivo previene sobrecostos, minimiza el riesgo de desarrollo de funcionalidades innecesarias y asegura que el sistema entregado se alinee con las expectativas de los usuarios.

En el contexto de este proyecto, la metodología de análisis ha partido de un estudio minucioso de la situación actual en el ámbito docente, identificando las limitaciones operativas existentes.

3.1.1. Situación actual y necesidades de negocio

El escenario de partida presenta una fricción operativa notable: creación de actividades y recogida de entregas con alto componente manual, dificultad para mantener trazabilidad uniforme y baja capacidad de evolución. Las necesidades de negocio priorizadas son reducir la carga administrativa, ofrecer al estudiante un flujo de entrega más guiado, centralizar evaluación y mejorar la mantenibilidad del sistema. Estas necesidades conectan directamente con los bloques funcionales del MVP.

3.2– Especificación de Requisitos del Sistema (ERS)

3.2.1. Fundamentación teórica del ERS

La Especificación de Requisitos del Sistema (ERS) es el documento estándar que actúa como contrato entre las necesidades del negocio y la implementación técnica. De forma genérica, su objetivo es catalogar qué debe hacer el sistema (requisitos funcionales) y bajo qué restricciones de calidad y seguridad debe operar (requisitos no funcionales y reglas de negocio).

Para este Trabajo Fin de Grado en particular, el ERS tiene como objetivo específico delimitar de forma inquebrantable qué operaciones están permitidas para los roles de profesor, estudiante y administrador, así como definir los atributos esenciales de los cursos, actividades y entregables.

3.2.2. Fuentes de requisitos

La base fundamental para la definición de los requisitos del proyecto es el propio Documento ERS (Especificación de Requisitos del Sistema), el cual constituye de por sí el catálogo formal y exhaustivo de requisitos. Este documento fue redactado, modelado estructuralmente y versionado

haciendo uso de la herramienta Proteus (de Sevilla, 2026), lo que garantiza rigor metodológico en su definición.

A partir de este catálogo central establecido en el ERS, el flujo de desarrollo se ha complementado con el Documento DAS (Documento de Análisis del Sistema) para el modelado conceptual, así como con el uso dinámico de un Product Backlog y un Sprint Backlog durante la ejecución iterativa. La interacción continua con estas herramientas, junto con la evidencia empírica obtenida en las pruebas funcionales y de integración, asegura que los requisitos no se mantengan como meras definiciones abstractas. En lugar de una especificación estática, los requisitos del ERS se priorizan, se integran en incrementos de valor y se validan con el comportamiento del producto en funcionamiento, evitando en todo momento cualquier desconexión entre la especificación teórica y la implementación real del sistema.

3.2.3. Estructura del catálogo

El catálogo se organiza en cuatro bloques complementarios: requisitos funcionales de información y operación, reglas de negocio que acotan el comportamiento permitido, requisitos no funcionales de calidad del sistema y restricciones e integraciones técnicas por entorno. Esta estructura permite cubrir tanto qué debe hacer el sistema como bajo qué condiciones debe hacerlo.

En Proteus se distinguen explícitamente requisitos funcionales (por ejemplo, RQF-001), reglas de negocio (REGN-001) y no funcionales (RQNF-001 a RQNF-011), lo que facilita su seguimiento durante la implementación.

3.2.4. Requisitos funcionales

El ERS define un abanico funcional amplio, del que se ha priorizado el núcleo necesario para el MVP. A nivel de dominio, la funcionalidad se agrupa en identidad y acceso (autenticación de usuarios y control por rol), estructura académica (cursos, grupos, actividades y vinculaciones), gestión de entregables (configuración de fechas, visibilidad, reenvíos y estructura esperada), gestión de entregas (subida, historial, descarga y trazabilidad) y evaluación y feedback (calificación, comentarios y publicación de notas). Esta agrupación garantiza cobertura del ciclo académico completo.

Como ejemplos representativos se incluyen **RQF-001** (creación de usuarios por administrador), **RQF-014–RQF-017** (operaciones de entrega y versionado) y **RQF-018–RQF-025** (evaluación, visibilidad e integraciones). Estos ejemplos reflejan la amplitud funcional sin agotar el inventario completo.

3.2.5. Reglas de negocio

Las reglas de negocio restringen de forma explícita qué acciones son admisibles. Una regla clave es **REGN-001**, que establece que solo el profesor autorizado puede asignar una calificación.

Esta norma se implementa en validaciones de servicio y controlador, evitando tanto la elevación de privilegios como la evaluación fuera de contexto (p. ej., docente no vinculado al grupo o actividad).

3.2.6. Requisitos no funcionales

Los requisitos no funcionales aportan condiciones de calidad del producto. La Tabla 3.1 resume los más relevantes para el alcance actual.

Código	Categoría	Aplicación en el sistema
RQNF-001	Fiabilidad	Mitigar pérdida de datos en operaciones críticas de entrega/evaluación.
RQNF-002	Usabilidad	Flujos entendibles para usuarios no técnicos, con navegación guiada por rol.
RQNF-004	Mantenibilidad	Bajo acoplamiento entre capas y separación estricta de responsabilidades.
RQNF-006	Eficiencia	Respuesta ágil en consultas frecuentes de panel, actividad y entregables.
RQNF-008	Portabilidad	Compatibilidad web en navegadores modernos de uso general.
RQNF-009	Seguridad	Exposición de datos condicionada por autenticación y autorización.

Tabla 3.1: Requisitos no funcionales representativos

3.2.7. Priorización y alcance

La priorización se realizó con criterio MoSCoW y validación continua con el backlog. El foco del MVP se situó en los procesos con mayor impacto docente: gestión de actividades y entregables, flujo completo de entrega y evaluación, seguridad por rol, y persistencia con trazabilidad de cambios. Esta selección maximiza valor operativo en el horizonte temporal disponible.

Funcionalidades avanzadas (integraciones extendidas, autenticación federada completa, observabilidad más profunda) se mantienen en backlog de evolución y no comprometen la viabilidad del alcance de TFG.

3.2.8. Trazabilidad requisito-backlog-código

La trazabilidad se gestiona en tres niveles estrechamente interconectados: en primer lugar, un **nivel de análisis** apoyado por el identificador unívoco establecido en el ERS a través de Proteus; en segundo lugar, un **nivel de planificación** guiado por los PBI (*Product Backlog Items*, elementos de trabajo discretos que representan funcionalidades con valor para el usuario dentro de las metodologías ágiles) y su sprint asociado; y por último, un **nivel de ejecución** que liga estos elementos al módulo de software implementado y a la prueba automatizada que lo valida. Esta triple vista permite justificar cualquier decisión de diseño con evidencia técnica medible.

La Tabla 3.2 muestra ejemplos representativos de esta trazabilidad, ilustrando cómo un requisito formal del ERS se convierte en un PBI dentro de un sprint de desarrollo y, finalmente, se materializa en código y pruebas específicas.

Req.	PBI / Sprint	Implementación principal	Validación
RQF-001	Usuarios y roles / S1-S2	AuthController, Usuario-Service, seguridad por rol	Tests auth + integración
RQF-018	Crear entregable / S3-S4	EntregableController, EntregableService, Entregable	Unit + API tests
RQF-023	Subir entrega / S4-S5	EntregaController, EntregaService, MaterialService	Integración + E2E
REGN-001	Evaluación docente / S5	Validaciones en servicio de entregas y evaluación	Pruebas de autorización
RQNF-009	Seguridad global / transversal	SecurityConfig, JwtAuthenticationFilter, RateLimitFilter	Tests de seguridad

Tabla 3.2: Ejemplo de trazabilidad entre requisitos, backlog e implementación

3.2.9. Contexto documental de partida (ERS y DAS)

La especificación documental de este TFG no se apoya en descripciones ad hoc, sino en dos artefactos de análisis formal mantenidos en Proteus. El **ERS (Especificación de Requisitos del Sistema)** define situación actual, necesidades de negocio, subsistemas y catálogo de requisitos (generales, de información, funcionales, no funcionales y reglas de negocio), mientras que el **DAS (Documento de Análisis del Sistema)** define arquitectura lógica, modelo de clases, diagramas de estado, diagramas de secuencia, navegabilidad e inventario de mockups de interfaz. Esta base asegura continuidad documental entre análisis y ejecución.

La reutilización de ambos documentos en esta memoria persigue dos objetivos: primero, preservar trazabilidad respecto al trabajo original de Ingeniería de Requisitos; segundo, demostrar que las decisiones de diseño e implementación adoptadas durante el TFG responden a una base de especificación explícita.

El ERS parte de un análisis detallado de un escenario académico real donde los procesos docentes presentaban una alta ineficiencia: los profesores requerían múltiples pasos manuales y plataformas dispersas para la definición de actividades, publicación de material y, de manera muy acusada, para la recogida, versionado y almacenamiento de las entregas de los alumnos. El rastreo de estas entregas carecía de una trazabilidad uniforme (fechas, reenvíos, visibilidad), lo que derivaba en pérdidas de información y en un soporte técnico que imposibilitaba la integración de lógicas de evaluación más avanzadas.

Ante este diagnóstico específico, las necesidades de negocio que fundamentan este TFG se orientan a un cambio de paradigma centralizado. El objetivo es proporcionar un entorno único donde la carga administrativa del profesor se reduzca mediante flujos estandarizados (desde la creación de un *entregable* hasta la introducción de calificaciones), y donde el alumno perciba transparencia absoluta sobre sus fechas límite, el estado de sus versiones y los comentarios (feedback) recibidos. Además, era imperativo garantizar que la evaluación solo pudiera ser efectuada bajo reglas de autorización y contexto académico estrictas (vinculación estudiante-grupo-profesor).

Estas necesidades condicionan el resto de los capítulos en tres planos tangibles: la definición arquitectónica (separando estrictamente los servicios de usuarios de la gestión académica), la implementación a través de módulos enfocados a dominios específicos, y una batería de pruebas concebida para validar tanto los flujos de experiencia de usuario como el cumplimiento inquebrantable de las reglas de autorización impuestas en el negocio.

3.2.10. Descomposición en subsistemas (ERS)

El ERS descompone el sistema en cinco grandes subsistemas lógicos para organizar sus funcionalidades. La Tabla 3.3 detalla esta descomposición, la cual ha sido reutilizada como guía fundamental de diseño modular a lo largo del desarrollo del TFG.

Código	Subsistema	Responsabilidad principal
SUBS-001	SIU (Interacción con el Usuario)	Portal web y vistas por rol para alumno, profesor y administrador.
SUBS-002	SGUS (Usuarios y Seguridad)	Autenticación, autorización por rol y control de acceso contextual.
SUBS-003	SGA (Gestión Académica)	Cursos, actividades, entregables, evaluación y feedback.
SUBS-004	SPA (Procesamiento de Archivos)	Validación técnica de ficheros, organización y versionado de entregas.
SUBS-005	SPD (Persistencia de Datos)	Persistencia relacional y repositorio de archivos con trazabilidad.

Tabla 3.3: Subsistemas definidos en ERS y reutilizados en el diseño del TFG

3.2.11. Catálogo exhaustivo de requisitos del ERS

Con objeto de incorporar de forma completa el contenido de ERS, se documenta el inventario de requisitos por tipología y su interpretación en el alcance real del TFG.

Requisitos generales (RQG)

La Tabla 3.4 detalla los requisitos generales del sistema, que establecen el marco de operación básico para la gestión de usuarios, actividades, entregables y la comunicación.

Código	Nombre	Descripción sintética
RQG-001	Gestión de usuarios y roles	Alta de usuarios con rol (administrador, profesor, alumno) y permisos diferenciados por perfil.
RQG-002	Gestión de actividades y entregables	Modelado de actividades evaluables/no evaluables y entregables con reglas propias.
RQG-003	Acceso de alumnos a entregables	Acceso de estudiante a entregables visibles y registro temporal de acciones.
RQG-004	Gestión de versiones y reenvíos	Control de versiones de entrega y soporte de reenvío cuando el docente lo habilita.
RQG-005	Configuración de actividades y entregables	Definición de fechas, formatos y condiciones de publicación por contexto académico.
RQG-006	Comunicación y feedback	Canal de comunicación docente-estudiante vinculado a la evaluación de entregas.
RQG-007	Seguridad y control de acceso	Restricción de visibilidad/operación por identidad, rol y pertenencia a curso-grupo.

Tabla 3.4: Inventario de requisitos generales (RQG) en ERS

Requisitos de información (RQI)

La Tabla 3.5 describe los requisitos de información que definen qué datos esenciales deben persistirse en el sistema (actividades, entregables, usuarios, etc.).

Código	Nombre	Descripción sintética
RQI-001	Información sobre actividades	Persistencia de metadatos de actividades creadas por profesorado.
RQI-002	Información sobre entregables	Persistencia de configuración de cada entregable y su vinculación con actividad.
RQI-003	Información de usuarios	Registro de identidad, rol y datos de operación por tipo de usuario.
RQI-004	Información de cursos	Persistencia de estructura académica (curso/grupo y relaciones).
RQI-005	Información de feedback	Persistencia de calificaciones, comentarios y estado de publicación.

Tabla 3.5: Inventario de requisitos de información (RQI) en ERS

Requisitos funcionales (RQF)

La Tabla 3.6 y la Tabla 3.7 recogen, de manera estructurada y en dos bloques para facilitar su lectura, las capacidades operativas que el sistema debe ofrecer al usuario final, desde la administración de identidades hasta la integración cloud.

Código	Nombre	Descripción sintética
RQF-001	Crear usuarios	Alta de usuarios por administrador con asignación de rol.
RQF-002	Iniciar sesión	Autenticación de usuarios en la plataforma.
RQF-003	Editar usuarios	Modificación de datos de usuario según permisos.
RQF-004	Eliminar usuarios	Baja de usuarios por perfil administrador.
RQF-005	Crear actividades	Creación de actividad académica por docente.
RQF-006	Visibilizar actividades	Publicación/ocultación de actividades para estudiantes.
RQF-007	Proporcionar adjuntos	Asociación de material de apoyo a actividad/entregable.
RQF-008	Modificar actividades	Edición de configuración de actividad por docente propietario.
RQF-009	Eliminar actividades	Eliminación de actividad según reglas de propiedad.
RQF-010	Ver actividades	Consulta de actividades por rol y contexto académico.
RQF-011	Crear entregables	Alta de entregables dentro de actividad.
RQF-012	Modificar entregables	Edición de entregable por docente autorizado.

Tabla 3.6: Inventario de requisitos funcionales (RQF-001 a RQF-012)

Código	Nombre	Descripción sintética
RQF-013	Eliminar entregable	Baja de entregable por docente autorizado.
RQF-014	Realizar entrega	Subida de archivo por estudiante para un entregable habilitado.
RQF-015	Enviar actividad	Confirmación de envío de actividad completa.
RQF-016	Versionar entregables	Mantenimiento de historial de versiones por entregable.
RQF-017	Versionar actividades	Mantenimiento de versiones de resolución de actividad.
RQF-018	Dar feedback	Publicación de comentarios docentes sobre entrega/actividad.
RQF-019	Realizar calificación	Asignación de nota por profesor autorizado.
RQF-020	Ver entregables	Consulta de entregas y versiones por rol permitido.
RQF-021	Crear curso	Alta de curso/asignatura por administración.
RQF-022	Modificar curso	Edición de datos de curso por administración.
RQF-023	Vista de estudiante	Visualización por profesor de la experiencia de alumno.
RQF-024	Visibilizar entregable	Publicación/ocultación de entregables concretos.
RQF-025	Integración cloud	Uso de almacenamiento en nube según configuración de entorno.

Tabla 3.7: Inventario de requisitos funcionales (RQF-013 a RQF-025)

Reglas de negocio (REGN)

La Tabla 3.8 agrupa las restricciones ineludibles que condicionan las operaciones del sistema, definiendo estrictamente quién cuenta con los privilegios para ejecutar cada acción, bajo qué contexto académico particular y respetando qué condiciones temporales o de visibilidad.

Código	Nombre	Regla de negocio
REGN-001	Corrección de entregables	Solo el profesor asignado al entregable puede emitir nota.
REGN-002	Visibilidad de entregables	El alumno solo accede a sus propias entregas/versiones.
REGN-003	Corrección de actividades	Solo el profesor asignado a la actividad puede calificar.
REGN-004	Visibilidad de actividades	El alumno solo visualiza sus propias resoluciones de actividad.
REGN-005	Fecha límite de entrega	No se admiten envíos fuera de plazo salvo configuración explícita.
REGN-006	Recursos no visibles	Se deniega interacción del alumno con elementos ocultos.
REGN-007	Creación de recursos	Solo profesorado crea actividades y entregables.
REGN-008	Emisión de feedback	Solo el docente autorizado publica feedback evaluativo.

Tabla 3.8: Inventario de reglas de negocio (REGN) en ERS

Requisitos no funcionales (RQNF)

La Tabla 3.9 resume las condiciones de calidad técnica y operacionales que debe cumplir el sistema para garantizar una ejecución fiable, mantenible y segura, más allá de la estricta funcionalidad visible por el usuario.

Código	Nombre	Condición de calidad
RQNF-001	Tolerancia a fallos	Persistencia local temporal ante fallos de conectividad para evitar pérdida de datos.
RQNF-002	Intuitividad	Interfaz comprensible para nuevos usuarios con patrones de uso habituales.
RQNF-003	Uniformidad	Coherencia visual y de estilo con directrices de marca institucional.
RQNF-004	Acoplamiento bajo	Separación de responsabilidades con dependencias estrictamente necesarias.
RQNF-005	Cohesión alta	Organización interna de módulos orientada a mantenimiento evolutivo.
RQNF-006	Tiempo de carga	Tiempo de transición objetivo inferior a 2 segundos en condiciones óptimas.
RQNF-007	Tiempo de búsqueda	Respuesta rápida en consultas frecuentes y minimización de búsquedas redundantes.
RQNF-008	Compatibilidad de navegadores	Soporte para navegadores modernos de uso común.
RQNF-009	Exposición de información	Visualización de datos restringida a usuarios autenticados y autorizados.
RQNF-010	Encriptación	Protección criptográfica de información personal/sensible.
RQNF-011	Portabilidad web	Compatibilidad explícita con Chrome, Firefox y Edge en escritorio/móvil.

Tabla 3.9: Inventario de requisitos no funcionales (RQNF) en ERS

3.3– Documento de Arquitectura de Software (DAS)

3.3.1. Fundamentación teórica del DAS

El Documento de Arquitectura de Software o Documento de Análisis del Sistema (DAS) constituye el puente entre la especificación teórica y la construcción real. Genéricamente, su finalidad es modelar los componentes lógicos, la navegabilidad, la interacción entre clases y los estados de las entidades principales.

En el marco de este proyecto, el objetivo específico del DAS es establecer el mapa visual que justifique la arquitectura cliente-servidor adoptada, las secuencias transaccionales para la entrega y evaluación, y los mockups de interfaz que guían la experiencia de usuario final.

3.3.2. Reutilización de diagramas y mockups del DAS

Para ilustrar la arquitectura y la interacción del usuario con el sistema, se incluyen a continuación los diagramas y mockups principales definidos en el DAS.

Artefacto DAS	Archivo original en assets	Uso en memoria
Arquitectura lógica	ArquitecturaTFG.drawio.png	Diseño global
Diagrama de clases	Diagrama de clases.png	Modelo de dominio
Diagrama de estado	Diagrama de estado.png	Ciclo de vida
Navegabilidad	Diagrama de navegabilidad (1).png	Flujo de interfaz
Secuencia autenticación	Autenticacion y Acceso.png	Flujo de acceso
Secuencia crear actividad	Backend Activity Creation-2026-03-18-161459.png	Flujo docente
Secuencia realizar entrega	Realizar Entrega.png	Flujo estudiante
Secuencia evaluación	Evaluacion y Feedback.png	Flujo de corrección

Tabla 3.10: Diagramas principales reutilizados desde DAS

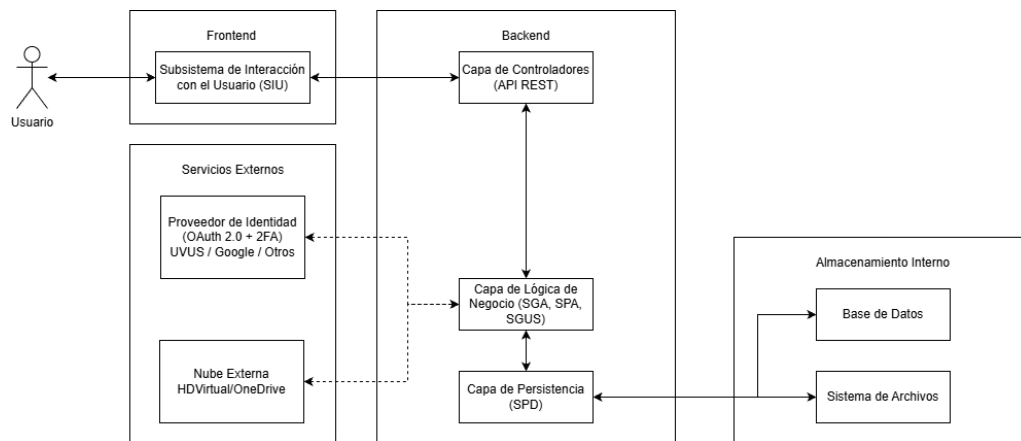


Figura 3.1: Arquitectura lógica del sistema (DAS)

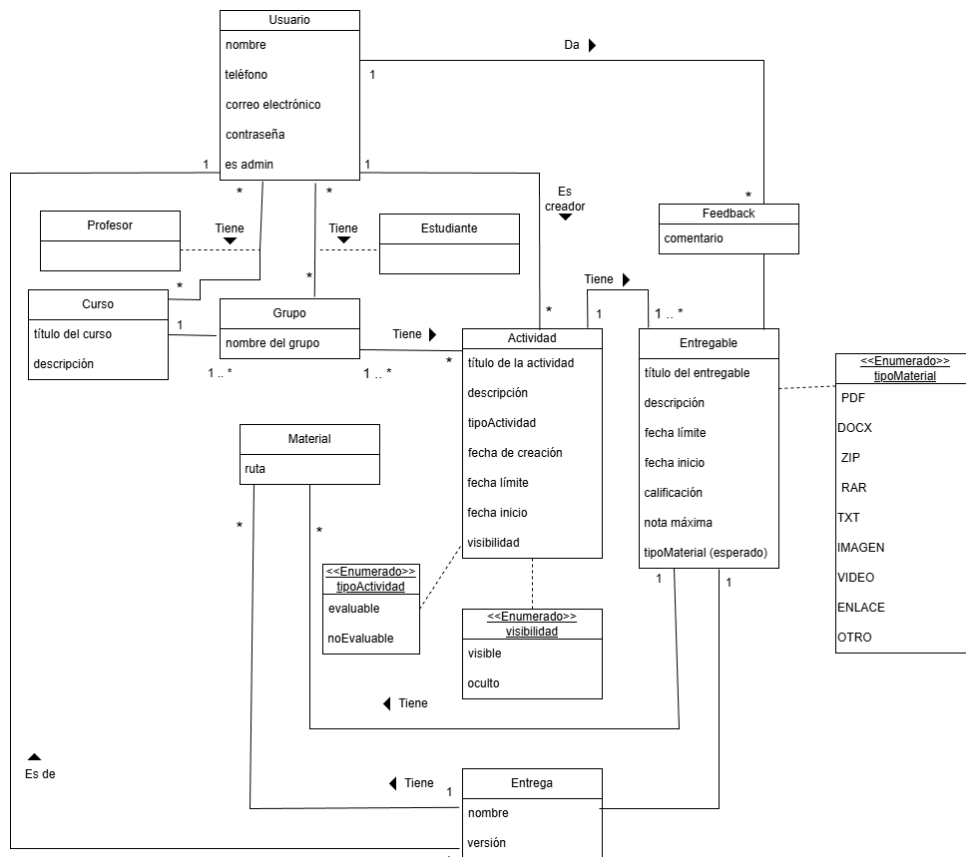


Figura 3.2: Modelo de clases del sistema (DAS)

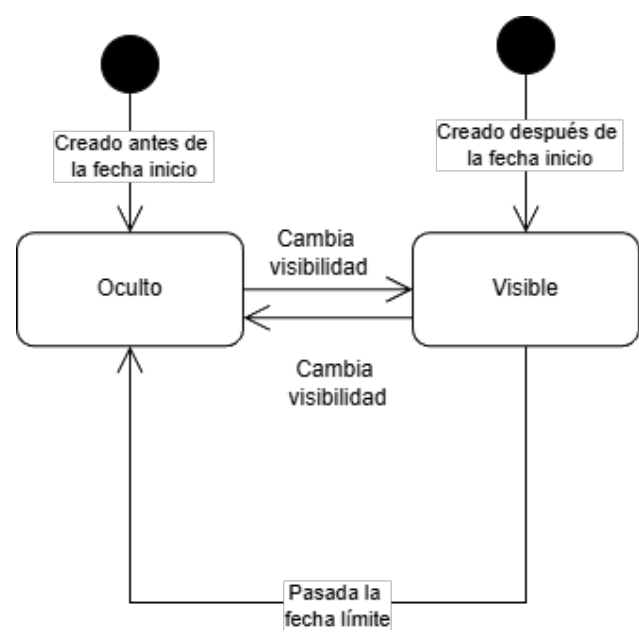


Figura 3.3: Diagrama de estado de actividades (DAS)

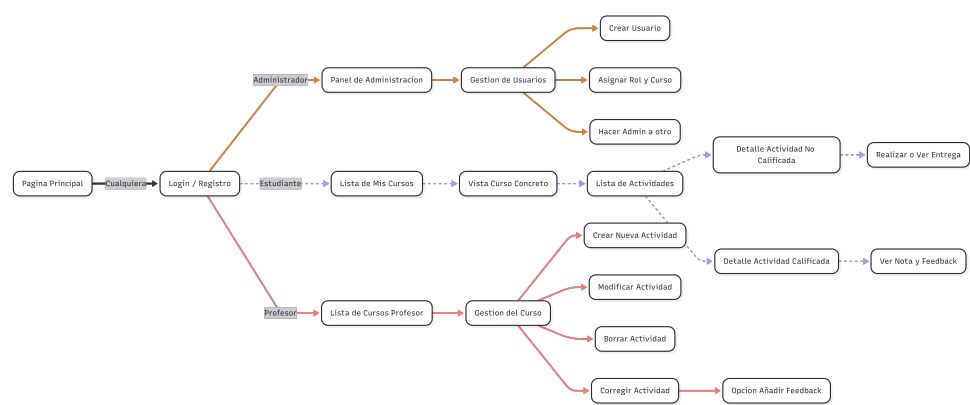


Figura 3.4: Navegabilidad principal de la interfaz (DAS)

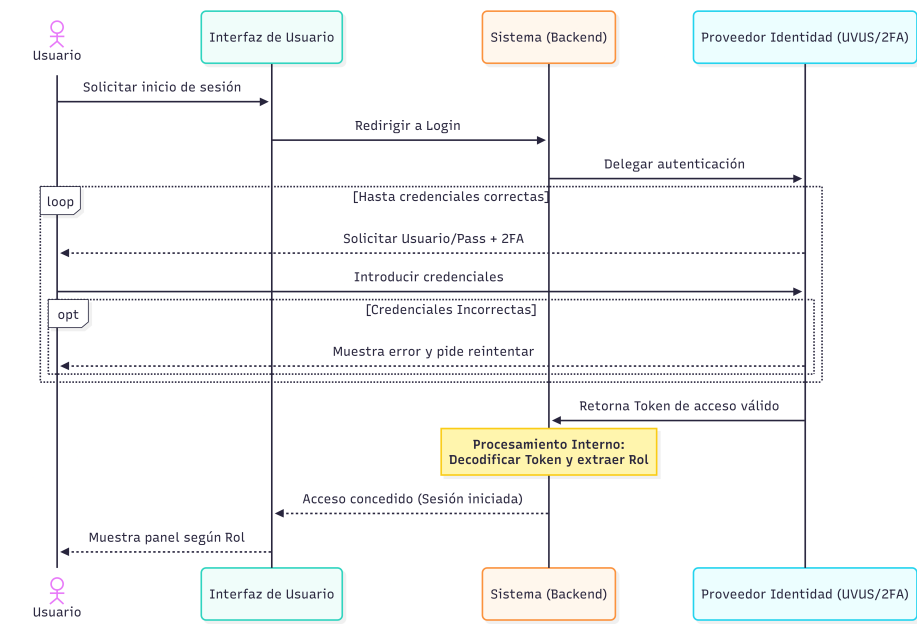


Figura 3.5: Secuencia de autenticación y acceso (DAS)

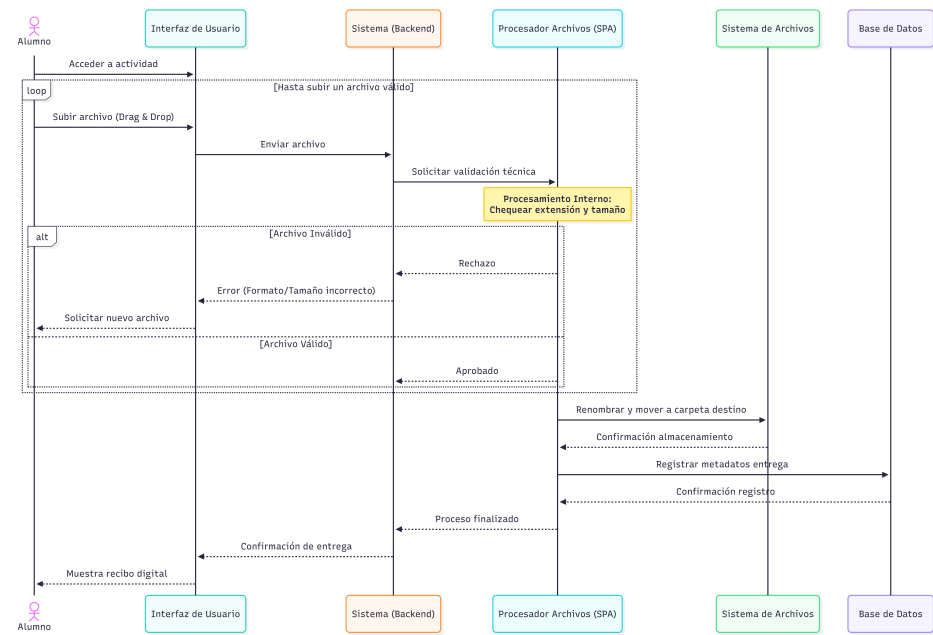


Figura 3.6: Secuencia de realización de entrega (DAS)

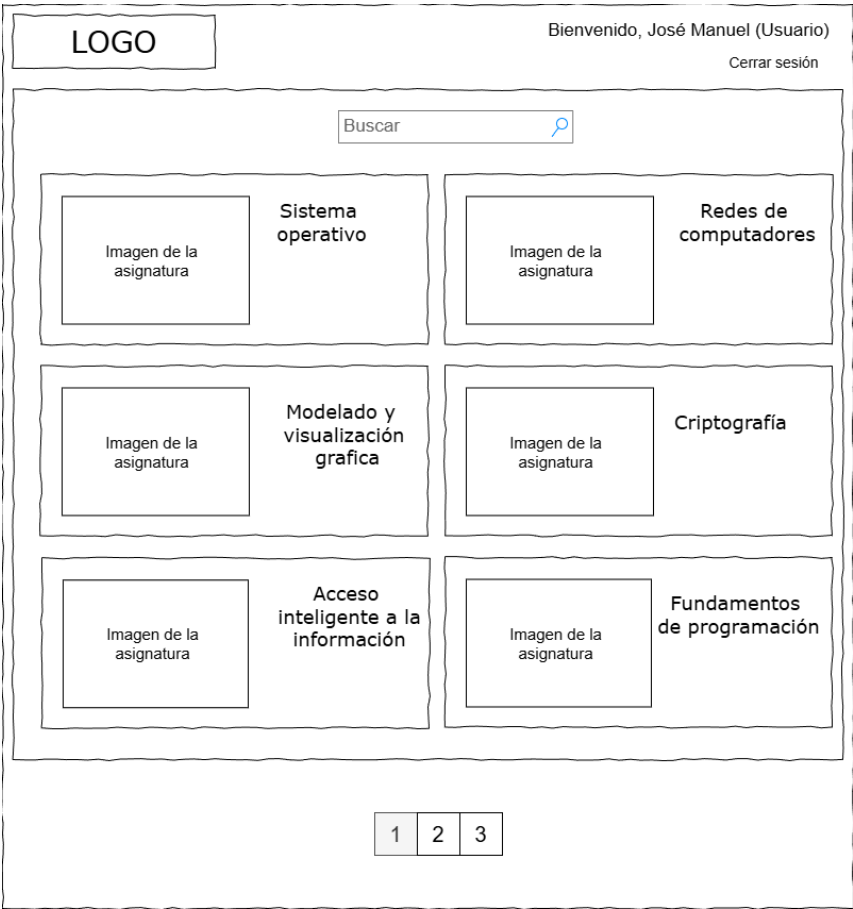


Figura 3.7: Mockup de panel principal de estudiante (DAS)

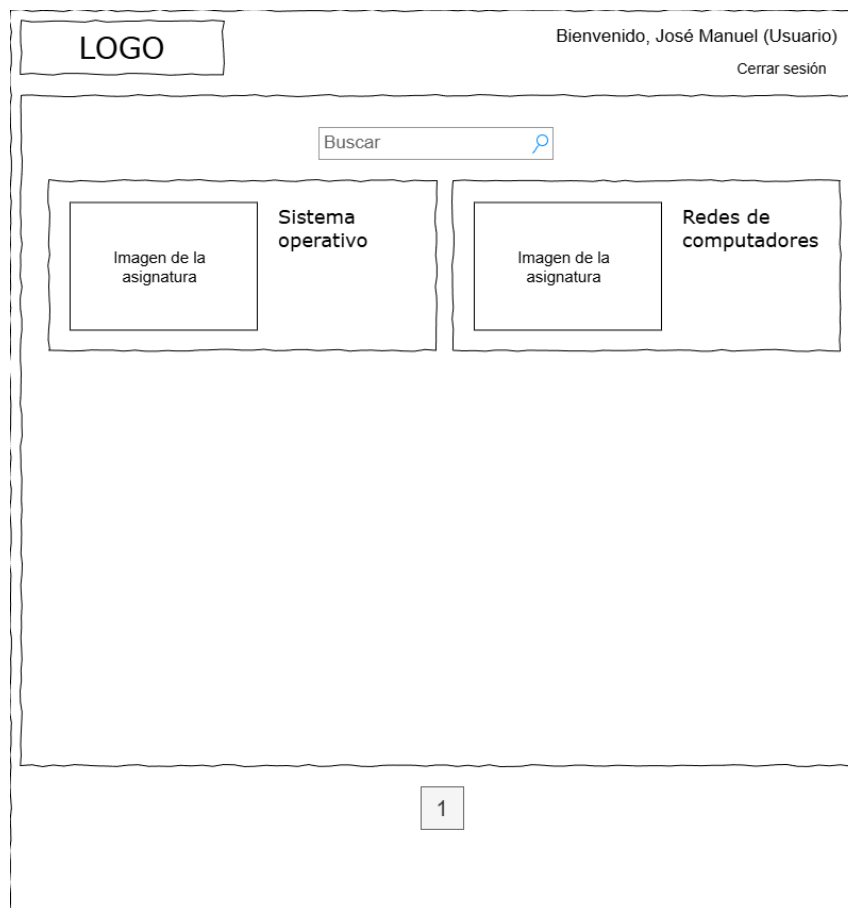


Figura 3.8: Mockup de panel principal de profesor (DAS)

Adicionalmente, el DAS incluye un inventario de dieciséis mockups que cubre las operaciones esenciales por rol. Para mantener equilibrio entre exhaustividad y legibilidad, su trazado completo se resume en la Tabla 3.11.

Archivo de mockup	Escenario principal representado
InterfazDeUsuarioTFG-InicioDeSesión.drawio.png	Inicio de sesión y entrada al sistema.
InterfazDeUsuarioTFG-Registrarse.drawio.png	Registro o alta inicial de usuario.
InterfazDeUsuarioTFG-LogueadoComoEstudiante.png	Panel inicial de estudiante autenticado.
VistaEstudianteDentroDeUna-Asignatura.png	Vista de asignatura desde rol estudiante.
LogueadoComoEstudiante-SeleccionandoCheckbox.png	Selección de entregables por estudiante.
LogueadoComoEstudianteEn-Calificaciones.png	Consulta de calificaciones y feedback.
LogueadoComoProfesor.png	Panel inicial de profesor autenticado.
VistaProfesorEnAsignatura.png	Vista de asignatura desde rol profesor.
Creacion de actividad.png	Creación de actividad académica.
Edición de actividad.png	Edición de actividad existente.
Realizar calificaciones.png	Proceso de calificación por docente.
DentroDeCalificacionComo-Profesor.png	Vista detallada de corrección.
EntregaSinNadaPuesto.png	Estado inicial de entrega sin adjuntos.
EntregaConAlgoPuesto.png	Estado de entrega con archivo adjunto.
PonerComentarios.png	Edición de comentarios de evaluación.
PaginaEstudiantesProfesores.png	Consulta de estudiantes y trazabilidad docente.

Tabla 3.11: Inventario de mockups de interfaz reutilizados desde DAS

3.3.3. Lectura técnica de los diagramas estructurales del DAS

La reutilización de las Figuras 3.1, 3.2, 3.3 y 3.4 no se limita a un uso ilustrativo, sino que establece una línea de diseño que condiciona decisiones de implementación en backend, frontend y datos.

En particular, la arquitectura lógica permite justificar la separación entre interacción con usuario, seguridad, gestión académica, procesamiento de archivos y persistencia. Este reparto reduce acoplamiento transversal, facilita pruebas por componente y mejora mantenibilidad evolutiva.

El modelo de clases se emplea como referencia para mantener coherencia entre entidades persistentes, DTOs y contratos de API. Del mismo modo, el diagrama de estado de actividades se refleja en validaciones de servicio que evitan transiciones no permitidas (por ejemplo, publicar, ocultar o cerrar fuera de las reglas definidas).

Por último, el diagrama de navegabilidad refuerza la consistencia entre estructura funcional y experiencia de usuario por rol, evitando rutas incoherentes con las restricciones de autorización descritas en ERS.

Artefacto DAS	Decisión de diseño derivada	Materialización técnica	Requisitos ERS reforzados
Arquitectura lógica	Separación funcional en subsistemas	Frontend SPA + API REST + servicios por dominio + repositorios	SUBS-001 a SUBS-005, RQNF-004
Diagrama de clases	Normalización del dominio académico	Entidades Usuario, Curso, Actividad, Entregable, Entrega y Feedback	RQI-001 a RQI-005, RQF-005 a RQF-020
Diagrama de estado	Control explícito del ciclo de vida	Validaciones de transición y reglas temporales en servicios	REGN-005, REGN-006, RQF-006, RQF-024
Navegabilidad	Flujos de interfaz por rol y contexto	Rutas protegidas, vistas diferenciadas y control de visibilidad	RQG-001, RQG-007, RQNF-002, RQNF-009

Tabla 3.12: Interpretación técnica de los diagramas estructurales del DAS

3.3.4. Flujos operativos del DAS y su encaje en la solución

Además de los diagramas estructurales, el DAS aporta una visión de comportamiento detallada mediante secuencias y mockups operativos. Esta capa es especialmente relevante para documentar el recorrido completo de profesor y estudiante en los procesos críticos del MVP.

El flujo de creación de actividad docente se explicita en la Figura 3.9. En ella se observa la coordinación entre interfaz, validaciones de negocio y persistencia, alineada con RQF-005, RQF-008 y RQF-011.

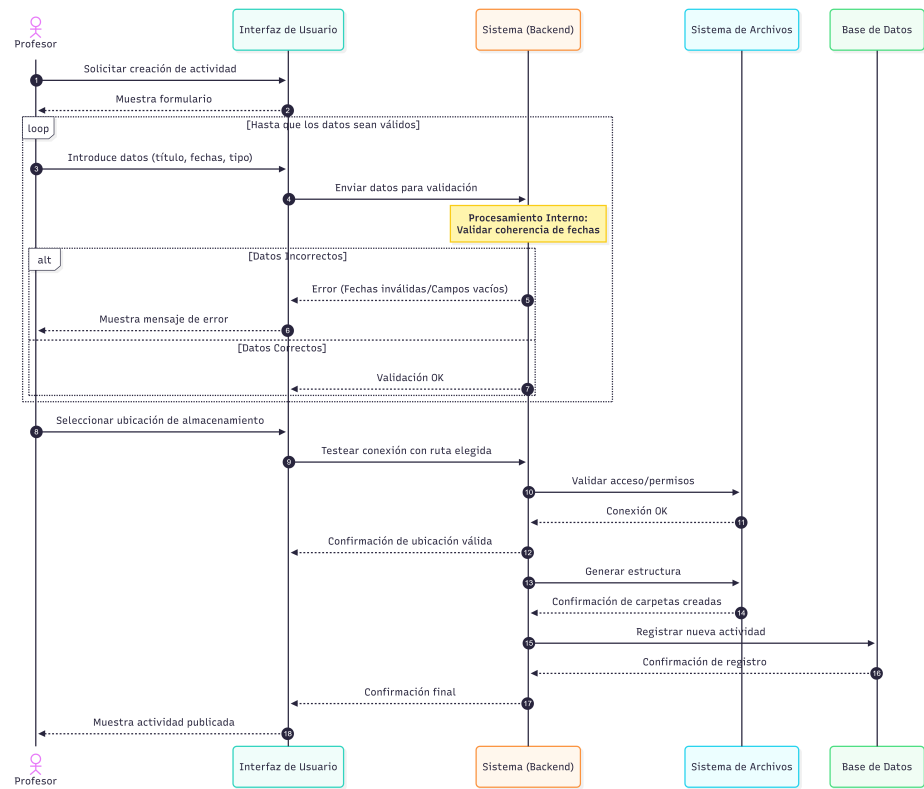


Figura 3.9: Secuencia de creación de actividad en backend (DAS)

El ciclo de evaluación y retroalimentación queda recogido en la Figura 3.10, que sirve de base para justificar la integración entre calificación, comentarios y publicación de resultados, en coherencia con RQF-018, RQF-019 y REGN-001/REGN-008.

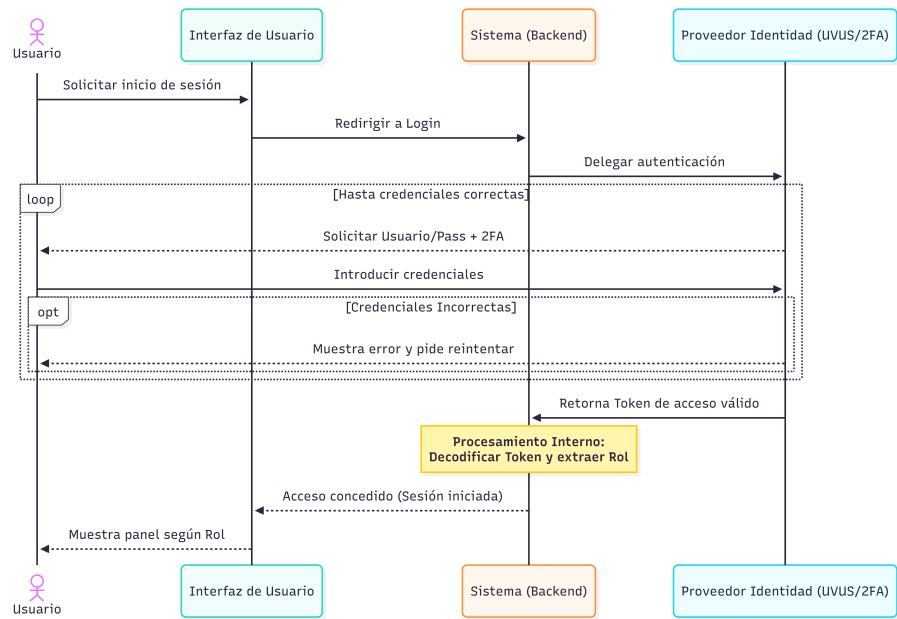


Figura 3.10: Secuencia de evaluación y feedback (DAS)

En la dimensión de interfaz, los mockups de trabajo muestran cómo se materializa la navegación por rol en escenarios reales de uso. Las Figuras 3.11 y 3.12 capturan dos momentos clave del producto: seguimiento de actividad por el estudiante y corrección detallada por el profesorado.



Figura 3.11: Mockup operativo: estudiante dentro de una asignatura (DAS)

LOGO

Bienvenido, José Manuel (Usuario)
Cerrar sesión

Imagen de la asignatura

Sistema operativo

Prueba de comandos

Fecha de vencimiento: 7/2/2024 20:11

Detalles:

Examen para ver que conocimientos se conocen de Linux

Memoria: Memoria_VNK5300.pdf [Descargar](#)

Una función **random** (o función aleatoria) es aquella que genera resultados que no siguen un patrón fijo, sino que varían de manera impredecible dentro de un rango definido.

En programación, suele referirse a funciones que devuelven **números pseudoaleatorios**, es decir, valores que parecen aleatorios pero en realidad son generados por un algoritmo matemático a partir de una "semilla" inicial.

Sin calificar

Calificar

Poner comentario

Asignaturas

Estudiantes

Figura 3.12: Mockup operativo: detalle de calificación como profesor (DAS)

Como apoyo adicional, la Figura 3.13 ilustra el estado de entrega con contenido adjunto, útil para justificar la trazabilidad de versiones y el control de evidencias de envío.



Figura 3.13: Mockup de entrega con archivo adjunto (DAS)

Actor	Operación central	Evidencia DAS reutilizada	Requisitos ERS asociados
Administrador	Alta y administración de identidades/cursos	Mockups de alta de usuario y creación de curso	RQF-001 a RQF-004, RQF-021, RQF-022
Profesor	Diseño y publicación de actividad/entregable	Fig. 3.9 y mockup de creación de actividad	RQF-005 a RQF-013, REGN-007
Estudiante	Entrega y seguimiento de estado	Fig. 3.6, Fig. 3.13	RQF-014 a RQF-017, REGN-005
Profesor evaluador	Corrección y feedback	Fig. 3.10, Fig. 3.12	RQF-018, RQF-019, REGN-001, REGN-008
Profesor y estudiante	Consulta y trazabilidad de resultados	Mockups de comentarios y vista cruzada docente-estudiante	RQF-020, RQI-005, RQNF-009

Tabla 3.13: Cobertura operativa por rol con evidencia de DAS y requisitos de ERS

3.3.5. Catálogo de casos de uso y matriz de trazabilidad ERS-DAS-implementación

El DAS define un catálogo amplio de casos de uso; en esta memoria se han seleccionado cuatro diagramas representativos (Figuras 3.14 a 3.17) que cubren acceso, gestión académica, entregas y evaluación. Esta selección se corresponde con los flujos implementados en el MVP y sirve como base visual para enlazar el modelo operativo con la evidencia de implementación y pruebas. Como representación visual de dicha cobertura, se incluyen cuatro diagramas de casos de uso

de referencia (Figuras 3.14 a 3.17), para documentar el enlace entre el catálogo funcional y los flujos reales desarrollados.

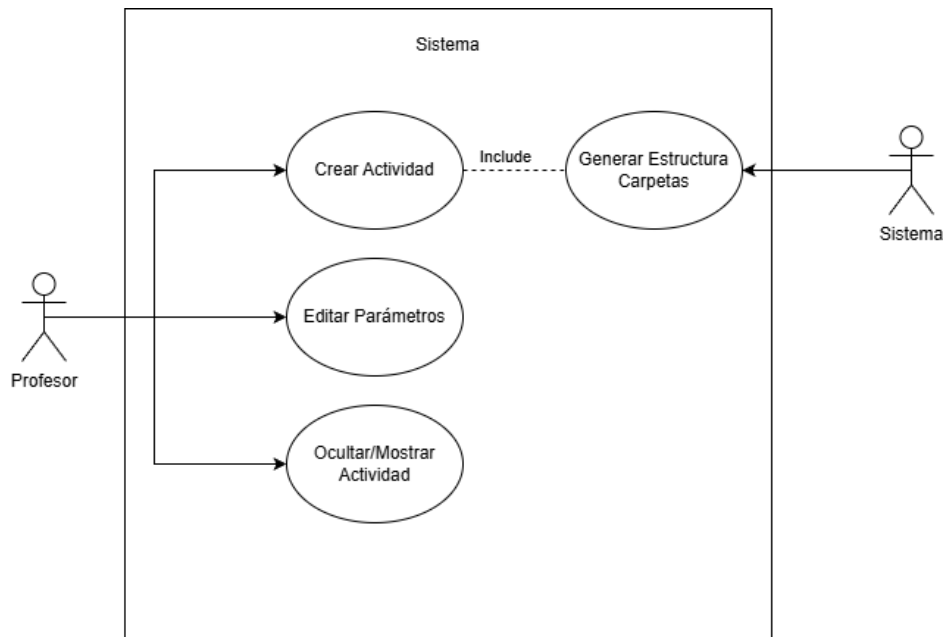


Figura 3.14: Caso de uso representativo CU-1 (DAS)

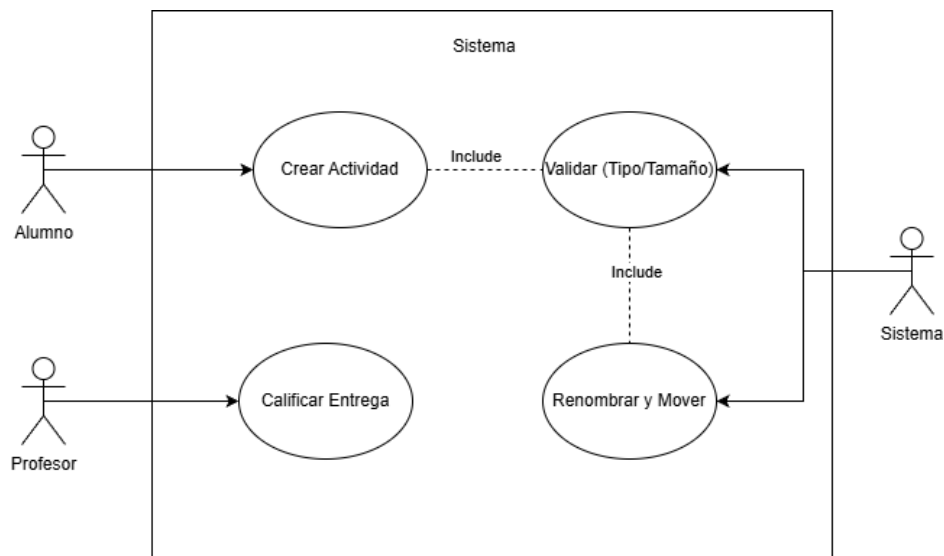


Figura 3.15: Caso de uso representativo CU-2 (DAS)

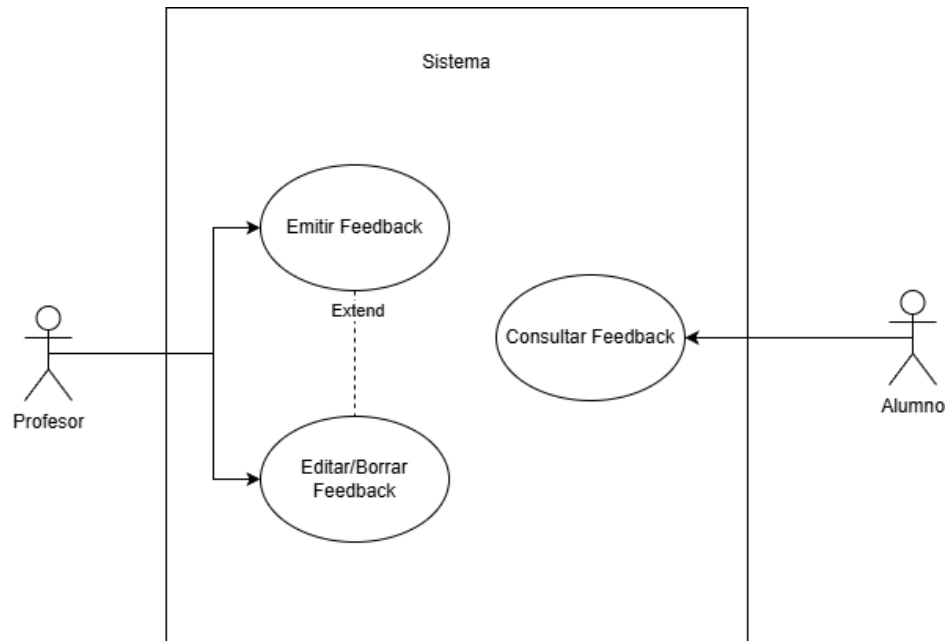


Figura 3.16: Caso de uso representativo CU-3 (DAS)

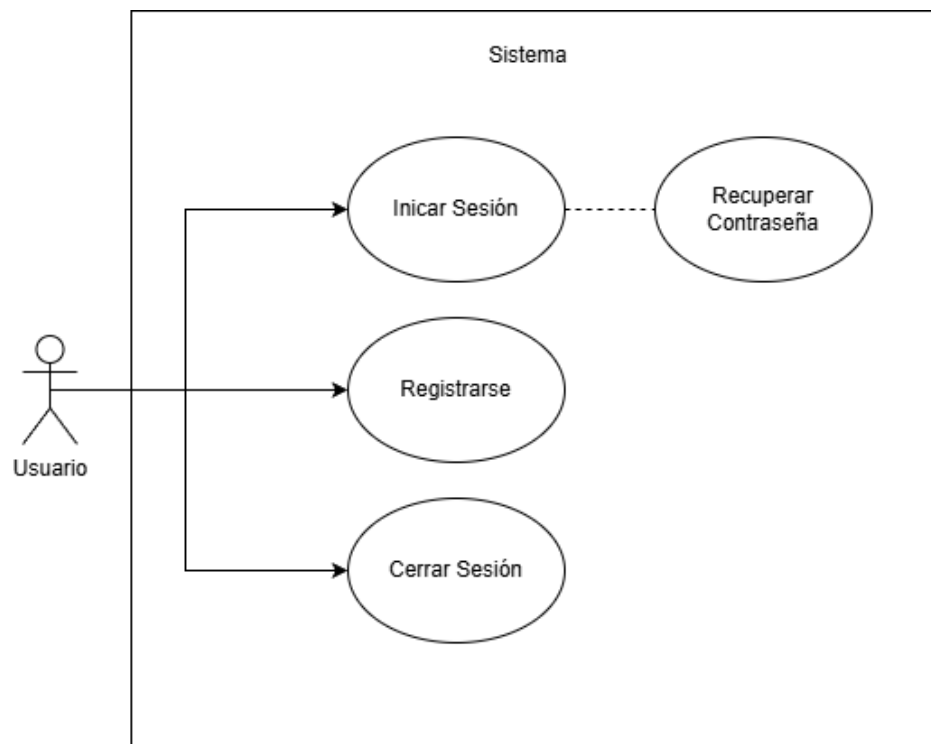


Figura 3.17: Caso de uso representativo CU-4 (DAS)

Para resumir la cobertura por familias operativas dentro del MVP, la Tabla 3.14 agrupa el alcance funcional, el actor dominante y el grado de cobertura logrado.

Familia operativa	Alcance funcional en DA-S/ERS	Actor dominante	Cobertura en MVP
Acceso y sesión	Inicio de sesión, control de identidad, continuidad de sesión	Todos los roles	Completa
Administración académica base	Alta/modificación de cursos, usuarios y contexto de trabajo	Administrador	Completa
Diseño docente de actividades	Creación, edición, visibilidad y configuración de entregables	Profesor	Completa
Gestión de entregas	Publicación de trabajo, versionado y consulta de historial	Estudiante	Completa
Evaluación y feedback	Calificación, comentarios y visibilidad de resultados	Mixto	Completa
Supervisión y trazabilidad	Vistas cruzadas profesor-estudiante y revisión del estado de entregas	Profesor	Parcial avanzada

Tabla 3.14: Cobertura por familias de casos de uso del DAS dentro del alcance del MVP

Para reforzar la coherencia documental, la Tabla 3.15 integra tres niveles de evidencia: requisito formal en ERS, artefacto de diseño en DAS y materialización en componentes software y pruebas del proyecto.

Bloque ERS	Artefacto DAS reutilizado	Implementación principal	Evidencia de verificación
RQG-001, RQG-007	Arquitectura + navegabilidad + mockups de sesión	Configuración de seguridad, filtros JWT, control de rutas	Tests de autenticación/autorización
RQI-001 a RQI-005	Diagrama de clases + vistas por rol	Entidades JPA, DTOs y contratos API	Pruebas de persistencia e integración
RQF-005 a RQF-013	Secuencia de creación docente + mockups de actividad	Servicios de actividad/entregable y controladores asociados	Unit tests y API tests de gestión docente
RQF-014 a RQF-017	Secuencia de entrega + estados de mockup	Servicio de entregas, validaciones y versionado	Integración de entregas y E2E de alumno
RQF-018, RQF-019	Secuencia de evaluación + detalle de calificación	Flujo de feedback/calificación en backend y frontend	Pruebas funcionales de evaluación
REGN-001 a REGN-008	Casos de uso por rol + navegabilidad	Validaciones contextuales en servicios y filtros	Pruebas de reglas de negocio
RQNF-002, RQNF-003	Inventario de mockups y patrones de interfaz	Diseño de pantallas y mensajes de estado	E2E sobre flujos completos
RQNF-004, RQNF-005, RQNF-009	Arquitectura de subsistemas + separación de capas	Estructura modular backend/frontend + control de acceso	CI/CD, test suites y auditoría técnica

Tabla 3.15: Trazabilidad ampliada entre ERS, DAS e implementación efectiva

3.3.6. Cobertura documental alcanzada

Con la incorporación anterior, este capítulo integra de forma explícita los cinco bloques completos del catálogo ERS (RQG, RQI, RQF, REGN y RQNF), la descomposición en subsistemas usada durante diseño e implementación, los diagramas estructurales y de comportamiento del DAS, el catálogo operativo de casos de uso y familias funcionales del sistema, el conjunto de mockups de interfaz como evidencia de diseño funcional y una matriz ampliada de trazabilidad entre especificación, diseño, implementación y verificación. Esta integración evita vacíos entre análisis y ejecución.

De esta forma, la memoria no solo resume resultados: también preserva la trazabilidad documental completa entre especificación inicial, diseño técnico y comportamiento implementado.

Diseño y Arquitectura

4.1– Diseño del sistema

4.1.1. Arquitectura de alto nivel

La arquitectura adoptada es cliente-servidor desacoplada, con un **frontend SPA** en React + TypeScript para presentación y experiencia de usuario, un **backend REST** en Spring Boot para lógica de negocio y seguridad, y **persistencia relacional** con JPA/Hibernate y migraciones Flyway. Esta separación delimita responsabilidades y facilita evolución modular.

En backend se aplica diseño por capas (controller, service, repository y entity), que mejora mantenibilidad, testabilidad y capacidad de evolución.

4.1.2. Aportación arquitectónica y tecnológica

Desde la perspectiva técnica, la contribución principal se concreta en una separación frontend-backend mediante API REST, una arquitectura por capas en backend que aísla responsabilidades, el uso de DTOs para desacoplar contratos de API del modelo de persistencia y una evolución de esquema controlada por migraciones versionadas. Este diseño reduce el acoplamiento transversal, facilita el mantenimiento evolutivo a medio plazo y habilita la incorporación incremental de nuevas integraciones sin ruptura del núcleo transaccional.

4.1.3. Vista de despliegue

La vista de despliegue se organiza en bloques principales que cubren el **cliente web** (navegador del usuario final), el **frontend** como SPA estática servida desde plataforma PaaS (Render), el **backend** como API REST desplegada en contenedor Docker en Render, los **datos** en PostgreSQL gestionada accesible solo desde backend, y los **servicios externos** de almacenamiento cloud, OneDrive y correo transaccional por API HTTP. Esta disposición mantiene una frontera de seguridad clara entre capas.

La separación de componentes permite escalar y evolucionar cada capa sin romper el resto del sistema, manteniendo además una frontera de seguridad clara entre cliente, lógica de negocio y persistencia.

Nodo	Tecnología	Responsabilidad principal
Cliente	Navegador moderno	Interacción de usuario y consumo de API sobre HTTPS.
Frontend	React + Vite en Render	Renderizado de interfaz, estado de sesión y experiencia por rol.
Backend	Spring Boot (Java 21) en Render	Exposición de endpoints, reglas de negocio, seguridad y orquestación de persistencia.
Persistencia	Render PostgreSQL	Almacenamiento transaccional y trazabilidad de datos académicos.
Correo transaccional	Brevo API HTTP	Recuperación de contraseña y notificaciones sin depender de puertos SMTP salientes.

Tabla 4.1: Vista de despliegue simplificada del sistema

4.1.4. Diseño de dominio y datos

El dominio se modela alrededor de la relación docente-estudiante-actividad, con `Usuario` como identidad base, `Profesor` y `Estudiante` como especializaciones operativas, `Curso`, `Grupo` y `Actividad` para el contexto académico, `Entregable` y `Entrega` para el ciclo de entrega, y `Feedback`, `Material` y entidades de soporte (tokens, notificaciones). Esta estructura facilita trazabilidad y coherencia de reglas de negocio.

La persistencia evoluciona de forma versionada mediante migraciones, lo que permite incorporar nuevas capacidades sin perder consistencia histórica.

4.1.5. Diseño de API

La API sigue orientación REST y separa contratos mediante DTOs. Se definen módulos de endpoint coherentes por contexto de negocio, incluyendo `/api/auth` para login, refresh y recuperación de contraseña, `/api/actividades` para gestión de actividades y consulta por usuario, `/api/entregables` para administración de entregables, y `/api/entregas` para subida, revisión, evaluación y descarga de entregas. Esta segmentación facilita evolución de contratos por dominio.

El uso de DTOs evita exponer directamente entidades de persistencia, reduce acoplamiento y simplifica evoluciones de contrato.

4.1.6. Diseño de seguridad

El diseño de seguridad incorpora mecanismos complementarios: autenticación basada en JWT (token de acceso y refresh token), filtros de validación en la cadena de seguridad, autorización por rol y contexto académico, cifrado de contraseñas con BCrypt y limitación de tasa en operaciones sensibles de autenticación. Esta combinación reduce superficie de ataque sin penalizar la experiencia de uso.

Se configura CORS de forma explícita y se diferencia entre rutas públicas y protegidas, minimizando superficie de ataque sin comprometer usabilidad.

4.1.7. Diseño del frontend

El frontend se organiza en módulos `pages`, `components`, `context` y `services` con objetivos de experiencia coherente para profesor y estudiante, protección de rutas según sesión y rol, gestión centralizada de autenticación y manejo uniforme de errores y estados de carga. Esta organización evita duplicidades y mejora mantenibilidad.

La capa de red encapsula llamadas HTTP y gestión de tokens, incluyendo renovación automática de sesión al expirar el token de acceso.

Esta granularidad no es habitual en soluciones generalistas, que suelen tratar todo como subida de fichero sin reglas contextuales.

Versionado y reenvío con trazabilidad

Cada entrega queda registrada con versión y fecha, manteniendo un historial completo y una versión activa por entregable. Cuando el reenvío está permitido, el sistema conserva la trazabilidad sin sobrescribir el historial previo, facilitando auditoría docente.

Operativa docente optimizada

Se incorporan descargas masivas por entregable o actividad, previsualización de archivos en línea y listado de contenido de ZIP, reduciendo tareas manuales de comprobación y clasificación.

Robustez en el flujo de estudiante

El flujo de entrega incorpora borradores automáticos en cliente, revalidación antes de enviar y mensajes de error contextualizados. Esto minimiza pérdida de trabajo en conexiones inestables y mejora la experiencia del alumnado.

4.2– Alternativas de arquitectura y stack

4.2.1. Arquitectura de aplicación

Se valoraron dos enfoques principales: **A1 - SPA + API desacoplada** (opción elegida) y **A2 - Aplicación monolítica server-side renderizada**. Esta comparativa permite sopesar flexibilidad de evolución frente a simplicidad inicial.

A1 permite separar claramente la capa de experiencia de usuario de la lógica de negocio y persistencia, facilitando pruebas independientes, despliegues diferenciados y evolución incremental.

A2 reduce complejidad inicial de infraestructura, pero limita flexibilidad de frontend y complica evolución de UX avanzada en escenarios con roles y paneles diferenciados.

4.2.2. Backend framework

Se compararon principalmente **B1 - Spring Boot (Java)** (elegida), **B2 - Node.js (Express/-Nest)** y **B3 - Django (Python)**. El análisis consideró compatibilidad con seguridad, persistencia y tiempos de entrega.

La elección de **Spring Boot** se justifica por su ecosistema maduro para seguridad, JPA, validación y testing, por la integración natural con migraciones Flyway y por el buen soporte a arquitectura por capas y control transaccional. Estos factores reducen incertidumbre técnica en el alcance temporal disponible.

Node.js y Django son opciones válidas, pero para este proyecto requerían más decisiones de ensamblaje en piezas de seguridad/persistencia para alcanzar un nivel equivalente en el tiempo disponible.

4.2.3. Frontend framework

Alternativas consideradas: **F1 - React + TypeScript + Vite** (elegida), **F2 - Angular** y **F3 - Vue**. La selección se orientó a equilibrio entre velocidad de desarrollo y control de complejidad.

React con TypeScript y Vite ofrece buen equilibrio entre velocidad de desarrollo, ecosistema y control de complejidad para una SPA de tamaño medio. Angular aporta una estructura más

opinionada, pero con curva de entrada superior para el alcance temporal del TFG. Vue mantiene una DX excelente, aunque en este caso la experiencia previa del equipo y la alineación con librerías de testing/CI favorecieron React.

4.3– Alternativas de persistencia y almacenamiento

4.3.1. Base de datos relacional

Opciones comparadas: **D1 - PostgreSQL gestionado** (elegida en producción), **D2 - MySQL gestionado** y **D3 - H2 embebida** (solo desarrollo/pruebas locales). La selección equilibra estabilidad productiva con agilidad local.

PostgreSQL se mantuvo como opción principal por estabilidad, compatibilidad con el stack y buen comportamiento en entornos PaaS. H2 se reserva para contextos de desarrollo y pruebas, no para producción.

Dentro de las alternativas de base de datos desplegadas, inicialmente se decidió utilizar **Neon** como proveedor PostgreSQL por sus mejores características percibidas para el arranque del proyecto: aprovisionamiento rápido, enfoque serverless y facilidad de conexión desde un backend desplegado en PaaS. No obstante, durante la validación operativa se detectaron limitaciones en su plan gratuito que condicionaban la continuidad del despliegue académico. Por ello se migró a **Render PostgreSQL**, manteniendo PostgreSQL como decisión técnica de base y cambiando solo el proveedor gestionado. La migración se realizó sin cambios de código de dominio, apoyándose en variables de entorno y configuración portable del datasource, lo que confirma bajo acoplamiento a proveedor concreto.

4.3.2. Estrategia de esquema

Se compararon **E1 - Flyway como autoridad de esquema** (elegida) y **E2 - Hibernate ddl-auto=update en producción**. La decisión prioriza trazabilidad y control explícito del cambio.

Se adopta **E1 (Flyway)** por trazabilidad y control explícito de cambios, reduciendo el riesgo de divergencias de esquema entre entornos. En un entorno académico y de desarrollo rápido, el uso de herramientas automáticas como Hibernate puede generar esquemas inconsistentes al cambiar ramas de código o desplegar nuevas versiones. Con Flyway, cada alteración de la base de datos queda versionada (*scripts SQL*), permitiendo reproducir exactamente el mismo estado desde el entorno de desarrollo local hasta el de producción.

4.3.3. Almacenamiento de archivos

Alternativas evaluadas: **S1 - Almacenamiento local en servidor**, **S2 - Cloudinary** (elegida para producción persistente) y **S3 - OneDrive/Microsoft Graph** (integración opcional por contexto académico). La opción final combina persistencia en producción con flexibilidad institucional.

La opción local es simple y adecuada en desarrollo, pero en plataformas con sistema de archivos efímero (como despliegues free) puede provocar pérdida de datos tras reinicio. Por ello se adopta **Cloudinary** en producción.

La integración **OneDrive** aporta valor en casos docentes que requieren sincronización con ecosistemas Microsoft, pero se mantiene como capacidad configurable y no obligatoria del núcleo.

4.4– Alternativas de seguridad y sesión

4.4.1. Modelo de autenticación

Se compararon dos enfoques principales: **A1 - JWT (access + refresh)** (elegido) y **A2 - Sesión server-side tradicional**. La elección favorece coherencia con arquitectura SPA y escalado

stateless.

En una SPA desacoplada, JWT con refresh ofrece una experiencia más natural para cliente API-first y facilita escalado stateless del backend. La sesión server-side reduce cierta complejidad de token, pero exige estrategia adicional de estado distribuido en despliegues horizontales.

4.4.2. Extensiones de seguridad

Se han considerado funcionalidades avanzadas (inicio de sesión federado con OAuth2/OIDC y 2FA) como línea de evolución. La decisión de no imponerlas en el MVP responde al criterio de maximizar cobertura funcional core sin comprometer plazos.

4.5– Alternativas investigadas y descartadas

4.5.1. SharePoint para OneDrive corporativo

Se valoró el uso de SharePoint como intermediario para acceder a cuentas corporativas de OneDrive. La opción facilitaba el encaje institucional, pero implicaba perder capacidades clave que sí ofrece la integración directa, como la selección de carpeta de destino o la gestión flexible de rutas durante la subida de entregas. Dado que estas funciones son parte del valor diferencial de la plataforma, se descartó esta vía.

4.5.2. Registro de usuarios con OAuth

Se estudió habilitar registro de usuarios mediante OAuth/OIDC para que el alumnado pudiera crear cuentas de forma autónoma. Finalmente se priorizó el flujo habitual en entornos académicos, donde el administrador crea usuarios y los asigna a cursos y grupos. Esto evita altas desalineadas con la matrícula real y simplifica el control de permisos y pertenencia a grupos.

4.6– Alternativas de despliegue y operación

4.6.1. Estrategia de despliegue

Opciones principales: **P1 - PaaS con automatización GitHub Actions + Render** (elegida), **P2 - VPS autogestionado con pipeline manual o Jenkins** y **P3 - Orquestación completa en Kubernetes**. La selección busca minimizar carga operativa y mantener viabilidad académica.

P1 reduce carga operativa, permite despliegue rápido y mantiene un coste ajustado para contexto académico. La opción elegida se concreta en Render como PaaS gestionado para el servicio backend, el frontend estático y PostgreSQL gestionado, manteniendo la configuración por variables de entorno. **P2** y **P3** ofrecen más control, pero su complejidad no aporta retorno proporcional dentro del alcance del TFG.

Dentro de esta estrategia, el correo transaccional se resuelve con Brevo mediante API HTTP. Inicialmente se empleó SendGrid como proveedor de correo, pero tras la finalización de su plan gratuito se migró a Brevo. Se evaluó también Resend, pero su plan gratuito requiere un dominio verificado para enviar a destinatarios arbitrarios. Esta alternativa se prefiere frente a SMTP en producción porque los PaaS pueden bloquear o restringir puertos propios del protocolo de correo, mientras que las peticiones HTTPS salientes son compatibles con el modelo operativo del despliegue.

4.6.2. CI/CD y calidad

Se compararon **C1 - CI/CD integrada en GitHub Actions** (elegida) y **C2 - CI externa (GitLab CI/Jenkins) con mayor administración**. La decisión prioriza trazabilidad y menor sobrecarga operativa.

La alternativa elegida integra lint, tests, build, análisis de seguridad, mutación y publicación de artefactos en un flujo unificado y reproducible.

4.7– Comparativa multicriterio

La Tabla 4.2 sintetiza la comparación global entre enfoques de solución.

Criterio	Solución elegida	Alternativa intermedia	Alternativa avanzada
Tiempo de entrega MVP	5	3	2
Mantenibilidad	4	3	4
Coste operativo	5	3	2
Escalabilidad potencial	4	3	5
Complejidad de operación	4	3	1
Adecuación al TFG	5	3	2

Tabla 4.2: Comparativa global (escala 1-5, mayor es mejor)

Interpretación: la alternativa seleccionada no maximiza todos los ejes teóricos, pero sí ofrece el mejor equilibrio entre valor entregado, riesgo técnico y viabilidad temporal del proyecto.

4.8– Decisiones adoptadas y justificación final

Las decisiones finales que mejor explican el resultado del proyecto son la arquitectura SPA + API desacoplada para separar responsabilidades, un backend Spring Boot con persistencia relacional y migraciones Flyway, un frontend React + TypeScript para productividad con control tipado, almacenamiento cloud persistente en producción y local en desarrollo, y despliegue en PaaS con CI/CD automatizada y verificaciones de calidad. Esta combinación asegura equilibrio entre viabilidad temporal y robustez técnica.

Esta combinación ha permitido llegar a un sistema funcional, con cobertura de pruebas alta y capacidad de evolución, sin sobredimensionar infraestructura ni desviarse del objetivo académico.

4.9– Conclusiones

El análisis y estudio de alternativas confirma que la solución implementada no responde a una selección arbitraria de herramientas, sino a una evaluación razonada frente a alternativas reales.

En cada ámbito de decisión, el enfoque adoptado prioriza el equilibrio entre robustez técnica, viabilidad temporal y bajo coste operativo. La decisión por una arquitectura desacoplada (SPA + API), respaldada por frameworks de amplio uso (Spring Boot y React) y persistencia relacional estricta, provee una plataforma sólida y extensible. Al mismo tiempo, el descarte de integraciones más complejas (como SSO o SharePoint) permite concentrar los recursos del MVP en el núcleo de la operativa docente.

Este conjunto de alternativas y su posterior cribado aseguran la construcción de un TFG de alto perfil profesional, que cumple rigurosamente con sus objetivos limitando conscientemente los riesgos técnicos y de calendario.

Implementación y Desarrollo

5.1– Implementación

5.1.1. Implementación backend

La implementación backend se estructura en el paquete principal de la aplicación, donde destacan **controladores** como entrada HTTP y validación de contratos, **servicios** para lógica de negocio y reglas del dominio, **repositorios** con acceso a datos mediante Spring Data JPA y un bloque de **seguridad** con configuración central y filtros especializados. Esta separación refuerza la testabilidad y el control de responsabilidades.

Un punto clave es `EntregaService`, que centraliza validaciones de entrega, control de versiones, restricciones temporales y operativa de evaluación, con apoyo de servicios auxiliares para materiales y notificaciones.

5.1.2. Implementación de flujos críticos

Los flujos con mayor impacto funcional son la **autenticación y sesión** (login, emisión de tokens, refresco y cierre seguro), la **publicación de entregables** (creación por docente y configuración de fechas y condiciones), la **entrega de material** (subida de ficheros, verificaciones, versionado y almacenamiento de metadatos) y la **evaluación** (corrección por profesor autorizado, feedback y visibilidad de nota). Esta selección concentra los riesgos funcionales del sistema.

Cada flujo incorpora validaciones de entrada, control de errores y verificación de permisos, asegurando consistencia en escenarios concurrentes y previniendo acciones fuera de alcance.

5.1.3. Implementación frontend

La implementación del cliente cubre pantallas y operaciones necesarias para el MVP, incluyendo acceso y gestión de sesión, paneles de actividades y entregables, formularios de entrega y seguimiento, y vista de evaluaciones con retroalimentación. Esta cobertura garantiza recorrido completo por rol.

Se han introducido medidas prácticas para mejorar usabilidad: mensajes de validación temprana, estados visuales de proceso, y comportamiento resiliente ante errores de red.

5.1.4. Persistencia, migraciones e integraciones

Se utiliza PostgreSQL en escenarios de despliegue y un perfil de desarrollo local para iteración rápida. Flyway gestiona la evolución del esquema, incluyendo ampliaciones como atributos para validación estructural de entregas ZIP, campos para integración cloud (OneDrive/Cloudinary),

tablas y preferencias de notificación, y tokens de recuperación de contraseña. Esta evolución versionada protege la consistencia histórica de datos.

Las integraciones externas se diseñan por configuración, de modo que el sistema puede operar en local sin dependencia obligatoria de servicios de terceros. En producción el envío de correos se realiza mediante Brevo a través de peticiones HTTPS, evitando depender del protocolo SMTP, que puede estar restringido en plataformas PaaS. Este canal se utiliza para recuperación de contraseña y notificaciones transaccionales.

5.1.5. Verificación de implementación

La verificación se realiza en capas (unitaria, integración, E2E y mutación), apoyada por pipelines CI/CD y controles de seguridad en preproducción. Este enfoque proporciona evidencia objetiva de robustez y reduce el riesgo de regresiones.

5.2– Discusión técnica

5.2.1. Decisiones y compromisos

Las decisiones principales del diseño implican compensaciones explícitas. No se buscó seleccionar la alternativa más avanzada en cada eje, sino construir una solución coherente con el alcance del MVP, el tiempo disponible, los requisitos del ERS y la necesidad de mantener trazabilidad entre especificación, implementación y pruebas. Por ello, cada decisión se evaluó atendiendo a cuatro criterios: valor funcional para el flujo de entregables, mantenibilidad, riesgo de integración y coste operativo en un entorno académico.

Arquitectura SPA + API

La separación entre frontend SPA y backend REST se adoptó porque el dominio requiere experiencias diferenciadas por rol y una API reutilizable para operaciones de entrega, evaluación y administración. Esta elección mejora la mantenibilidad y permite evolucionar la interfaz sin modificar el núcleo de negocio. El compromiso asumido es un mayor esfuerzo de orquestación: gestión de CORS, contratos HTTP, renovación de sesión y pruebas de integración entre cliente y servidor. Se consideró aceptable porque el proyecto ya incorporaba pipelines, pruebas automatizadas y una estructura de servicios capaz de absorber esa complejidad.

Seguridad y gestión de sesión

El uso de JWT con refresh token se eligió por su ajuste natural a una aplicación desacoplada y por reducir estado de sesión en servidor. Frente a una sesión server-side tradicional, esta opción facilita despliegues donde el backend puede escalar o reiniciarse sin depender de memoria local de sesión. La contrapartida es que obliga a controlar expiración, renovación, cierre de sesión y protección de rutas con mayor disciplina. Para mitigarlo, se centralizó la validación en filtros de seguridad y se reforzó la autorización por rol y contexto académico.

Persistencia, migraciones y almacenamiento

La persistencia relacional se mantuvo como base del sistema porque el dominio contiene relaciones fuertes entre usuarios, cursos, grupos, actividades, entregables, entregas y feedback. Esta decisión favorece integridad referencial y consultas trazables. Flyway se adoptó para que la evolución del esquema quedara versionada, especialmente al introducir campos de validación ZIP, preferencias de notificación e integraciones cloud. Como compromiso, cada cambio de datos exige disciplina de migración, pero a cambio se reduce el riesgo de divergencias entre entornos.

En almacenamiento de archivos se separaron dos necesidades. Para persistencia operativa de producción se priorizó un proveedor cloud configurable, evitando depender del sistema de archivos efímero de la plataforma PaaS. Para contextos académicos donde el profesorado ya trabaja con Microsoft 365, se incorporó OneDrive como integración opcional, no como dependencia obligatoria del núcleo. Esta separación permite que el sistema funcione sin servicios externos durante desarrollo y que, en producción, pueda activar capacidades institucionales cuando el entorno lo permita.

API de OneDrive frente a SharePoint

La integración con OneDrive se resolvió mediante Microsoft Graph sobre la unidad del usuario autenticado. Esta decisión responde al problema operativo concreto del proyecto: el profesorado necesita conectar su propia cuenta, explorar carpetas, seleccionar rutas de destino por actividad o por entregable y recibir entregas con una estructura controlada. El acceso directo mediante OAuth2 y permisos delegados permite trabajar sobre la unidad del usuario con operaciones de subida, descarga, listado de carpetas y renovación de token, manteniendo la integración acotada al flujo de entregables.

SharePoint se valoró como alternativa para escenarios corporativos, pero su orientación principal es la gestión de sitios, bibliotecas documentales y permisos compartidos a nivel de organización. Ese enfoque encaja bien cuando se quiere centralizar documentación institucional, pero introduce dependencias adicionales de tenant, aprovisionamiento de sitios, bibliotecas y políticas administradas. Para este TFG, esa complejidad no aportaba una mejora proporcional sobre la necesidad principal: seleccionar carpetas del OneDrive del usuario y organizar entregas de forma flexible. Por ello se descartó como vía principal y se priorizó la API de OneDrive mediante Microsoft Graph, dejando SharePoint como posible extensión futura si la implantación institucional exigiera repositorios compartidos.

Despliegue y servicios externos

El despliegue en PaaS con CI/CD automatizada se adoptó para reducir carga operativa y mantener reproducibilidad. Del mismo modo, el correo transaccional mediante Brevo se seleccionó por operar sobre API HTTPS, evitando restricciones frecuentes de SMTP saliente en plataformas gestionadas. Estas elecciones no maximizan el control de infraestructura, como ocurriría con un VPS autogestionado, pero reducen riesgos de administración y concentran el esfuerzo del TFG en el producto, las pruebas y la trazabilidad funcional.

5.2.2. Limitaciones y trabajo futuro

Aunque el núcleo funcional está cubierto, las decisiones anteriores dejan una hoja de ruta técnica clara. La autenticación local con JWT no sustituye a un SSO institucional completo, por lo que una evolución natural sería incorporar OAuth2/OIDC, SAML o LDAP como mecanismos de identidad federada. La integración con OneDrive depende de la configuración de Azure y de permisos concedidos por el usuario, de modo que un despliegue institucional debería revisar políticas de consentimiento, auditoría y retención de datos. Además, el sistema valida estructura, extensión y contenido esperado de entregas, pero no incorpora todavía análisis antivirus exhaustivo ni observabilidad avanzada con métricas de uso, tiempos de corrección o alertas operativas.

Estas limitaciones no condicionan la validez del TFG, sino que delimitan una continuidad razonable: reforzar identidad, auditoría, seguridad documental, métricas de producción y requisitos *Should/Could* pendientes sin reescribir el núcleo transaccional.

5.3– Conclusiones

El análisis realizado demuestra coherencia entre lo especificado y lo implementado. Los requisitos funcionales priorizados se traducen en flujos operativos reales, los no funcionales se abordan mediante decisiones técnicas concretas, y el diseño modular facilita mantenimiento, verificación y evolución.

Por tanto, el resultado no es solo una implementación funcional, sino una solución con fundamento de ingeniería de software, alineada con los objetivos académicos y profesionales del TFG.

Justificación detallada de decisiones

6.1– Introducción

En un TFG de ingeniería de software, implementar funcionalidades no es suficiente si no se explica por qué se ha elegido cada opción técnica, qué alternativas se consideraron y qué compromisos se asumieron.

Este capítulo amplía de forma exhaustiva la justificación de decisiones adoptadas durante el proyecto, con foco en cuatro ejes: producto, arquitectura, calidad y operación.

6.2– Marco de toma de decisiones

6.2.1. Criterios utilizados

La selección de alternativas se realizó usando un enfoque multicriterio, priorizando la alineación con requisitos funcionales y no funcionales del ERS, la viabilidad temporal dentro de 660 horas de proyecto, la mantenibilidad y capacidad de evolución posterior, el coste operativo compatible con entorno académico y la seguridad con trazabilidad del ciclo completo. Esta pauta evita decisiones aisladas y garantiza coherencia transversal.

6.2.2. Ponderación de criterios

Para evitar decisiones ad-hoc o basadas únicamente en preferencias personales, se aplicó una ponderación orientativa de criterios que actuó como filtro primario en todos los bloques de diseño (arquitectura, stack tecnológico, seguridad y despliegue). Esta ponderación (Tabla 6.1) exige que cada decisión justifique prioritariamente su adecuación a los requisitos, y de manera secundaria, la mantenibilidad y el riesgo temporal. Sólo cuando dos alternativas empatan en estos ámbitos, se dirige en función del coste operativo o la robustez técnica.

Criterio	Peso (%)
Adecuación funcional y cobertura de requisitos	30
Mantenibilidad y evolución	20
Tiempo de implementación y riesgo de entrega	20
Coste operativo y sostenibilidad	15
Seguridad y robustez técnica	15
Total	100

Tabla 6.1: Ponderación orientativa para selección de decisiones

6.3– Decisiones de alcance funcional

6.3.1. Decisión D1: definir un MVP centrado en flujo académico nuclear

Alternativas consideradas: incorporar desde inicio todas las funcionalidades del backlog o limitar el alcance a un MVP con ciclo completo de entrega y evaluación. La comparativa se centró en riesgo de entrega y capacidad de validación.

Decisión adoptada: segunda alternativa.

Razón principal: maximizar valor entregado en plazo, evitando dispersión en funcionalidades accesorias que podían comprometer la estabilidad del núcleo.

Impacto positivo: mayor probabilidad de cierre funcional completo, mejor cobertura de pruebas sobre procesos críticos y documentación más sólida de extremo a extremo. Este impacto refuerza la validez académica del resultado.

Compromiso asumido: funcionalidades *Should/Could* pasan a hoja de ruta, sin bloquear la validez del resultado principal.

6.3.2. Decisión D2: diseñar experiencia por rol desde el inicio

Decisión adoptada: separar claramente experiencias de administrador, profesor y estudiante.

Razones: reduce ambigüedad de permisos en interfaz y API, simplifica pruebas de autorización y casos negativos, y mejora usabilidad al mostrar solo operaciones pertinentes. Esta separación evita errores de contexto en operaciones sensibles.

6.4– Decisiones de arquitectura

6.4.1. Decisión D3: arquitectura SPA + API desacoplada

Alternativas: aplicación monolítica renderizada en servidor o frontend SPA con backend REST desacoplado. La decisión se evaluó en términos de evolución y coste de integración.

Decisión adoptada: SPA + API.

Razones de peso: separación de responsabilidades y menor acoplamiento, evolución independiente de capa de presentación y lógica, y mejor ajuste a un dominio con flujos ricos de interfaz (evaluaciones, filtros, historial). Esta elección maximiza mantenibilidad en el medio plazo.

Trade-off: aumenta esfuerzo de integración y de pruebas E2E, compensado por mayor mantenibilidad a medio plazo.

6.4.2. Decisión D4: arquitectura backend por capas

Decisión adoptada: patrón `controller-service-repository-entity` con DTOs de frontera.

Razones: control claro de responsabilidades, testabilidad por módulo, evitación de exponer modelo de persistencia en contratos externos y facilidad para versionado de API y evolución de dominio. El patrón elegido reduce riesgos de acoplamiento a futuro.

6.4.3. Decisión D5: uso de DTOs y mapper dedicado

Problema que resuelve: evitar dependencia directa entre entidades JPA y respuestas HTTP.

Razones: minimiza acoplamiento entre capa de datos y capa API, reduce riesgo de sobreexposición de atributos sensibles y permite modelar respuestas orientadas al caso de uso. Esto asegura contratos más estables y seguros.

Coste asumido: mayor volumen de código estructural, aceptado por mejora en mantenibilidad y seguridad de contrato.

6.5– Decisiones de stack tecnológico

6.5.1. Decisión D6: backend en Spring Boot + Java 21

Razones principales: ecosistema maduro en seguridad, validación y persistencia, integración robusta con Spring Security, JPA y testing, y soporte industrial con documentación extensa. Esta base reduce incertidumbre en implementación.

Comparativa resumida: frente a opciones como Node.js o Django, Spring reduce incertidumbre de ensamblaje en aspectos críticos (seguridad por rol, transacciones, control de acceso por contexto).

6.5.2. Decisión D7: frontend en React + TypeScript + Vite

Razones: equilibrio entre productividad y control tipado, ecosistema amplio para rutas, formularios y testing, y arranque y build rápidos con Vite. Esta combinación facilita entregas incrementales sin perder control de calidad.

Compromiso: mayor libertad implica mayor disciplina de arquitectura en frontend, mitigada mediante organización por módulos (pages/components/context/services).

6.6– Decisiones de seguridad

6.6.1. Decisión D8: autenticación JWT con refresh token

Motivación: compatible con arquitectura desacoplada y despliegue stateless.

Razones de elección: reduce estado de sesión en servidor, habilita renovación transparente de sesión en cliente y facilita escalado horizontal del backend. Esto mejora la coherencia con un modelo API-first.

Riesgos identificados: control de expiración y revocación más complejo que sesión tradicional.

Mitigación aplicada: expiraciones diferenciadas, refresh controlado, filtros de seguridad y validación de contexto por recurso.

6.6.2. Decisión D9: autorización por rol + contexto académico

Razón: un rol global no es suficiente en un dominio donde un usuario puede operar con permisos distintos según curso, grupo o actividad.

Aplicación concreta: validación en controladores y servicios de pertenencia real al contexto objetivo antes de permitir operaciones sensibles.

6.7– Decisiones de datos e integraciones

6.7.1. Decisión D10: PostgreSQL como base de datos principal

Razones: robustez transaccional y consistencia, buena compatibilidad con JPA/Hibernate y adaptación sencilla a entornos gestionados. Esta elección reduce riesgos de despliegue.

6.7.2. Decisión D11: Flyway como autoridad de esquema

Razones: trazabilidad explícita de cambios en base de datos, despliegues reproducibles entre entornos y menor riesgo de divergencias silenciosas de esquema. Se prioriza control sobre automatismos implícitos.

Decisión asociada: usar `ddl-auto=validate` en producción para evitar mutaciones implícitas del esquema fuera de control.

6.7.3. Decisión D12: estrategia híbrida de almacenamiento de archivos

Decisión adoptada: almacenamiento local en desarrollo y cloud persistente en producción.

Razones: simplicidad y velocidad en entorno local, persistencia y resiliencia en entornos con filesystem efímero, y capacidad de habilitar integraciones externas sin romper el núcleo. Se busca equilibrio entre agilidad y robustez operativa.

6.8– Decisiones de calidad y verificación

6.8.1. Decisión D13: testing en capas

Razones: unitarias para retroalimentación rápida, integración para contratos y seguridad, E2E para validar experiencia real por rol y mutación para medir eficacia real de la suite. Esta mezcla permite cubrir profundidad y visión de conjunto.

Resultado esperado: combinar profundidad y coste de mantenimiento, evitar dependencia exclusiva de una sola capa de pruebas.

6.8.2. Decisión D14: incluir mutación (PIT/Stryker)

Razón de fondo: la cobertura porcentual no garantiza detección efectiva de fallos.

Valor aportado: los mutantes supervivientes ayudan a localizar huecos de aserciones y mejorar calidad de tests en código crítico. Esto ha quedado verificado por las pruebas reales ejecutadas en el repositorio, donde la integración de PIT (backend) y Stryker (frontend) en el pipeline CI/CD ha permitido elevar la confianza en los conjuntos de aserciones.

6.9– Decisiones de operación y despliegue

6.9.1. Decisión D15: GitHub Actions como columna vertebral CI/CD

Razones: integración nativa con repositorio, trazabilidad de checks por commit/PR y menor sobrecarga de operación para equipo reducido. Esta decisión minimiza fricción en el ciclo diario.

6.9.2. Decisión D16: despliegue en PaaS gestionado

Razones: minimiza administración de infraestructura, acelera iteración y validación de cambios y mantiene coste compatible con contexto académico. Se prioriza estabilidad operativa frente a control extremo.

En la implementación concreta se utiliza Render como PaaS gestionado para el despliegue de los servicios y PostgreSQL gestionado. Para el canal de correo se adopta Brevo mediante API

HTTP, evitando depender de SMTP en producción, ya que algunos PaaS restringen puertos salientes asociados al protocolo de correo. Inicialmente se utilizó SendGrid como proveedor de correo transaccional; tras la finalización de su plan gratuito se migró a Brevo, que permite enviar correo a cualquier destinatario en su plan gratuito. El diseño adaptativo del EmailService soporta múltiples proveedores (Brevo, Resend, SendGrid) mediante un patrón desacoplado que permite migrar de proveedor sin modificar lógica de negocio.

Trade-off: menor control fino de infraestructura frente a VPS/Kubernetes, considerado aceptable para objetivos y alcance del TFG.

6.10– Matriz consolidada de decisiones

ID	Decisión	Alternativa principal descartada	Razón dominante
D1	Alcance MVP priorizado	Backlog completo desde inicio	Reducir riesgo de no entrega funcional
D3	SPA + API desacoplada	Monolito SSR	Mantenibilidad y evolución de UX
D6	Spring Boot + Java	Node.js/Django	Menor incertidumbre en seguridad/persistencia
D7	React + TS + Vite	Angular/Vue	Equilibrio productividad-control tipado
D8	JWT + refresh	Sesión server-side	Ajuste natural a cliente API-first
D11	Flyway + validate	ddl-auto update	Trazabilidad y control de esquema
D12	Cloud en producción	Solo almacenamiento local	Evitar pérdida en entornos efímeros
D13	Testing en capas + mutación	Solo unitarias/cobertura	Robustez real frente a regresiones
D16	PaaS gestionado	VPS/Kubernetes	Tiempo y coste operativos contenidos

Tabla 6.2: Resumen de decisiones clave y justificación dominante

6.11– Análisis coste-beneficio por bloques de decisiones

Para reforzar la justificación, se analiza cada bloque de decisiones en términos de coste inicial, beneficio obtenido y retorno en el contexto del TFG.

6.11.1. Bloque de arquitectura

Coste inicial: incremento de complejidad de integración entre frontend y backend, definición de contratos API y mayor superficie de pruebas.

Beneficio: separación de responsabilidades, facilidad para aislar incidencias, evolución independiente de interfaces sin rehacer lógica de negocio.

Retorno: alto. El coste adicional de arranque se recupera al reducir deuda técnica estructural y facilitar extensiones del sistema.

6.11.2. Bloque de stack tecnológico

Coste inicial: tiempo de configuración de toolchain, definición de convenciones de proyecto y aprendizaje cruzado entre capas.

Beneficio: ecosistema maduro, documentación abundante, integración directa con seguridad, testing y despliegue automatizado.

Retorno: alto. La reducción de incertidumbre durante la implementación ha compensado sobradamente la configuración inicial.

6.11.3. Bloque de seguridad

Coste inicial: diseño de flujo de tokens, gestión de expiraciones y tratamiento de errores de autenticación.

Beneficio: control de acceso robusto, alineación con arquitectura stateless, capacidad de endurecimiento incremental.

Retorno: muy alto. En un sistema académico con datos sensibles, la inversión en seguridad es estructural, no opcional.

6.11.4. Bloque de calidad y pruebas

Coste inicial: esfuerzo significativo en diseño de suites, mantenimiento de mocks y pipelines de verificación.

Beneficio: disminución de regresiones, detención temprana de defectos, confianza objetiva para despliegue continuo.

Retorno: muy alto. La calidad de salida mejora de forma medible y se reduce el tiempo de corrección en fases tardías.

6.11.5. Bloque de despliegue y operación

Coste inicial: configuración de entornos, variables seguras y automatismos de publicación.

Beneficio: repetibilidad, trazabilidad de releases, validación continua sobre entorno similar a producción.

Retorno: alto. La operativa se vuelve estable y menos dependiente de ejecuciones manuales frágiles.

6.12– Riesgos asociados a decisiones y medidas de contingencia

Ninguna decisión de ingeniería es neutra; por ello se explicitan riesgos y mitigaciones por cada eje principal.

Eje de decisión	Riesgo principal	Mitigación aplicada
Arquitectura desacoplada	Errores de contrato entre cliente y API	DTOs explícitos, pruebas de integración y E2E
JWT + refresh	Gestión incorrecta de expiraciones/revocación	Políticas de expiración, filtros dedicados y validación de contexto
Flyway como autoridad	Fallos de migración en despliegue	Versionado incremental, validación previa y backups de datos
Cloud en producción	Dependencia de servicio externo	Configuración por entorno y estrategia fallback local
CI/CD automatizada	Falsos positivos por inestabilidad de tests	Separación por etapas, reintentos controlados y revisión de flaky tests

Tabla 6.3: Riesgos derivados de decisiones y medidas de contingencia

6.13– Trazabilidad de decisiones con requisitos y evidencia

Para que la justificación no quede en el plano discursivo, se vinculan decisiones con requisitos y con evidencia empírica observada en implementación y verificación.

ID	Requisito(s) relacionado(s)	Evidencia principal	Conclusión de valor
D3	RQNF-004, RQNF-008	Separación frontend/backend en repositorio y pipelines	Mejora mantenibilidad y portabilidad
D8	RQNF-009, REGN-001	Filtros JWT y validaciones de autorización en servicios	Refuerza control de acceso por rol/contexto
D11	RQNF-001, RQNF-004	Histórico de migraciones versionadas	Reduce riesgo de deriva de esquema
D13	RQNF-001, RQNF-006	Suites unitarias/integración/E2E en verde y mutación alta	Mayor fiabilidad y menor regresión
D16	RQNF-008, RQNF-004	Workflows CI/CD y despliegue automatizado en PaaS	Operación reproducible con bajo coste

Tabla 6.4: Trazabilidad entre decisiones, requisitos y evidencia

6.14– Decisiones descartadas y por qué no se adoptaron en esta fase

6.14.1. Autenticación federada completa en MVP

No se adoptó como requisito de salida del MVP por impacto en integración, auditoría de seguridad y superficie de pruebas, frente al valor incremental inmediato sobre el núcleo funcional.

6.14.2. 2FA integral desde primera versión

Se consideró relevante, pero su adopción temprana podía desviar esfuerzo de procesos académicos críticos (entregas/evaluaciones) dentro del horizonte temporal disponible.

6.14.3. Orquestación avanzada de infraestructura

Kubernetes y configuraciones equivalentes se descartaron por sobrecoste operativo para un equipo reducido y un contexto de TFG, siendo más adecuado reservarlo para fases de escalado posterior.

6.15– Conclusiones

La justificación detallada muestra que las decisiones del proyecto no se basaron en preferencia tecnológica aislada, sino en una evaluación sistemática de valor, riesgo, coste y viabilidad.

El resultado es una arquitectura coherente con el ERS/DAS, con compromisos explícitos y argumentos técnicos defendibles ante tribunal, que proyecta una hoja de ruta de evolución realista para fases futuras.

Pruebas

7.1– Introducción

La fase de pruebas tiene como objetivo validar el sistema desde una doble perspectiva: **correctitud funcional** y **calidad de implementación**. En un proyecto con arquitectura desacoplada (frontend y backend), la validación debe cubrir tanto el comportamiento interno de los módulos como los flujos de usuario de extremo a extremo.

Por este motivo se adopta una estrategia de pruebas en capas, complementada con análisis de mutación y ejecución automatizada en CI/CD.

7.2– Objetivos de verificación

Los objetivos principales de esta fase son verificar que los casos de uso prioritarios funcionan de forma estable, prevenir regresiones al introducir cambios en lógica de negocio, garantizar que las restricciones de seguridad y autorización se respetan, medir cobertura y fortaleza de tests de forma objetiva, e integrar la validación en el ciclo continuo de construcción y despliegue. Esta formulación conecta calidad funcional y operativa en una misma línea de evidencia.

7.3– Estrategia de pruebas en capas

La estrategia sigue una pirámide de testing con cuatro niveles: pruebas unitarias para validar lógica aislada de servicios, utilidades y componentes; pruebas de integración para validar interacción entre capas y contratos de API; pruebas E2E para validar flujos de usuario en navegador; y pruebas de mutación para evaluar la capacidad real de los tests de detectar defectos. Esta combinación equilibra rapidez de feedback y profundidad de verificación.

La Tabla 7.1 resume las tecnologías aplicadas.

Nivel	Herramientas	Finalidad principal
Unitarias frontend	Vitest + Testing Library	Validación de componentes, contextos, servicios y utilidades del cliente.
Unitarias e integración backend	JUnit + Spring Test + MockMvc	Verificar servicios, controladores, seguridad y persistencia.
E2E frontend	Playwright	Simular escenarios reales de uso en navegador con flujos completos.
Mutación frontend	Stryker	Medir fortaleza de tests frente a mutantes en servicios y utilidades.
Mutación backend	PIT	Medir capacidad de detección de fallos en servicios/controladores objetivo.

Tabla 7.1: Matriz de niveles y herramientas de prueba

7.4– Inventario cuantitativo de pruebas realizadas

Para documentar de forma explícita el alcance de verificación, se ha consolidado el recuento de pruebas por capa a partir de los reportes generados por Vitest, Playwright y Surefire.

El número total de pruebas automáticas ejecutadas en la versión actual (MVP final) es:

$$N_{\text{total}} = N_{\text{frontend unitarias}} + N_{\text{frontend E2E}} + N_{\text{backend}} = 138 + 3 + 850 = 991$$

El desglose principal es el siguiente: frontend unitarias con 138 casos de prueba en 21 archivos, frontend E2E con 3 escenarios end-to-end y backend con 850 casos de prueba agregados en 57 suites Surefire. Esta distribución refleja mayor concentración en verificación de servicios.

7.5– Pruebas en frontend

7.5.1. Pruebas unitarias y cobertura

El frontend utiliza Vitest como runner principal, con entorno `happy-dom` y configuración de cobertura por V8. La suite actual incluye pruebas sobre componentes (navegación y constructor de estructura ZIP), contextos de autenticación y tema, páginas clave (login, dashboard, actividades por curso, evaluaciones y perfil), servicios de API (auth, usuarios, cursos, actividades, entregables, entregas, feedback, notificaciones y OneDrive) y utilidades de parsing y validación de estructura ZIP. Esta cobertura prioriza flujos de uso frecuentes y puntos de integración con mayor probabilidad de regresión.

Se han definido umbrales mínimos de cobertura en la configuración de Vitest: `lines` $\geq 95\%$, `functions` $\geq 95\%$, `statements` $\geq 95\%$ y `branches` $\geq 85\%$. Estos umbrales obligan a mantener un nivel de verificación consistente.

7.5.2. Módulos testeados en frontend

Además de la cobertura porcentual, se ha realizado un inventario por módulo de la suite unitaria para dejar evidencia de qué zonas funcionales reciben mayor esfuerzo de verificación.

Módulo frontend	Casos
Servicios (src/services)	51
Componentes (src/components)	30
Utilidades (src/utills)	22
Páginas (src/pages)	16
Contextos (src/context)	11
Raíz de aplicación (src/)	8
Total frontend unitarias	138

Tabla 7.2: Distribución de casos unitarios por módulo en frontend

7.5.3. Pruebas E2E

Las pruebas end-to-end se ejecutan con Playwright sobre Chromium, y validan flujos de interacción completos desde la interfaz.

El enfoque de ejecución adoptado en este proyecto es **E2E mockeada**: el navegador se levanta contra el frontend real y se simulan endpoints API en los escenarios definidos. Esto reduce inestabilidad ambiental y facilita reproducibilidad en CI.

Se validan, entre otros, los escenarios de autenticación correcta, autenticación fallida y flujo de evaluaciones con filtros y descarga mockeada. En la versión actual (MVP final) se ejecutan 3 escenarios y los 3 finalizan en verde; durante la ejecución pueden aparecer avisos de proxy de Vite para endpoints no mockeados de notificaciones, sin impacto en el resultado de la suite.

7.5.4. Mutación en frontend

Stryker se aplica sobre código de producción de servicios y utilidades, excluyendo archivos de test. El objetivo es verificar que los tests no solo ejecutan líneas, sino que detectan alteraciones semánticas en el comportamiento.

La cobertura mediante mutación en frontend se ha realizado con un procedimiento que selecciona el alcance en `src/services/**/*.ts` y `src/utills/**/*.ts`, excluye explícitamente tests y archivos de infraestructura (`*.test.*`, `*.spec.*`, `api.ts`, `index.ts`), ejecuta `npm run test:mutation` generando mutantes y relanzando Vitest por lotes, y analiza mutantes `survived/timeout` para reforzar casos de prueba débiles. Este enfoque prioriza secciones con mayor impacto funcional.

En la versión actual (MVP final), `npm run test:mutation` instrumenta 10 ficheros de servicios y utilidades con 435 mutantes. El resultado global es un *mutation score* del 92.10 %, con 267 mutantes eliminados, 23 supervivientes y 1 *timeout*. Como criterio de aceptación de esta fase, Stryker aplica umbrales `high=75`, `low=60` y `break=50`, siendo este último el que determina fallo de pipeline.

7.6– Pruebas en backend

7.6.1. Pruebas unitarias e integración

En backend se emplea JUnit junto con Spring Test y MockMvc. La validación cubre lógica de servicios de dominio, validaciones y reglas de negocio, contratos REST de controladores, seguridad y autorización por contexto, repositorios sobre H2, entidades y conversiones DTO. Este alcance garantiza consistencia entre reglas de negocio, persistencia y API expuesta.

Entre los tests de referencia de mayor carga funcional destaca `EntregaServiceTest`, que valida escenarios de entrega, restricciones y evolución de estado.

7.6.2. Módulos testeados en backend

El backend se ha cuantificado a partir de nodos `testcase` en reportes generados por Maven Surefire (el plugin oficial de construcción y ejecución de pruebas de Java que orquesta la fase de test en el ciclo de vida del proyecto). Esto permite reflejar correctamente pruebas parametrizadas y escenarios anidados.

Módulo backend	Casos
Servicios (<code>service</code>)	542
Controladores (<code>controller</code>)	149
Repositorios (<code>repository</code>)	47
DTOs y validaciones (<code>dto</code>)	40
Seguridad (<code>security</code>)	31
Entidades (<code>entity</code>)	22
Manejo de excepciones (<code>exception</code>)	10
Configuración (<code>config</code>)	8
Aplicación base (<code>root</code>)	1
Total backend	850

Tabla 7.3: Distribución de casos de prueba por módulo en backend

7.6.3. Cobertura backend

La cobertura se genera con JaCoCo (Java Code Coverage Library, un marco de trabajo de análisis de código abierto para medir cómo el código fuente es ejercitado por los tests automáticos) durante la fase de test, produciendo informe HTML en `backend/target/site/jacoco/index.html`. En la versión actual (MVP final), el reporte agregado indica 89.55 % de instrucciones, 70.31 % de ramas, 90.75 % de líneas y 91.60 % de métodos cubiertos. Este reporte se utiliza como soporte de revisión técnica para detectar módulos subprobados y orientar ampliaciones de suite.

7.6.4. Mutación en backend

El perfil Maven `mutation` ejecuta PIT (PIT Mutation Testing, un sistema de pruebas de mutación para la máquina virtual de Java que altera artificialmente el bytecode de los programas para medir qué porcentaje de esos defectos es capaz de detectar la suite de pruebas) sobre clases objetivo de servicios y controladores seleccionados. La configuración define umbrales de mutación y cobertura, y genera reportes estructurados para su análisis.

En esta implementación se mutan de forma controlada 10 clases objetivo: 7 de servicio y 3 de controlador. La selección cubre integración con OneDrive, validación ZIP, feedback, mapeo de entidades, contexto de seguridad, recuperación de contraseña, Cloudinary y los controladores asociados a OneDrive, OAuth de Microsoft y feedback. Esta selección concentra el esfuerzo de mutación en servicios de integración y flujo crítico.

El flujo seguido para cobertura de mutación en backend consiste en ejecutar el perfil con `mvn -Pmutation ... mutationCoverage`, generar mutantes con operadores de retorno, aritmética e incrementos, relacionar mutante y test para identificar qué prueba mata cada defecto sintético, y revisar mutantes supervivientes para crear pruebas adicionales o ajustar aserciones. Este proceso mejora la eficacia real de la suite.

En la versión actual (MVP final), PIT genera 144 mutantes, de los que 134 son eliminados por la suite. El resultado es un *mutation score* del 93 %, una cobertura de línea del 95 % sobre clases mutadas y una fuerza de test del 96 %. El perfil exige además un mínimo de mutación del 70 % y de cobertura del 75 %, reforzando que la cobertura no sea solo de líneas, sino de detección efectiva de fallos.

7.7– Automatización en CI/CD

La validación se integra en pipelines de GitHub Actions, lo que garantiza ejecución repetible por cada cambio relevante.

Workflows principales: `frontend-ci.yml` para lint, unitarias con cobertura, E2E y build de frontend; `full-stack-ci.yml` para verificación backend (`mvn verify`) y frontend completo; y `mutation-ci.yml` para mutación backend (PIT) y frontend (Stryker) con subida de artefactos. Esta organización separa capas de validación sin perder coherencia global.

Esta automatización reduce el riesgo de integración tardía, y permite detectar regresiones antes de integrar código en la rama principal y desplegarlo en producción.

7.8– Resultados verificados

De acuerdo con el informe de estrategia de testing del proyecto (corte de verificación final), se obtienen los siguientes resultados representativos:

Métrica	Resultado
E2E frontend (Playwright)	3 escenarios ejecutados, 3 en verde.
Cobertura frontend	Statements 97.78 %, Branches 90.68 %, Functions 98.20 %, Lines 98.25 %.
Suite frontend	138 tests aprobados en 21 archivos durante la ejecución del comando <code>test:coverage</code> .
Suite backend completa	850 casos en 57 suites Surefire, 0 fallos y 0 errores.
Total pruebas automáticas	991 ejecuciones (138 unitarias frontend + 3 E2E + 850 backend).
Mutación frontend (Stryker)	Score global 92.10 % sobre 435 mutantes (por encima del umbral <code>break=50</code>).
Mutación backend (PIT)	144 mutantes generados, mutation score 93 %, line coverage 95 %, test strength 96 %.
Test objetivo backend	<code>EntregaServiceTest</code> : 89 tests, 0 fallos, 0 errores.

Tabla 7.4: Resumen de resultados de verificación

Estos valores muestran una madurez de testing consistente con el nivel de exigencia de un TFG de orientación profesional.

7.9– Criterios de aceptación

Como criterio operativo de calidad se establecen mínimos de lint sin errores, suites unitarias, integración y E2E en verde, coberturas por encima de umbrales configurados, métricas de mutación por encima de umbral de ruptura y ausencia de regresiones funcionales en flujos críticos. Estos criterios permiten cierre objetivo de iteraciones.

7.10– Limitaciones y riesgos

Aunque la estrategia es robusta, se identifican líneas de mejora como ampliar E2E con backend real en entorno de staging para ciertos casos, extender mutación a mayor superficie de clases backend y reforzar pruebas de carga y comportamiento en condiciones extremas. Estas mejoras consolidarían la robustez en escenarios de producción.

No obstante, las limitaciones actuales no comprometen la validez de la verificación para el alcance definido del proyecto.

7.11– Conclusiones

La estrategia de pruebas aplicada combina cobertura, profundidad y automatización, aportando evidencia objetiva de calidad funcional y técnica.

La incorporación de mutación como criterio adicional permite evaluar la eficacia real de la suite, superando una visión meramente cuantitativa de cobertura.

En conjunto, los resultados respaldan que el sistema dispone de una base de validación sólida para mantenimiento, evolución y despliegue continuo.

Manual de usuario

8.1– Introducción

Este capítulo describe el uso operativo del sistema desde la perspectiva de sus usuarios finales. El objetivo no es detallar la implementación interna, sino proporcionar una guía clara para ejecutar las tareas más frecuentes de forma correcta y segura.

El sistema dispone de tres perfiles principales: **administrador**, **profesor** y **estudiante**. Cada perfil accede a vistas y operaciones diferentes según permisos.

8.2– Acceso al sistema

8.2.1. Requisitos previos de uso

Antes de operar con normalidad en la plataforma, se recomienda verificar que el navegador está actualizado en versión estable, que existe conectividad de red suficiente durante la subida de archivos de gran tamaño, que se utilizan credenciales personales no compartidas y que el perfil asignado (administrador/profesor/estudiante) coincide con el rol esperado. Estas condiciones previas reducen incidencias de sesión y errores de permisos.

Estas precondiciones reducen incidencias de sesión, errores de permisos y fallos de subida de materiales.

8.2.2. Pantallas de acceso

El acceso se realiza desde la pantalla de inicio de sesión y las rutas funcionales principales son `/login` para iniciar sesión, `/forgot-password` para solicitar recuperación de contraseña y `/reset-password` para establecer una nueva contraseña mediante token. Estas rutas estructuran el ciclo de acceso y recuperación de cuenta.

Tras autenticarse correctamente, el usuario es redirigido al panel principal (`/dashboard`). Si el usuario ya tiene sesión activa, la ruta de inicio redirige automáticamente al panel.

8.2.3. Ciclo de sesión

La sesión se basa en token de acceso y token de refresco. Cuando el token de acceso expira, el cliente intenta renovarlo automáticamente. Si la renovación falla, la sesión se cierra y el usuario debe autenticarse de nuevo.

8.2.4. Recuperación de contraseña

El flujo de recuperación consiste en que el usuario solicita recuperación indicando su correo, el sistema emite un token temporal y envía instrucciones de restablecimiento, y el usuario accede al formulario de nueva contraseña para introducir token y clave nueva. Este procedimiento reduce riesgo de accesos indebidos y mantiene trazabilidad del cambio.

8.3– Perfiles y permisos

La Tabla 8.1 resume capacidades por perfil.

Perfil	Operaciones principales
Administrador	Gestionar usuarios, cursos y grupos; asignar o retirar roles; mantener estructura académica.
Profesor	Crear/editar actividades; crear/editar entregables; revisar entregas; calificar; publicar/ocultar notas.
Estudiante	Consultar cursos y actividades visibles; realizar entregas; consultar historial, feedback y calificaciones según visibilidad.

Tabla 8.1: Capacidades por perfil de usuario

8.4– Vistas comunes a todos los perfiles

Las siguientes pantallas son compartidas por los tres perfiles del sistema:

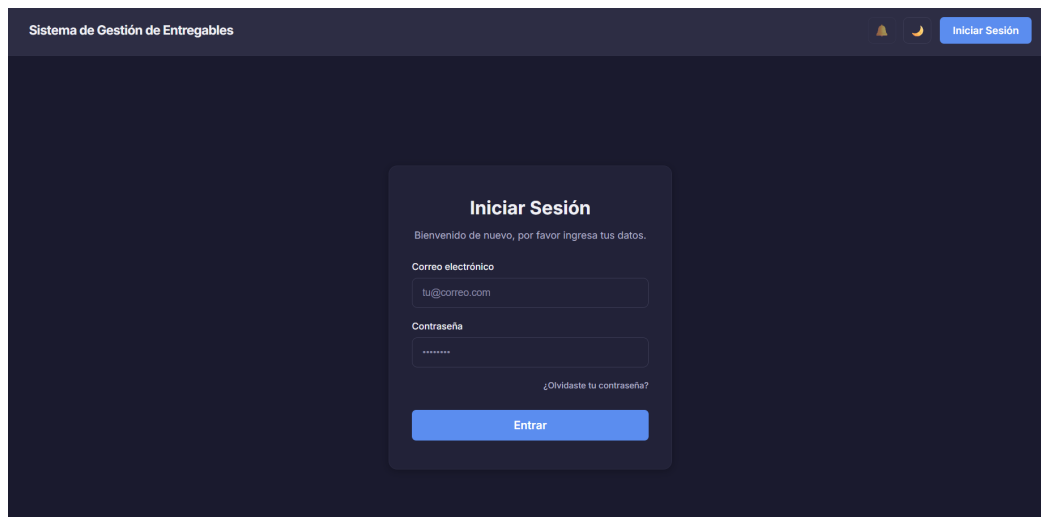


Figura 8.1: Pantalla de inicio de sesión. Vista compartida por todos los perfiles.



Figura 8.2: Sección de perfil (1/2). Vista compartida por todos los perfiles.



Figura 8.3: Sección de perfil (2/2). Vista compartida por todos los perfiles.

8.5– Guía de uso para profesorado

8.5.1. Vistas principales

El perfil de profesorado accede a vistas de panel, cursos, actividades, entregables y evaluaciones. Las capturas se agrupan en las subsecciones siguientes para mantener relación directa entre imagen y tarea.

8.5.2. Crear y configurar una actividad

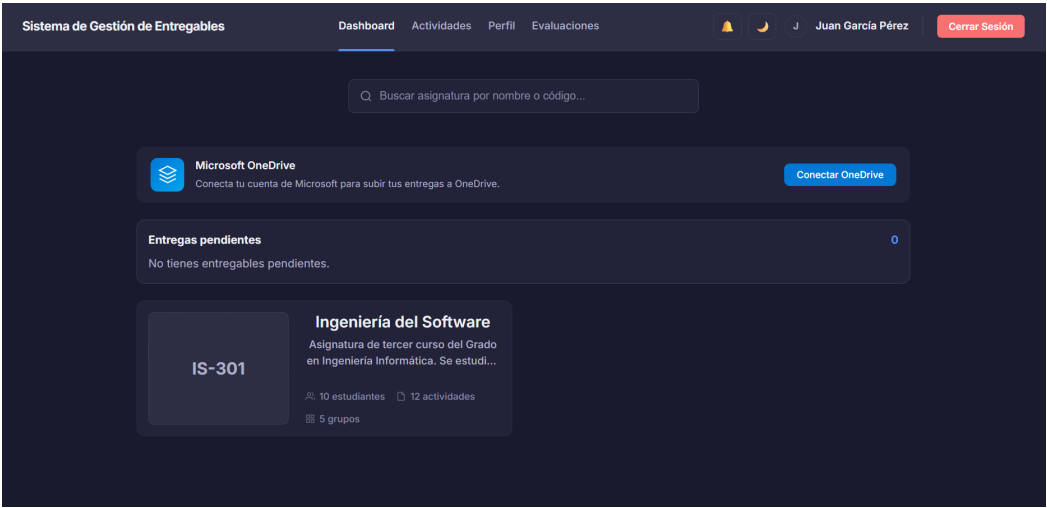


Figura 8.4: Dashboard docente. Secciones visibles: dashboard, actividades, perfil y evaluaciones.

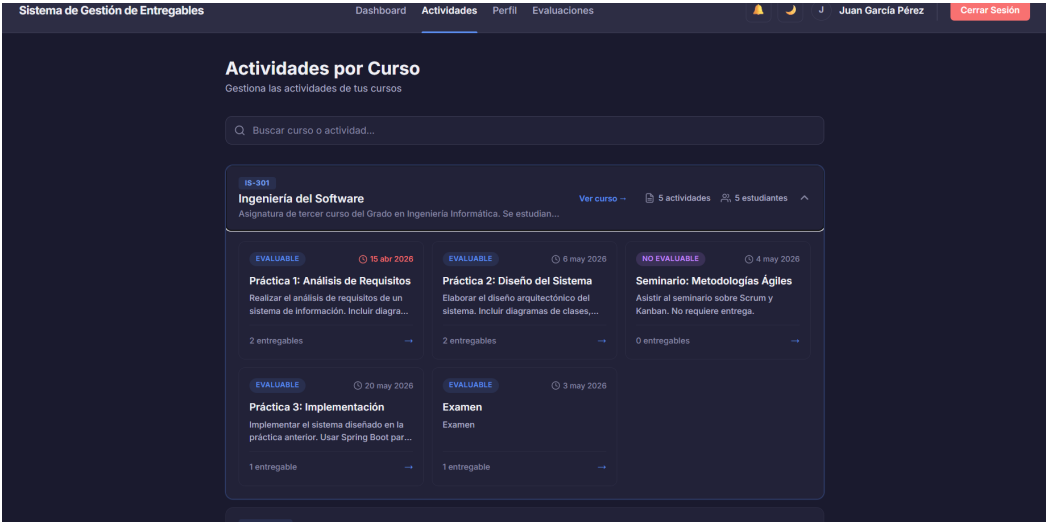


Figura 8.5: Sección de actividades por curso.

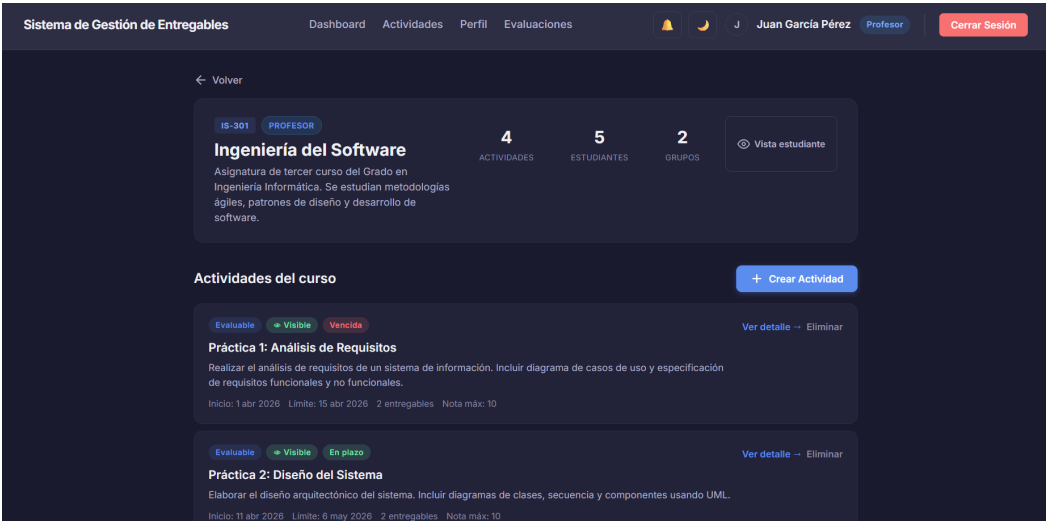


Figura 8.6: Detalle de asignatura con profesorado asignado.

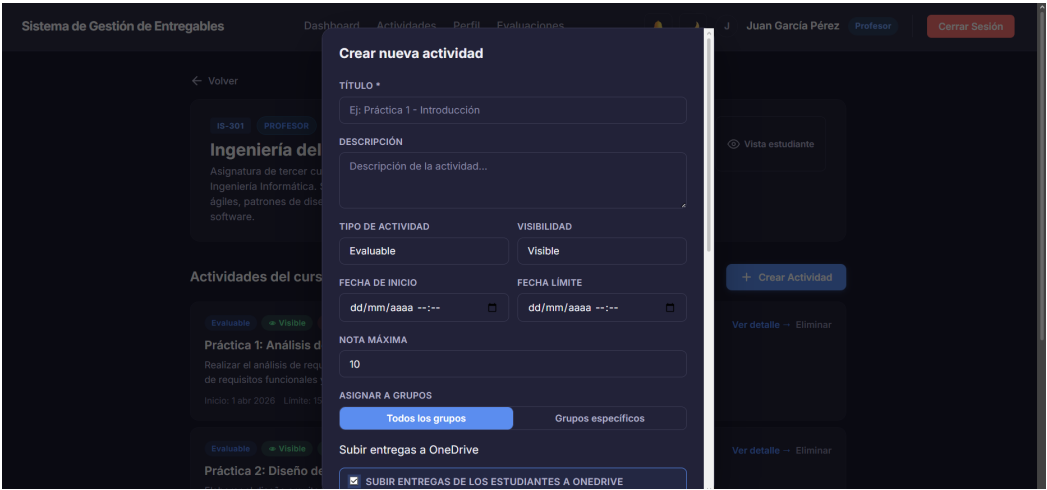


Figura 8.7: Creación de actividad (1/2).

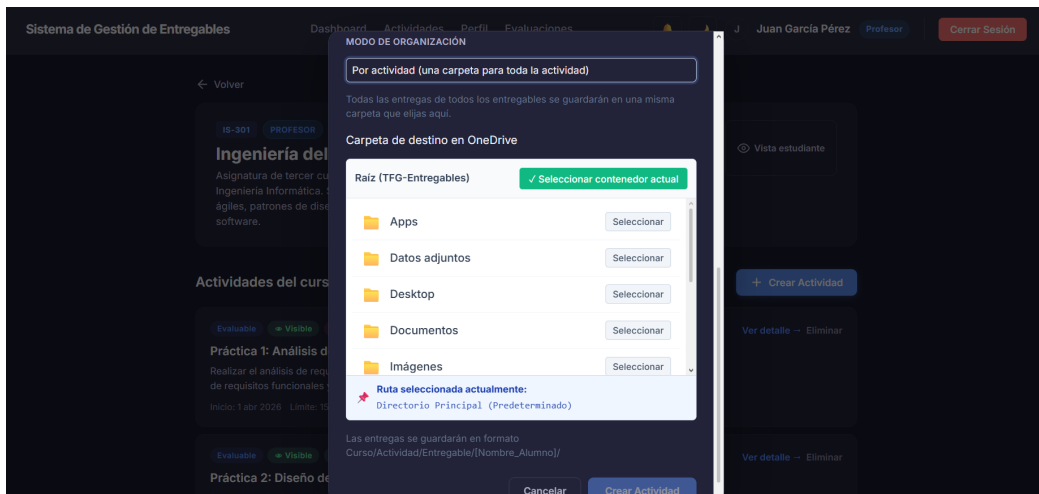


Figura 8.8: Creación de actividad (2/2).

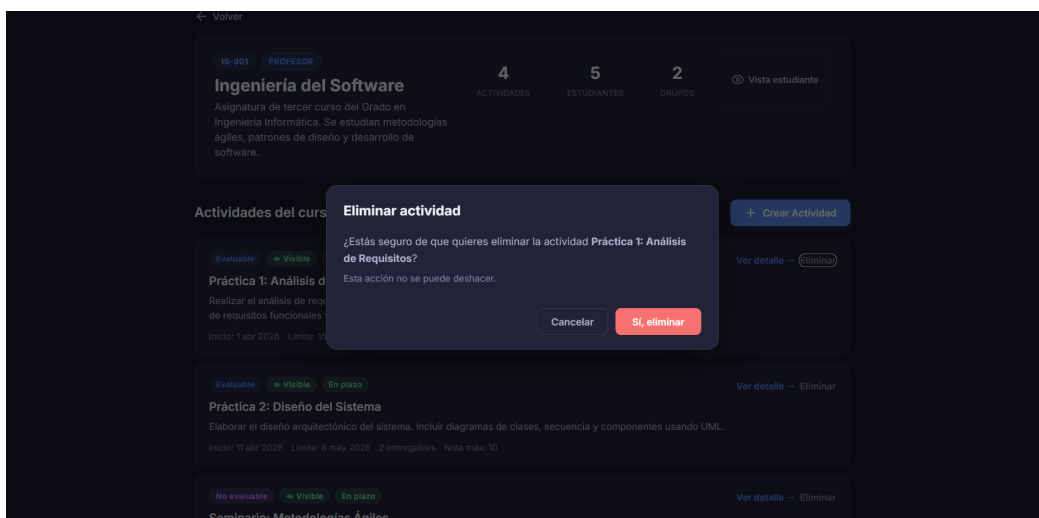


Figura 8.9: Eliminación de actividad desde la asignatura.

El flujo recomendado consiste en acceder a `/cursos` y seleccionar un curso, crear la actividad desde el detalle del curso, definir título, descripción, fechas y visibilidad, y asignar grupos destinatarios cuando aplique. Esta secuencia garantiza que la actividad queda contextualizada antes de publicarse.

Las validaciones clave establecen que el título es obligatorio, que la fecha de inicio no puede ser posterior a la fecha límite y que la visibilidad condiciona si el alumnado puede ver la actividad. Estas reglas previenen errores de configuración y evitan confusiones en el calendario docente.

8.5.3. Crear y editar entregables

Desde la actividad, el profesor puede crear entregables desde la vista de alta de entregables del detalle de actividad.

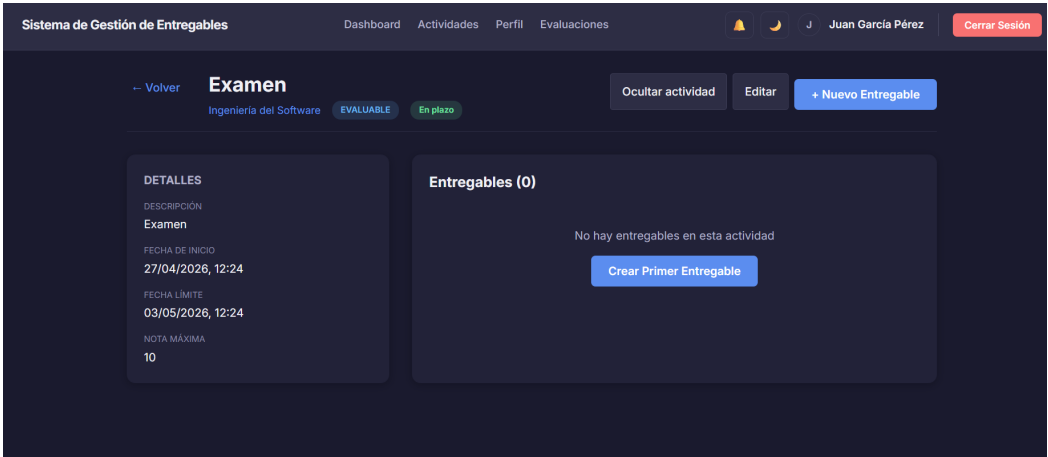


Figura 8.10: Detalle de actividad y entregables asociados.

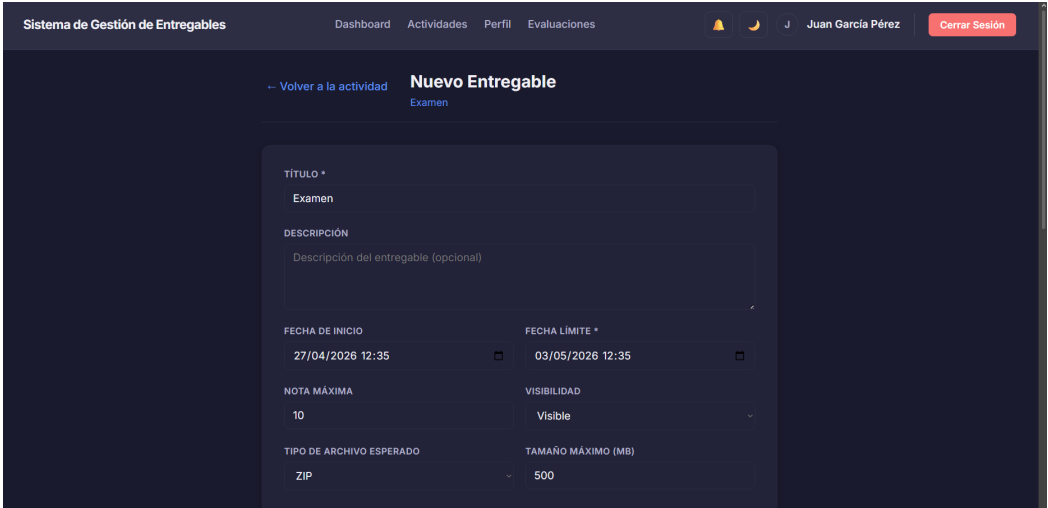


Figura 8.11: Creación de entregables (1/2).

Figura 8.12: Creación de entregables (2/2).

Los parámetros relevantes de configuración incluyen título, descripción y fecha límite, el tipo de archivo esperado según el catálogo `TipoMaterial` (PDF, DOCX, ZIP, RAR, TXT, IMAGEN, VIDEO, ENLACE, SOLO_TEXTO u OTRO), el tamaño máximo permitido, la posibilidad de reenvío y la visibilidad de notas para estudiantes. Esta configuración determina el comportamiento de validación y la comunicación con el alumnado.

Cuando el tipo esperado es ZIP, puede definirse estructura esperada y validación estricta, lo que obliga a que la entrega siga el formato indicado.

Figura 8.13: Edición de actividad (1/3).

NOTA MÁXIMA

10

GRUPOS ASIGNADOS

Todos los grupos Grupos específicos

SUBIR ENTREGAS A ONEDRIVE

☒ LAS ENTREGAS DE LOS ESTUDIANTES SE SUBIRÁN AUTOMÁTICAMENTE A TU ONEDRIVE

MODO DE ORGANIZACIÓN

Por actividad (una carpeta para toda la actividad)

Todas las entregas de todos los entregables se guardarán en una misma carpeta que elijas aquí.

CARPETA DE DESTINO EN ONEDRIVE

Raíz (TFG-Entregables) ✓ Seleccionar contenedor actual

Apps	Seleccionar
Datos adjuntos	Seleccionar
Desktop	Seleccionar
Documentos	Seleccionar
Imágenes	Seleccionar

Figura 8.14: Edición de actividad (2/3).

MODO DE ORGANIZACIÓN

Por actividad (una carpeta para toda la actividad)

Todas las entregas de todos los entregables se guardarán en una misma carpeta que elijas aquí.

CARPETA DE DESTINO EN ONEDRIVE

Raíz (TFG-Entregables) ✓ Seleccionar contenedor actual

Apps	Seleccionar
Datos adjuntos	Seleccionar
Desktop	Seleccionar
Documentos	Seleccionar
Imágenes	Seleccionar

Ruta seleccionada actualmente:
Pruebas TFG/Prueba 3

Las entregas se guardarán en "Pruebas TFG/Prueba 3/[Nombre_Entregable]/[Nombre_Alumno]"

Eliminar actividad Cancelar Guardar cambios

Figura 8.15: Edición de actividad (3/3).

8.5.4. Revisar entregas y evaluar

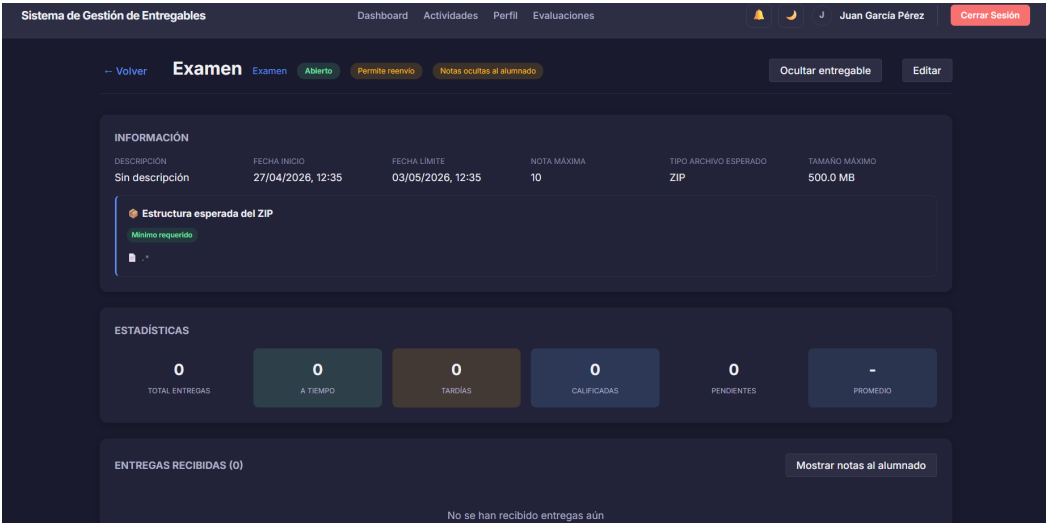


Figura 8.16: Detalle de entregable para revisiones.

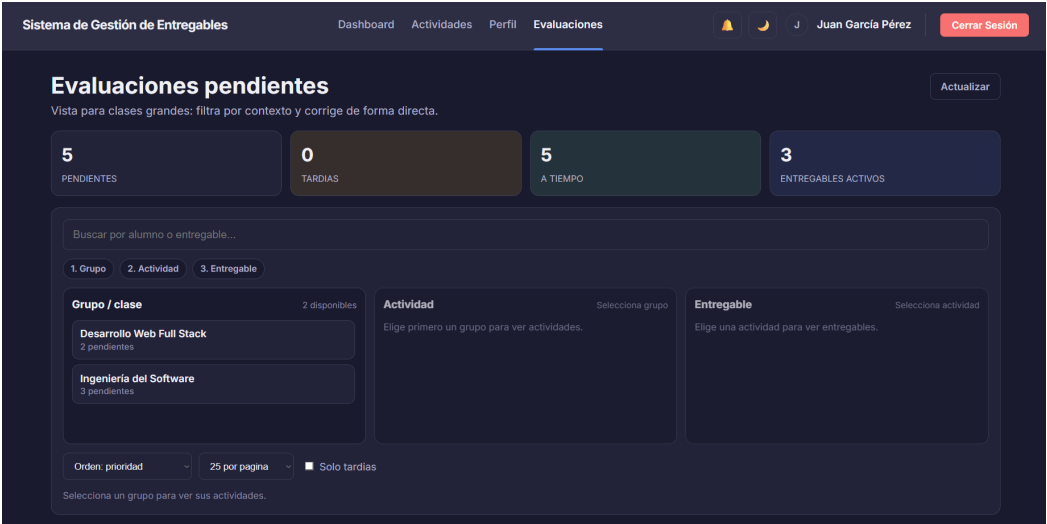


Figura 8.17: Bandeja de evaluaciones (1/2).

Mostrando 5 de 5 entregas pendientes. Actualizado: 12:51:21

Carga por entregable

Selecciona una actividad

Selecciona un grupo para continuar.

ENTREGABLE	ALUMNO	GRUPO	ENTREGA	ESTADO	VERSION	ACCIONES
Documento ERS	Miguel Ruíz Navarro miguel.ruiz@ull.edu.es	Grupo B - Tarde	12/04/2026, 12:05 hace 17 días	A TIEMPO	v1	Ver y calificar
Hito 1: Backend API REST	Pedro Sánchez Ramos pedro.sanchez@ull.edu.es	Grupo Prácticas	26/04/2026, 12:05 hace 3 días	A TIEMPO	v1	Ver y calificar
Diagrama de Clases UML	Ana Fernández Torres ana.fernandez@ull.edu.es	Grupo A - Mañana	27/04/2026, 12:05 hace 2 días	A TIEMPO	v1	Ver y calificar
Hito 1: Backend API REST	Laura Díaz Molina laura.diaz@ull.edu.es	Grupo Prácticas	27/04/2026, 12:05 hace 2 días	A TIEMPO	v1	Ver y calificar
Diagrama de Clases UML	Pedro Sánchez Ramos pedro.sanchez@ull.edu.es	Grupo A - Mañana	28/04/2026, 12:05 hace 1 días	A TIEMPO	v1	Ver y calificar

Página anterior

Página 1 de 1

Página siguiente

Figura 8.18: Bandeja de evaluaciones (2/2).

Sistema de Gestión de Entregables Dashboard Actividades Perfil Evaluaciones Juan García Pérez Cerrar Sesión

-- Volver

Backend Spring Boot - Laura

Hito 1: Backend API REST ENTREGADO v1

Estudiante

Laura Díaz Molina

Fecha de entrega

27/04/2026, 12:05 ✓ A tiempo

Calificación

Sin evaluar

Archivos (1)

backend-laura.zip 17.2 MB Ver Descargar

Calificar Entrega

Nota * (rango 0 - 4)

Ej: 8.5

Comentario / Feedback (opcional)

Escribe un comentario para el alumno...

0/5000

Cancelar Guardar Calificación

Figura 8.19: Evaluación de la actividad de un alumno.

Las entregas pueden revisarse desde /entregables/:id (listado por entregable) o desde /evaluaciones (bandeja de pendientes de calificación). Esta doble entrada permite organizar la corrección por entregable o por cola de evaluación.

Las acciones habituales incluyen filtrar por curso, actividad, entregable y estado temporal, abrir el detalle de entrega (/entregas/:id), revisar archivos, comentario y metadatos, registrar nota y feedback, y publicar u ocultar la visibilidad de la nota al estudiante. Este recorrido garantiza una evaluación consistente y documentada.

El sistema permite descargar entregas de forma agregada en ZIP, tanto a nivel de entregable como de actividad completa.

8.5.5. Flujo docente recomendado de extremo a extremo

Para maximizar consistencia y reducir retrabajo, se propone una secuencia de trabajo docente que comienza con la creación de la actividad con fechas y grupos delimitados, continúa con la publicación de entregables con requisitos de formato y tamaño explícitos, y se sostiene con revisiones periódicas del estado de entregas pendientes. A partir de ahí se recomienda corregir por lotes homogéneos (por grupo o por entregable), registrar feedback accionable con criterio de nota trans-

parente y publicar calificaciones cuando el bloque de corrección esté validado. Este flujo mejora la trazabilidad y evita inconsistencias entre estado de evaluación y comunicación al alumnado.

8.6– Guía de uso para estudiantes

8.6.1. Vistas principales

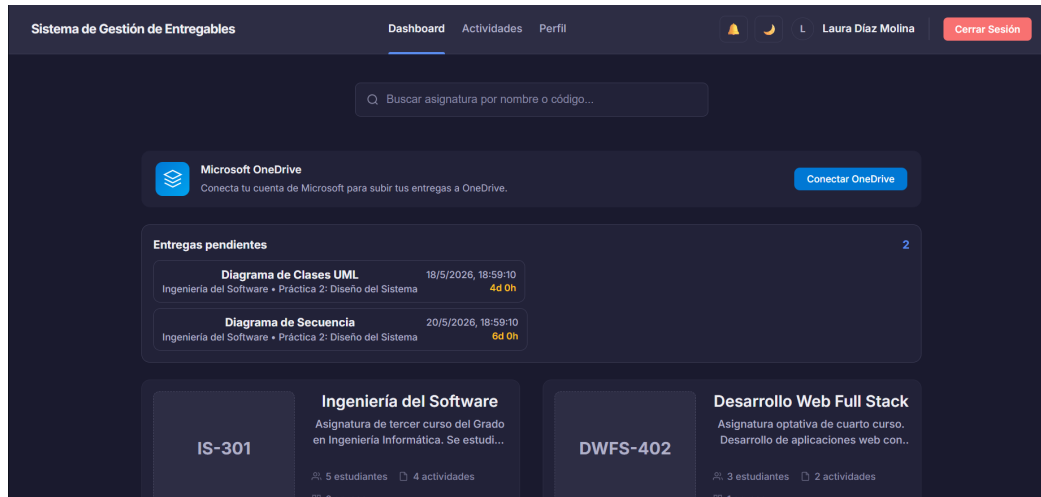


Figura 8.20: Dashboard de estudiante. Secciones visibles: panel principal, actividades, perfil y calificaciones.

El panel de estudiante concentra el acceso a cursos, actividades visibles, entregables pendientes, perfil y calificaciones publicadas.

8.6.2. Consultar actividades y entregables

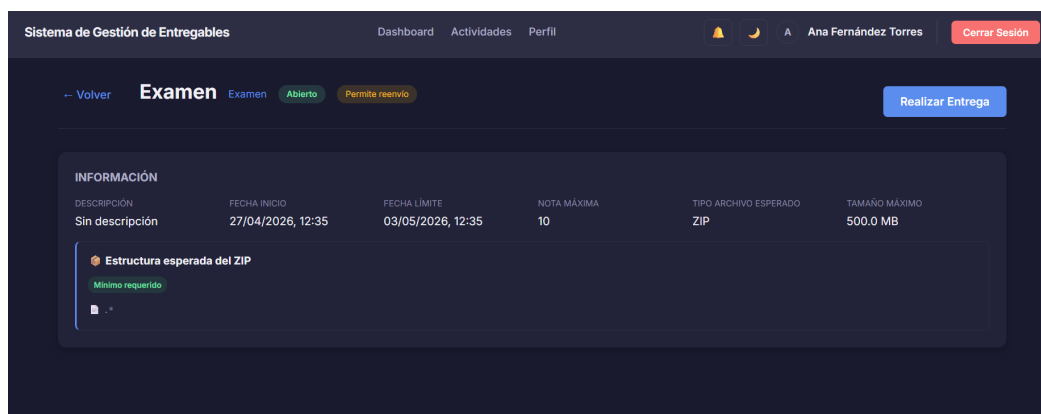


Figura 8.21: Detalle de entregable desde la visión de estudiante.

El flujo recomendado consiste en acceder al panel y entrar en `/cursos`, seleccionar el curso para revisar actividades visibles y entrar en el detalle de actividad para localizar entregables disponibles. Este recorrido asegura que el estudiante actúa dentro del contexto docente correcto.

La visibilidad de actividad/entregable depende de la configuración docente y de la pertenencia del estudiante a grupos autorizados.

8.6.3. Realizar una entrega

La subida se realiza en `/entregables/:id/entregar`. El formulario adapta su comportamiento al tipo esperado configurado en el entregable. Dicho tipo se corresponde con `TipoMaterial` del backend, que contempla **PDF**, **DOCX**, **ZIP**, **RAR**, **TXT**, **IMAGEN**, **VIDEO**, **ENLACE**, **SOLO_TEXTO** y **OTRO**. En **SOLO_TEXTO** no se admiten archivos y se requiere comentario; en **ENLACE** el contenido debe indicarse en el comentario; en **ZIP** puede exigirse archivo comprimido con validación de estructura; y en el resto de tipos se valida extensión, tamaño máximo y coherencia con la configuración docente. Esta adaptación previene errores antes de enviar.

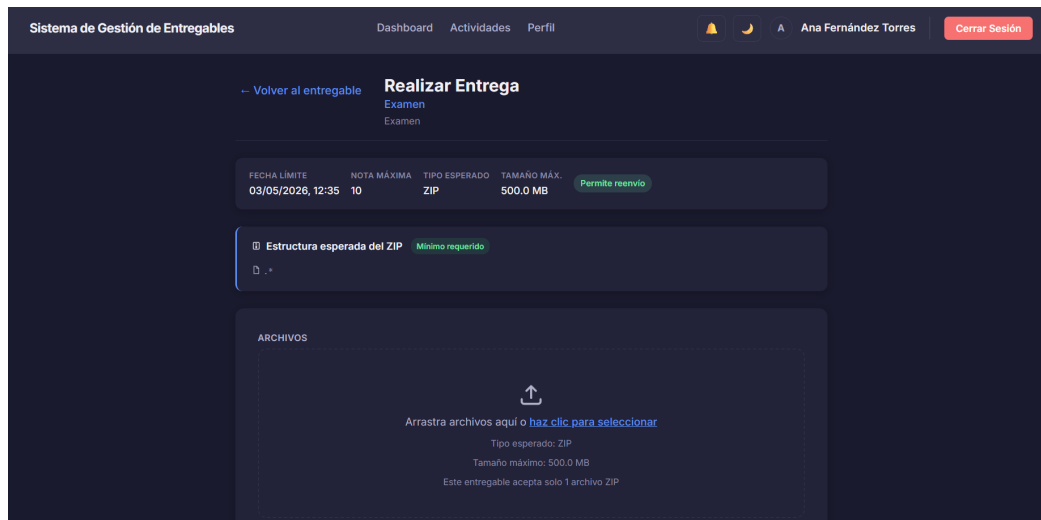


Figura 8.22: Formulario de entrega (1/2).

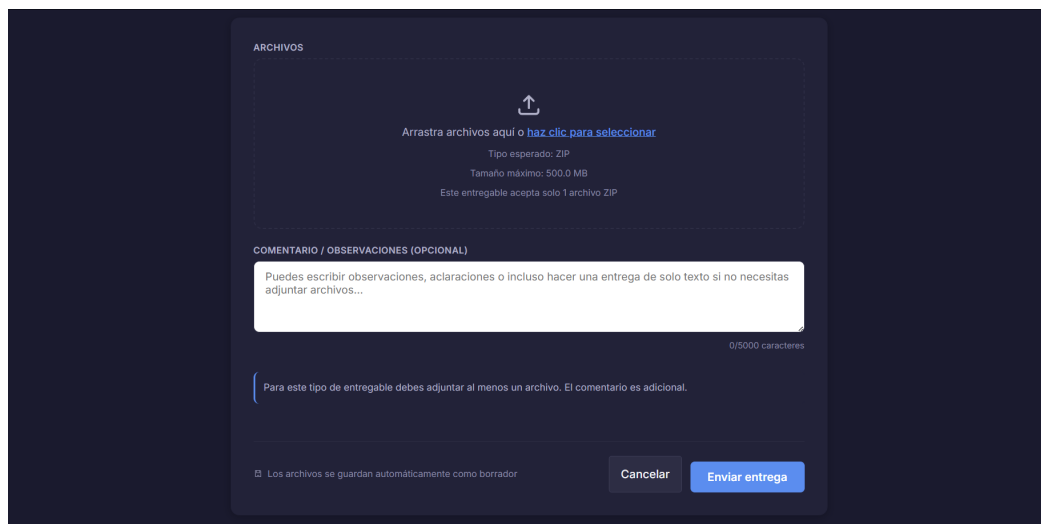


Figura 8.23: Formulario de entrega (2/2).

Además, el formulario incluye guardado temporal en local (borrador), lo que reduce riesgo de pérdida de trabajo si se interrumpe la sesión o falla la conexión.

8.6.4. Recomendaciones para entregas sin incidencias

Para minimizar errores de validación, se recomienda al estudiante revisar el tipo de entrega requerido antes de preparar el material, verificar nombre y extensión de archivos antes de subir, evitar subir archivos en el último minuto para mitigar riesgos de red y confirmar en la vista de detalle que la versión final ha quedado registrada. Estas prácticas reducen incidencias y retrabajo.

En entregas de tipo ZIP, si existe estructura obligatoria, conviene validar localmente el contenido antes de la subida.

8.6.5. Consultar historial y feedback

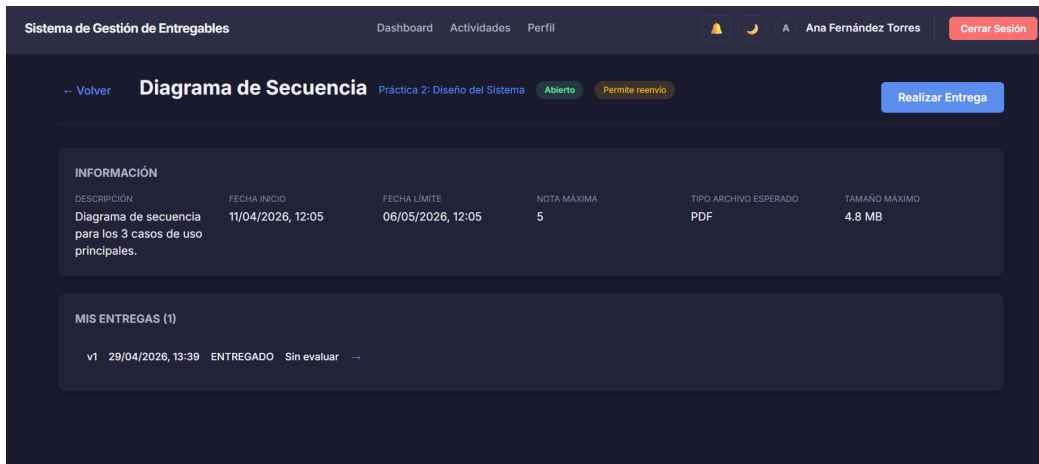


Figura 8.24: Entrega registrada correctamente.

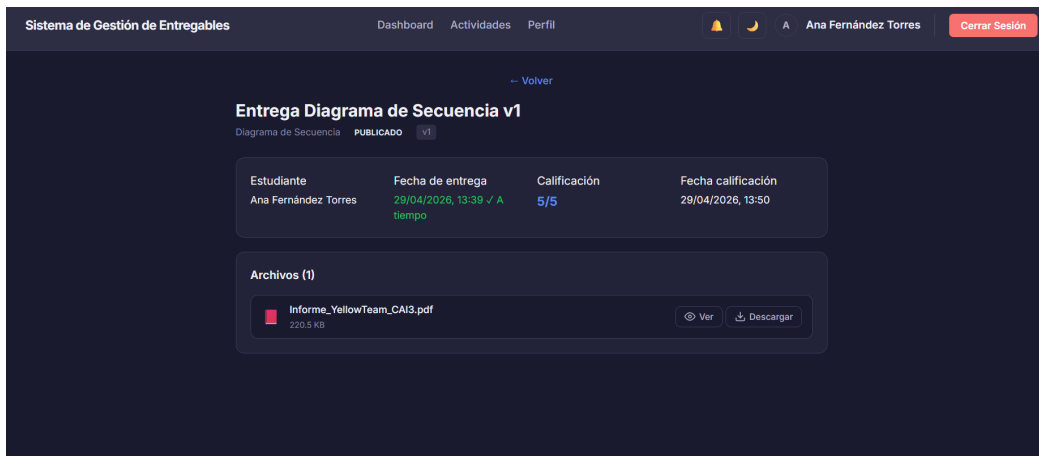


Figura 8.25: Consulta de calificación por estudiante.

En el detalle de entrega (/entregas/:id) el estudiante puede revisar versiones entregadas, descargar o previsualizar materiales y consultar feedback y nota cuando estén publicadas. Esta vista centraliza la evidencia de entrega y su estado evaluativo.

Si la nota no es visible, la entrega aparece sin calificación pública, aunque internamente ya haya sido evaluada por el profesor.

8.7– Guía de uso para administración

8.7.1. Administración de usuarios

El bloque de usuarios permite revisar el listado principal, crear nuevas identidades, modificar datos y retirar cuentas cuando proceda.

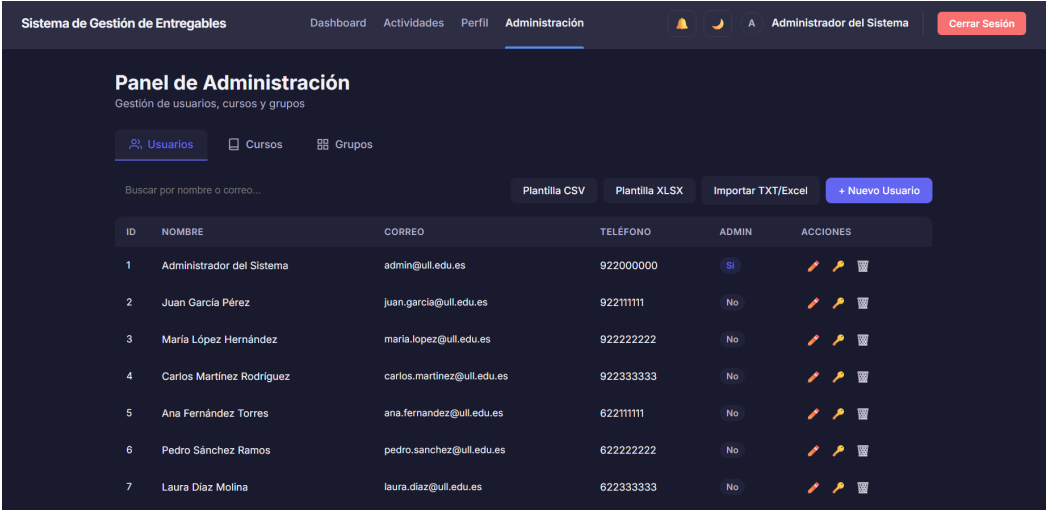


Figura 8.26: Panel de administración. Secciones visibles: usuarios, cursos y grupos.

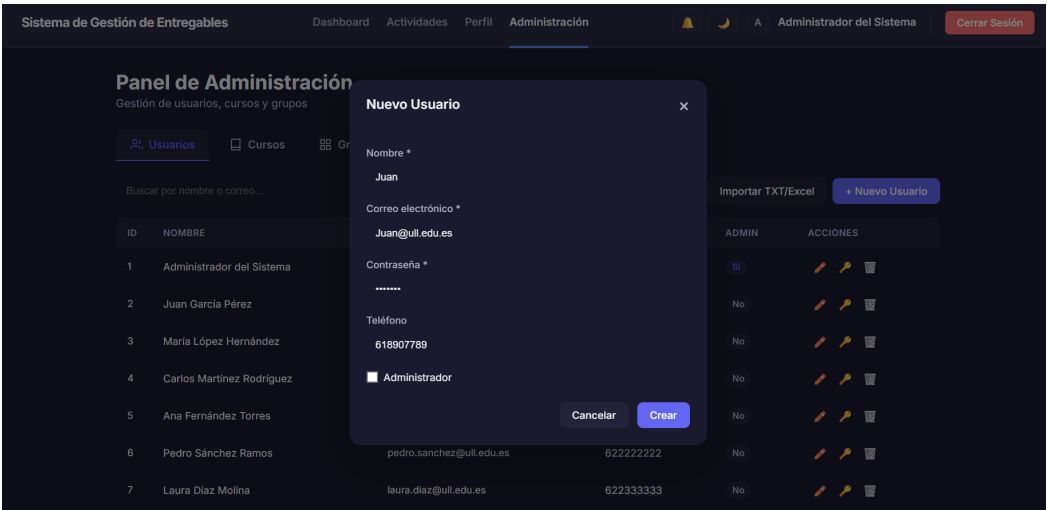


Figura 8.27: Creación de usuarios.

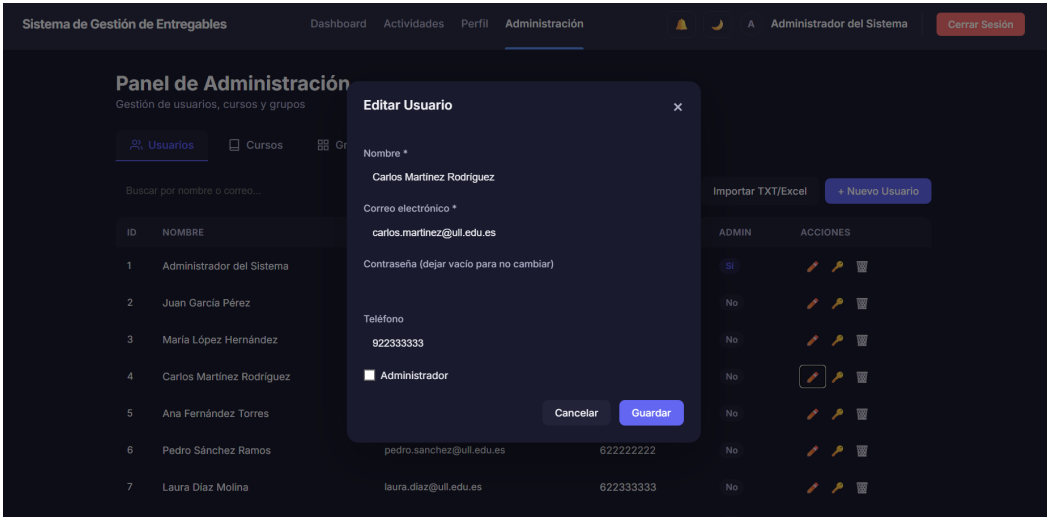


Figura 8.28: Modificación de usuarios.

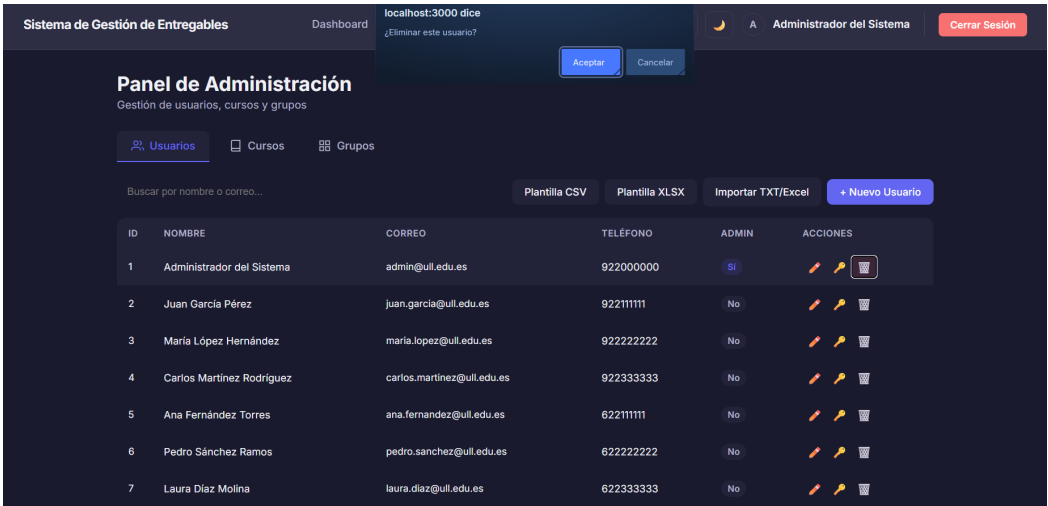


Figura 8.29: Eliminación de usuarios.

8.7.2. Administración de cursos

El bloque de cursos agrupa alta, edición, eliminación y asignación de profesorado responsable.

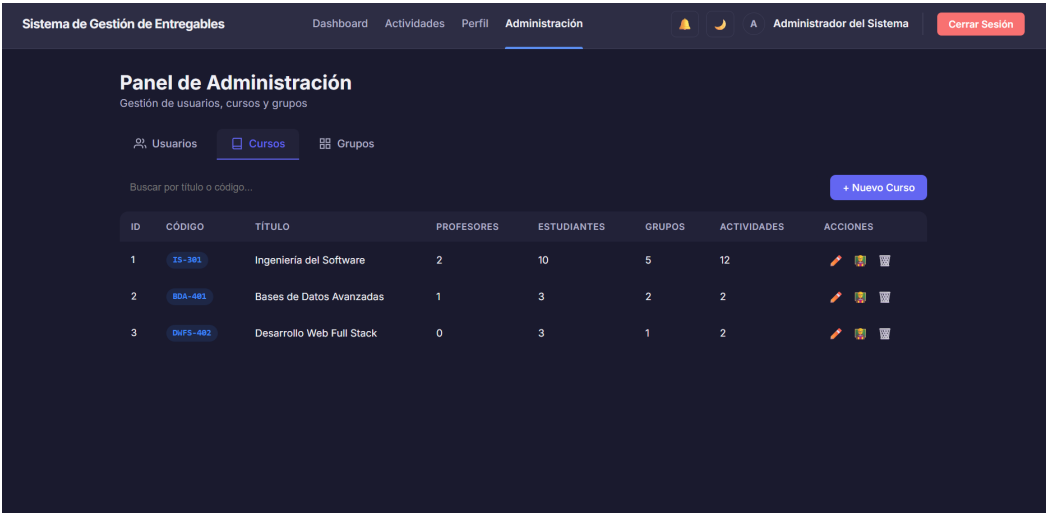


Figura 8.30: Gestión de cursos.

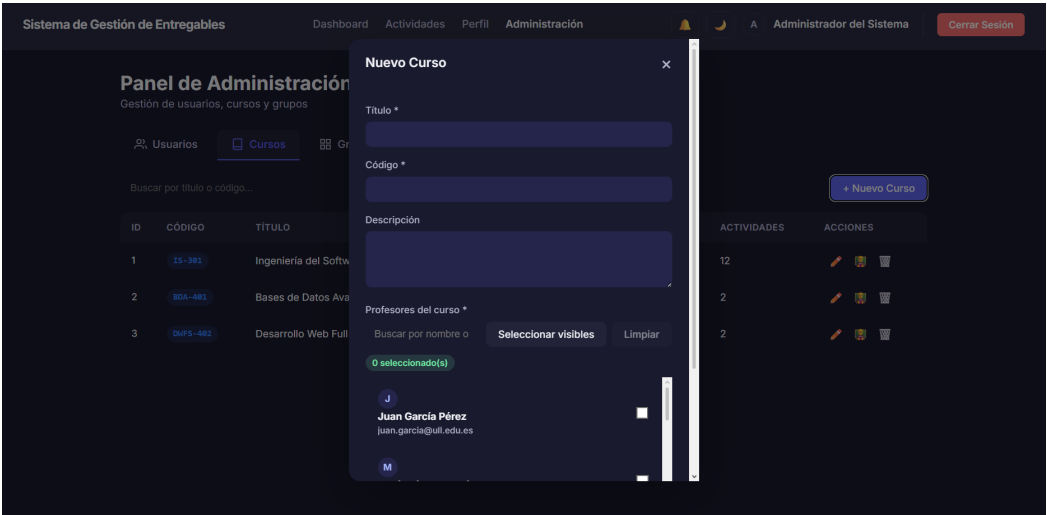


Figura 8.31: Creación de curso (1/2).

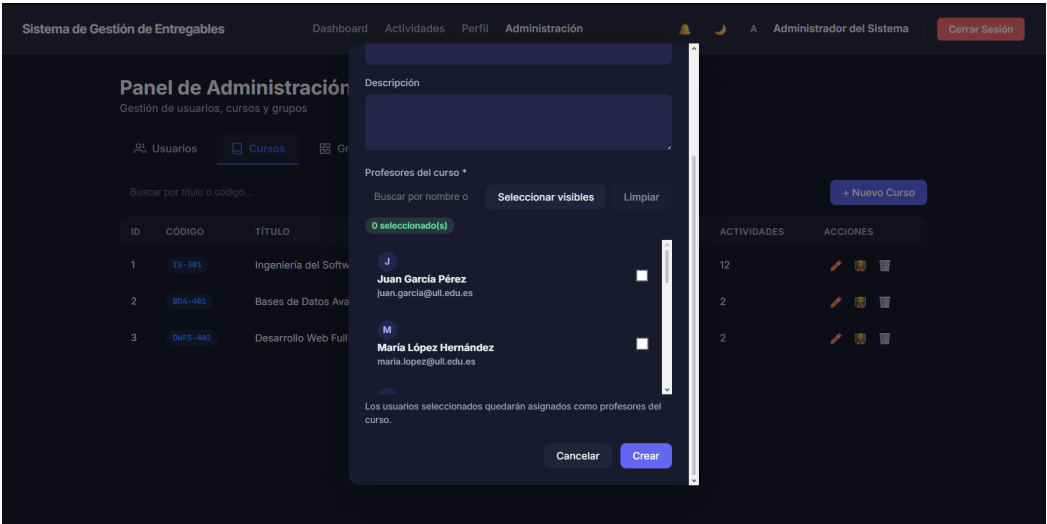


Figura 8.32: Creación de curso (2/2).

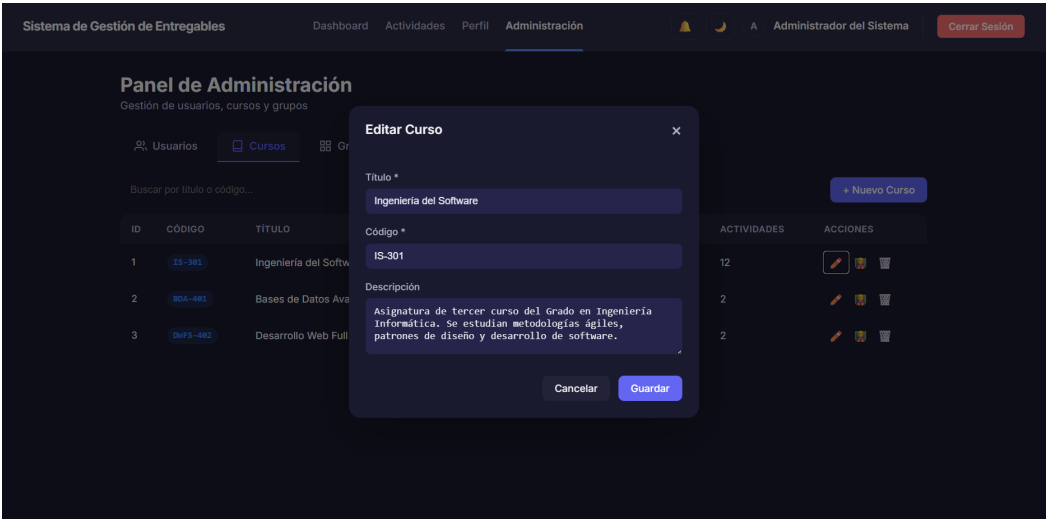


Figura 8.33: Edición de curso.

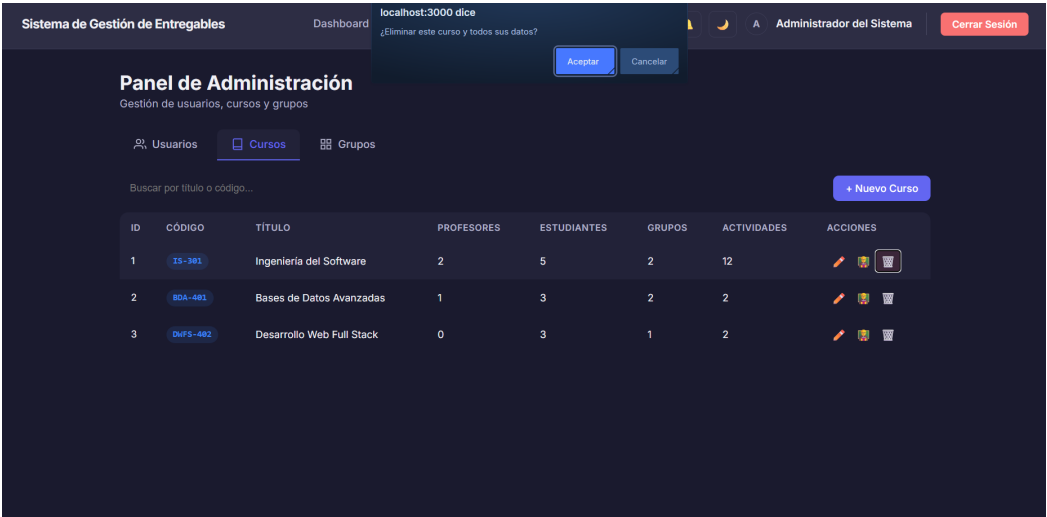


Figura 8.34: Eliminación de curso.

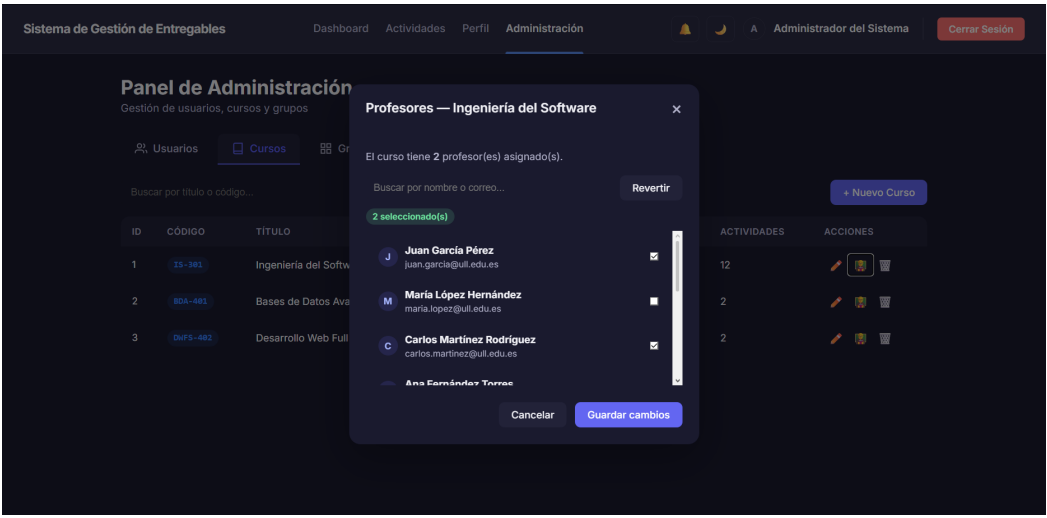


Figura 8.35: Asignación de profesorado a cursos.

8.7.3. Administración de grupos

El bloque de grupos permite crear agrupaciones académicas, mantener sus datos y asignar usuarios a cada grupo.

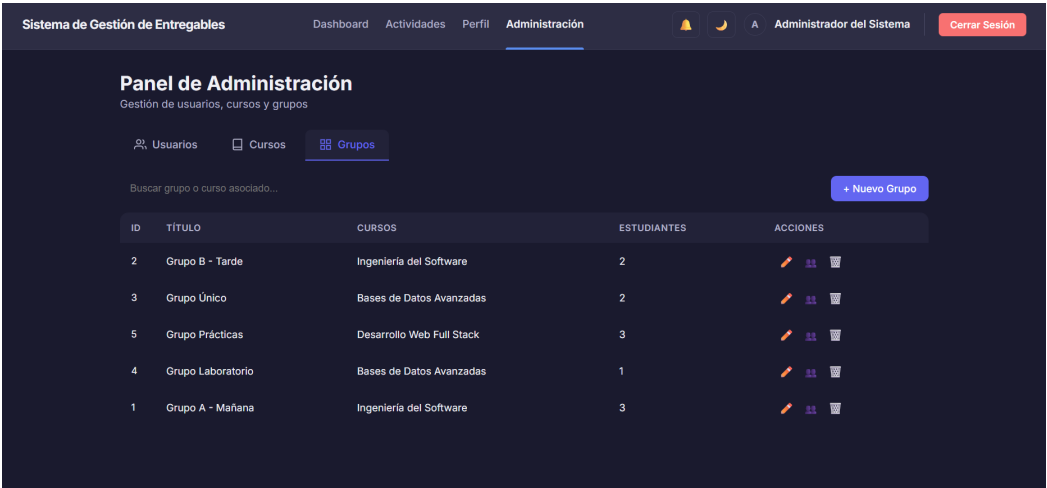


Figura 8.36: Gestión de grupos.

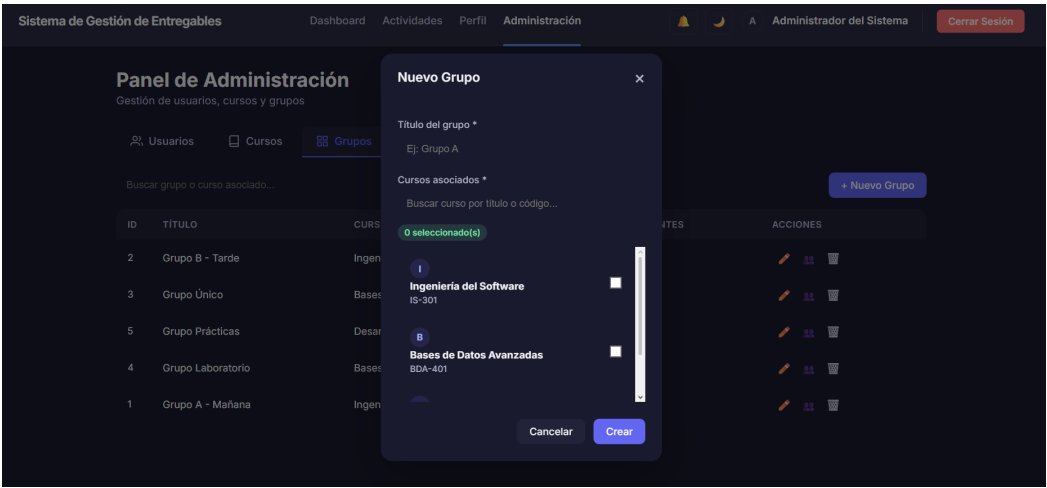


Figura 8.37: Creación de grupo.

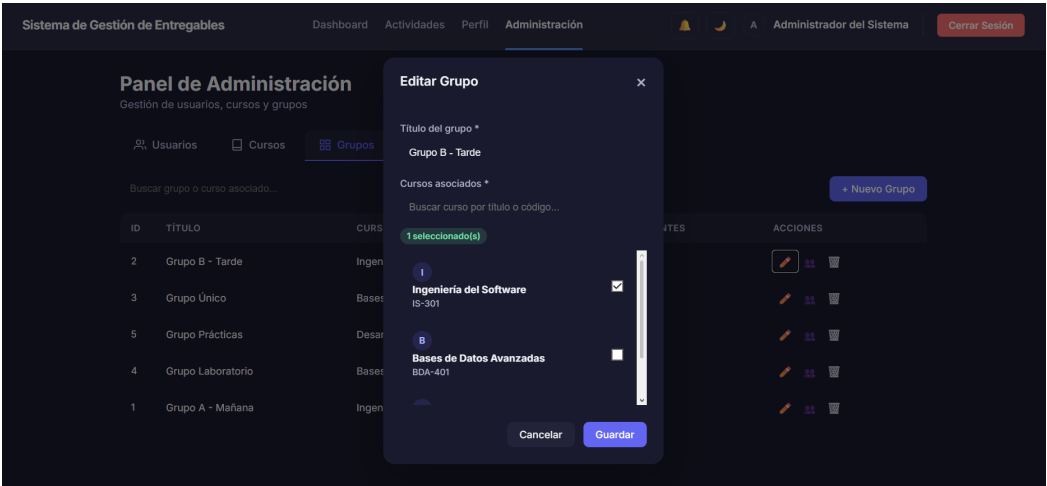


Figura 8.38: Edición de grupo.



Figura 8.39: Eliminación de grupo.

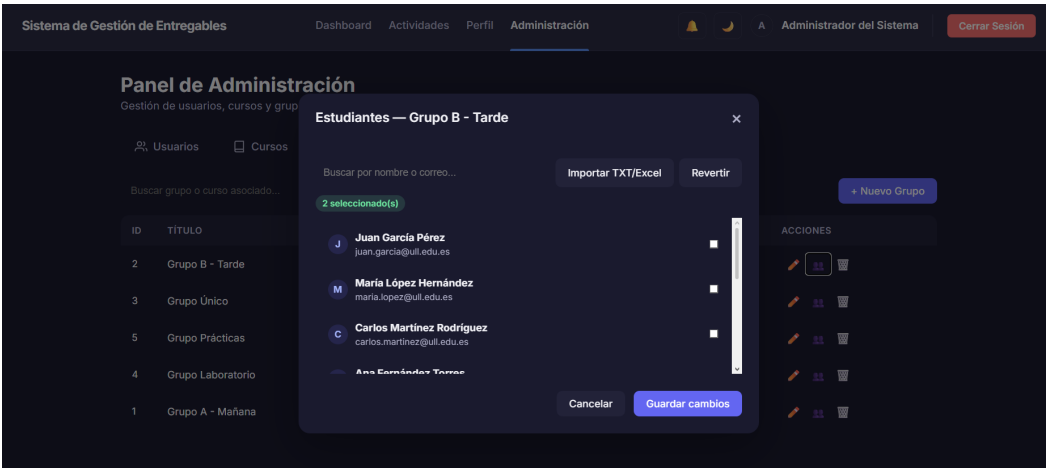


Figura 8.40: Asignación de usuarios en grupo.

8.7.4. Panel de administración

El panel /admin está organizado en tres pestañas principales: **Usuarios** para alta, edición, eliminación y asignación de roles, **Cursos** para creación y mantenimiento de cursos, y **Grupos** para creación y gestión de grupos asociados a cursos. Esta separación reduce errores de administración cruzada.

8.7.5. Operaciones destacadas

El perfil administrador puede crear usuarios individualmente o por importación de archivo, asignar rol de profesor o estudiante (con grupo en el caso de estudiante), crear cursos y vincular profesorado, y estructurar grupos con reasignación de alumnado. Estas operaciones concentran el gobierno funcional del sistema.

Este perfil concentra operaciones de gobierno funcional, por lo que se recomienda limitar su uso a personal autorizado.

8.7.6. Políticas recomendadas de gobierno de usuarios

Para mantener orden operativo y seguridad, se recomienda aplicar el principio de mínimo privilegio en asignación de roles, realizar revisión periódica de cuentas inactivas o duplicadas, separar cuentas de administración de cuentas de uso docente y registrar cambios sensibles en altas, bajas y cambios de rol. Estas medidas reducen riesgo de exposición accidental.

8.8– Mensajes de error y resolución de incidencias

Las incidencias más frecuentes y su acción recomendada incluyen credenciales inválidas (verificar correo y contraseña), sesión expirada (iniciar sesión de nuevo), archivo no permitido (revisar tipo, extensión y tamaño del entregable), ausencia de permisos (comprobar rol y pertenencia a curso o grupo) y error al enviar entrega (reintentar y comprobar conexión). Esta orientación reduce tiempo de resolución.

Como recomendación general, ante errores persistentes debe registrarse la hora, operación realizada y mensaje mostrado para facilitar diagnóstico.

8.8.1. Matriz de incidencias operativas

Incidencia	Causa habitual	Acción recomendada
No se puede iniciar sesión	Credenciales incorrectas o sesión expirada	Reintentar autenticación y, si procede, usar recuperación de contraseña
Entrega rechazada	Tipo de archivo o tamaño no permitido	Revisar configuración del entregable y ajustar archivo
No aparece actividad/entregable	Visibilidad desactivada o grupo no asignado	Verificar configuración por parte del profesorado
No se puede calificar	Falta de permisos por rol/-contexto	Confirmar vinculación docente con curso/grupo
Error en descarga ZIP	Interrupción de red o contenido no disponible	Reintentar descarga y revisar estado de materiales

Tabla 8.2: Matriz de incidencias frecuentes y resolución recomendada

8.9– Seguridad y protección de la información en el uso diario

La seguridad operativa depende tanto de mecanismos técnicos como de buenas prácticas de uso. Para preservar confidencialidad e integridad de datos académicos, se recomienda no compartir contraseñas ni tokens de sesión, cerrar sesión en dispositivos compartidos, evitar cargas de archivos desde equipos no confiables y revisar periódicamente permisos y visibilidad de notas. Estas prácticas reducen riesgos de acceso no autorizado.

Estas medidas reducen riesgos de acceso no autorizado y publicación accidental de información académica sensible.

8.10– Checklist operativo por perfil

Como apoyo de uso rápido se incluye un checklist resumido por rol (Tabla 8.3): el administrador se centra en el alta de usuarios y revisión de la estructura; el profesor en la configuración de actividades y evaluación; y el estudiante en la correcta realización de la entrega. Este recordatorio reduce omisiones operativas y mejora la consistencia del proceso académico.

Perfil	Checklist de operaciones clave
Administrador	• Alta y modificación de usuarios. • Asignación de roles (profesor/estudiante). • Revisión de la estructura de cursos y grupos.
Profesor	• Configuración de actividad y fechas límite. • Publicación del entregable y reglas de formato. • Corrección de entregas recibidas. • Publicación de calificaciones y feedback.
Estudiante	• Revisión de actividades visibles. • Verificación de los requisitos de entrega. • Envío de la entrega en plazo estipulado. • Verificación del estado, nota y feedback.

Tabla 8.3: Checklist operativo por perfil

8.11– Buenas prácticas de uso

Para minimizar incidencias operativas se recomienda comprobar siempre fecha límite y tipo de entrega antes de enviar, revisar la vista de detalle tras cada acción crítica, mantener actualizado el perfil de usuario, evitar compartir credenciales y cerrar sesión en equipos compartidos. Estas acciones consolidan un uso seguro y ordenado.

8.12– Alternativa a la enseñanza virtual y aspectos novedosos

La plataforma se centra en el ciclo de actividades, entregables y evaluación. En contextos donde no se requiere un LMS completo, puede funcionar como alternativa ligera, con menor complejidad operativa y una curva de aprendizaje más directa. Cuando ya existe un entorno como Moodle, el sistema puede actuar como complemento especializado para la gestión de entregas, correcciones y publicación de notas, manteniendo el resto de la docencia (contenidos, foros, cuestionarios) en el LMS matriz. La convivencia es viable mediante enlaces cruzados y coordinación de calendarios, evitando duplicidades en el trabajo docente.

Como aspecto novedoso, el proyecto introduce capacidades que amplían la operativa habitual de un LMS convencional. Por un lado, el uso de APIs de OneDrive y correo permite autorizar el acceso del usuario a su cuenta (OAuth2) y gestionar entregas en su almacenamiento en la nube, mientras que el canal de correo (operado mediante Brevo) soporta notificaciones y flujos seguros de recuperación. Por otro, la importación directa desde OneDrive habilita la incorporación de la estructura de carpetas del usuario para organizar entregables, manteniendo trazabilidad total con su repositorio personal de trabajo.

8.13– Desafíos técnicos encontrados en el proyecto

Los principales retos técnicos identificados durante el desarrollo y el despliegue incluyen la incompatibilidad de ESLint 9 con configuraciones clásicas (resuelta con migración a formato *flat config*), restricciones de Render en servicios Static Site (plataforma de despliegue principal ya descrita en el Capítulo 1) que exigieron ajustes del blueprint, y restricciones de correo saliente en PaaS mitigadas con proveedores HTTP (Brevo) para correos transaccionales. También se abordó la migración de proveedor de correo transaccional: inicialmente se utilizó SendGrid, pero al finalizar su plan gratuito se migró a Brevo; se evaluó Resend como alternativa, aunque su plan gratuito exige dominio verificado, y el diseño del `EmailService` con patrón de proveedor desacoplado permite intercambiar proveedores sin modificar lógica de negocio. Se suman conflictos y deprecaciones en SonarCloud solventados migrando al plugin de Maven, tiempos excesivos en OWASP Dependency Check sin API Key de NVD mitigados con credenciales dedicadas, errores CORS en despliegue (403) al no incluir el origen del frontend en la configuración, e inactividad del backend en el tier gratuito mitigada mediante *keep-alive* automatizado.

8.14– Análisis de la evolución del proyecto

El historial de commits con prefijo `feat` refleja una evolución en fases (36 registros). La fase de **arranque y base** incluye el commit inicial, definición de estructura y primeros flujos de CI; la fase de **infraestructura** agrupa despliegue continuo, Render y workflows de mantenimiento y seguridad (Dependabot, Sonar, auditorías). La **funcionalidad nuclear** incorpora JWT, recuperación de contraseña, mejoras de administración, integración OneDrive, descargas y panel de evaluaciones; la fase de **calidad y robustez** amplía test, mutación y cobertura eliminando *hotspots*; y la fase de **documentación** consolida la memoria del TFG. Esta progresión evidencia una evolución incremental y trazable.

8.15– Conclusiones

El manual de usuario consolida el uso operativo del sistema y garantiza que los tres perfiles principales disponen de un procedimiento claro, coherente con las reglas de negocio y alineado con los requisitos definidos.

Su existencia facilita adopción, reduce errores de uso y mejora la calidad del servicio académico prestado por la plataforma.

Conclusiones y desarrollos futuros

9.1– Síntesis final del trabajo

El desarrollo realizado ha permitido construir un sistema web de gestión de entregables académicos, integrando análisis formal de requisitos, diseño de arquitectura, implementación por capas, verificación en profundidad y despliegue continuo.

El proyecto implementa una solución especializada en la gestión integral del ciclo de vida de los entregables, integrando las fases de evaluación y retroalimentación (feedback) docente. A diferencia de las plataformas generalistas, este sistema ofrece un flujo automatizado de extremo a extremo (end-to-end), minimizando la intervención manual mediante una arquitectura adaptada específicamente a la validación y control de los archivos.

9.2– Grado de cumplimiento de objetivos

El objetivo general planteado en la memoria (diseñar e implementar una plataforma de gestión de entregables usable, segura y evolutiva) puede considerarse **cumplido**.

Respecto a los objetivos específicos, el balance es positivo. El **modelado del dominio académico** se considera cumplido al haberse implementado entidades, relaciones y reglas de negocio acordes al ERS/DAS. El **soporte completo del flujo de entrega y evaluación** está cubierto con el ciclo de vida de entregables, entregas, calificación y feedback, y el **control de acceso por roles y contexto** se materializa con verificaciones por perfil y recurso objetivo. Asimismo, la **calidad técnica y verificación** queda respaldada por una estrategia de pruebas en capas con métricas elevadas de cobertura y mutación, y el **despliegue y operación automatizada** se sostiene en pipelines CI/CD, controles de seguridad y configuración por entornos.

9.3– Aportaciones principales

Las aportaciones más relevantes del trabajo se articulan en cuatro planos claramente diferenciados:

- **Plano funcional:** Se entrega una plataforma centrada específicamente en la operativa docente, con flujos diferenciados y especializados para los perfiles de administración, profesorado y alumnado.
- **Plano técnico:** Se diseña y consolida una arquitectura modular separada en cliente (SPA) y servidor (API REST), con persistencia versionada controlada mediante Flyway, seguridad basada en tokens JWT y soporte para integraciones cloud (como almacenamiento en OneDrive o correo mediante Brevo).

- **Plano de calidad:** Se incorpora un sistema de verificación exhaustivo mediante testing multinivel y pruebas de mutación (con PIT y Stryker) como criterio objetivo para garantizar la robustez del código productivo.
- **Plano de operación:** Se establece un canal de integración y despliegue continuo (CI/CD) completamente reproducible, incorporando validaciones automáticas de código, calidad y seguridad en cada iteración.

9.4– Resultados y evidencia

La evidencia disponible en el proyecto permite sostener de forma objetiva la madurez alcanzada: existe cobertura alta en frontend con umbrales exigentes en líneas, funciones, sentencias y ramas; los resultados de mutación en frontend y backend son favorables, indicando que los tests detectan alteraciones de comportamiento; se han validado flujos de usuario críticos mediante E2E; y las verificaciones están automatizadas en pipelines de integración continua. Esta evidencia reduce el riesgo de regresión.

En conjunto, estos resultados reducen el riesgo de regresión y proporcionan una base fiable para mantenimiento y evolución del sistema.

Las decisiones de arquitectura, pruebas y seguridad se han contrastado con documentación técnica oficial de las tecnologías empleadas y recomendaciones de referencia en seguridad web (VMware, 2026; Meta, 2026; Redgate, 2026; Contributors, 2026; Microsoft, 2026a; O. Foundation, 2021).

9.5– Evaluación del impacto por tipo de usuario

El valor del proyecto se aprecia de forma diferente según el perfil de uso. Por ello, se realiza una evaluación diferenciada. Para el **profesorado** se observa disminución de tareas manuales repetitivas, mejor visibilidad del estado de entregas y consolidación del flujo de corrección. En el **estudiantado** el proceso de entrega es más guiado, mejora la comprensión del estado del trabajo y el acceso al feedback es más claro. En **administración** se obtiene mayor control sobre estructura académica y roles, con operaciones de gobierno centralizadas.

Desde esta perspectiva, el sistema no solo aporta funcionalidad técnica, sino también una mejora organizativa en la operativa académica.

9.6– Validación del enfoque de ingeniería adoptado

Una conclusión relevante del trabajo es que el enfoque metodológico aplicado (requisitos formalizados, arquitectura modular, pruebas en capas y CI/CD) ha sido adecuado para un proyecto de este alcance.

Los principales indicadores de validez del enfoque son la coherencia entre objetivos, requisitos, implementación y verificación, la estabilidad de las iteraciones sin desviaciones críticas de calendario y la capacidad de incorporar mejoras sin ruptura estructural del sistema. Estos indicadores refuerzan la disciplina metodológica aplicada.

Esto refuerza la idea de que, en proyectos software de carácter académico-profesional, la disciplina metodológica es un factor diferencial tan importante como la selección tecnológica.

9.7– Análisis de desviaciones en la planificación temporal

Durante el desarrollo del proyecto se monitorizó el grado de cumplimiento de los plazos estimados (Sprints y *Product Backlog Items*). Aunque el proyecto logró mantener un avance sólido

sin comprometer las fechas de entrega finales, se detectaron desviaciones específicas en la fase de integración de dependencias externas.

La implementación del módulo de integración con **OneDrive (OAuth2)** requirió más tiempo del inicialmente planificado (aproximadamente un 20 % de sobrecoste temporal en su respectivo Sprint) debido a la configuración de permisos granulares en el entorno de Azure AD y las restricciones de *CORS* en flujos federados. De forma similar, la **migración del proveedor de correo** desde SendGrid a Brevo introdujo una tarea de refactorización no prevista.

Estas desviaciones fueron absorbidas gracias a la flexibilidad de la metodología ágil adoptada, reordenando tareas de menor criticidad (funcionalidades secundarias de la UI) para garantizar la robustez del MVP. Este análisis evidencia que las dependencias externas (SaaS, APIs de terceros) representan el mayor factor de riesgo temporal en proyectos de esta naturaleza, validando así la decisión de encapsular estas integraciones en capas de servicio adaptables.

9.8– Limitaciones del trabajo

Como en todo proyecto con alcance temporal acotado, se identifican limitaciones que delimitan el estado actual. No todas las funcionalidades *Should/Could* del backlog están cerradas al 100 % en el corte del TFG, ciertas capacidades avanzadas (p. ej., autenticación federada completa, 2FA integral u observabilidad avanzada) se mantienen como evolución posterior, y la estrategia E2E predominante en el ciclo actual es mockeada, por lo que conviene reforzar escenarios con backend real en staging. Estas limitaciones orientan la hoja de ruta futura.

Estas limitaciones no invalidan el cumplimiento del objetivo principal, pero orientan de forma clara la hoja de ruta futura.

9.9– Lecciones aprendidas

Durante la ejecución del proyecto se consolidan varios aprendizajes de interés para trabajos similares. Diseñar la trazabilidad desde el inicio (requisito-backlog-código-prueba) simplifica justificación técnica y auditoría; la calidad debe construirse iterativamente y no solo en la fase final de cierre; desacoplar configuración por entorno reduce fricción en migraciones de infraestructura; e integrar seguridad y permisos en la lógica de dominio evita defectos estructurales costosos. Estos aprendizajes son transferibles a proyectos académicos similares.

9.10– Riesgos residuales y deuda técnica controlada

En el estado actual se identifican riesgos residuales, asumidos de forma explícita y controlada. Se reconoce cobertura funcional incompleta en funcionalidades *Should/Could*, la necesidad de ampliar E2E contra backend real para ciertos escenarios y la dependencia parcial de configuraciones externas en integraciones cloud. Esta deuda se considera controlada por estar identificada y priorizada.

La deuda técnica asociada se considera **controlada**, porque está identificada, priorizada y conectada con una hoja de ruta de evolución.

9.11– Desarrollos futuros

Se proponen líneas de evolución priorizadas por impacto y viabilidad.

9.11.1. Corto plazo

En el corto plazo se propone cerrar PBIs pendientes de usabilidad y reporting docente, ampliar E2E con entorno semirreal y pruebas de regresión ampliadas, y reforzar auditoría de permisos y

evidencias de seguridad en preproducción. Estas acciones aumentan robustez sin ampliar de forma significativa el alcance funcional.

9.11.2. Medio plazo

En el medio plazo se plantea incorporar autenticación federada institucional completa y doble factor transversal, observabilidad avanzada (métricas de negocio, trazas distribuidas y alertado) y funcionalidades analíticas para seguimiento de actividad y carga de corrección.

9.11.3. Largo plazo

En el largo plazo se contempla la incorporación de módulos de apoyo docente (detección de patrones de entrega, recomendación de revisión, etc.) y el estudio de interoperabilidad con LMS institucionales. Estas líneas amplían impacto sin comprometer el núcleo existente.

9.12– Hoja de ruta priorizada de continuidad

Para facilitar la continuidad del producto, se sintetiza una priorización por horizonte temporal y retorno esperado.

Horizonte	Iniciativas principales	Impacto esperado
Corto plazo	cierre de PBIs pendientes, ampliación de E2E y refuerzo de auditoría de permisos	mejora inmediata de robustez operativa
Medio plazo	federación de identidad, 2FA y observabilidad avanzada	aumento de seguridad y gobierno técnico
Largo plazo	interoperabilidad institucional y módulos avanzados de apoyo docente	escalabilidad funcional y adopción ampliada

Tabla 9.1: Hoja de ruta priorizada para evolución del sistema

9.13– Reflexión final

El trabajo realizado demuestra que es posible construir una solución tecnológica robusta capaz de digitalizar y orquestar el ciclo de vida completo de las actividades académicas. En concreto, el prototipo desarrollado cubre de principio a fin la creación estructurada de actividades, la parametrización de entregables (con fechas de inicio/fin, visibilidad y restricciones de formato) y el seguimiento en tiempo real del estado de cada entrega.

La principal aportación del proyecto frente a los flujos tradicionales es la automatización transparente del sistema de ficheros. Gracias a la integración nativa con la API de OneDrive, el sistema elimina por completo el trabajo manual de descarga masiva, descompresión y clasificación de documentos por parte del profesorado. Cada fichero subido por el alumno se valida, se renombra bajo una convención unificada y se almacena directamente en la estructura de carpetas en la nube del docente, asociándose inmediatamente a un panel de evaluación con retroalimentación estructurada. Este enfoque resuelve directamente la fricción operativa y reduce el riesgo de pérdida documental.

9.14– Cierre

Este Trabajo Fin de Grado culmina con la entrega de una plataforma web plenamente operativa, diseñada específicamente para centralizar y simplificar el flujo de entregas en el ámbito

universitario. Al combinar una arquitectura moderna y desacoplada con la automatización inteligente del almacenamiento, el producto no solo cumple con los rigurosos requisitos técnicos de trazabilidad, seguridad y pruebas automatizadas, sino que entrega valor operativo directo.

La solución obtenida permite a los docentes centrar su esfuerzo en la revisión y calificación académica en lugar de en la logística de archivos, proporcionando a su vez a los estudiantes un entorno guiado y seguro para el depósito de sus prácticas. Por consiguiente, el sistema desarrollado resuelve con éxito la problemática inicial y consolida una base sólida para una posible adopción o extensión institucional.

Manual de instalación

A.1– Requisitos previos

Para ejecutar la solución en entorno local se requiere JDK 21 y Maven 3.9+ para backend, Node.js 18+ y npm 9+ para frontend, y PostgreSQL 14+ disponible en local o contenedor. Estos requisitos aseguran compatibilidad con el stack desplegado.

A.2– Preparación del entorno

La preparación del entorno consiste en clonar el repositorio y situarse en la raíz del proyecto, y en verificar credenciales y variables según el Anexo B. Este orden evita configuraciones incompletas antes de arrancar servicios.

A.3– Base de datos

Para entorno local se recomienda PostgreSQL con la base `tf gdb` y el usuario `tf g`. El repositorio incluye un script de apoyo `backend/run-postgres.ps1` que comprueba el servicio y arranca el backend. En caso de preferir contenedor, puede utilizarse `docker-compose` (ver subsección correspondiente).

A.4– Arranque del backend

Pasos de ejecución recomendados:

```
cd backend
./mvnw clean install
./mvnw spring-boot:run
```

Si se desea forzar configuración local con PostgreSQL, puede activarse el perfil `local` mediante `SPRING_PROFILES_ACTIVE=local`. Para pruebas rápidas sin base externa, está disponible el perfil `h2`.

A.5– Arranque del frontend

```
cd frontend
npm install
npm run dev
```

Por defecto el frontend consumirá la API local. Si es necesario, ajustar `VITE_API_BASE_URL` según el entorno.

A.6– Ejecución con contenedores

Como alternativa reproducible, la solución puede ejecutarse con `docker-compose`, levantando base de datos y backend con healthchecks. Esta opción reduce diferencias entre máquinas de desarrollo.

A.7– Build de producción

Para generar artefactos de despliegue:

```
cd backend
./mvnw clean package -DskipTests

cd frontend
npm run build
```

Configuración por entornos

B.1– Variables críticas de backend

Las variables mínimas para el funcionamiento seguro en entornos no locales incluyen los parámetros de conexión a la base de datos (`DATABASE_URL`, `DATABASE_USER`, `DATABASE_PASSWORD`), la clave privada para la firma criptográfica de tokens de sesión (`JWT_SECRET`), el perfil de entorno activo para la inyección de dependencias (`SPRING_PROFILES_ACTIVE`), y la URL base del cliente web para configurar correctamente las reglas CORS (`APP_FRONTEND_BASE_URL`). Esta configuración garantiza conectividad, seguridad y coherencia de entorno.

En producción se habilitan además variables para servicios externos como almacenamiento cloud y correo transaccional. En el despliegue sobre Render, el correo se configura mediante Brevo usando API HTTP, por lo que deben definirse `BREVO_API_KEY` y `BREVO_FROM_EMAIL` con un remitente verificado en Brevo. Esta elección evita depender de conexiones SMTP salientes, que pueden estar restringidas por el PaaS.

B.2– Variables críticas de frontend

La variable principal de configuración de cliente es `VITE_API_BASE_URL`, que define el endpoint de API consumido.

Operación y verificación

C.1– Comprobaciones de salud

El sistema expone endpoints de salud para diferenciación operativa: el de *liveness* valida disponibilidad del proceso y el de *readiness* valida disponibilidad real de dependencias como la base de datos. Esta separación permite detectar fallos de conectividad de forma temprana.

Esta separación permite detectar de forma temprana incidencias de conectividad y reducir falsos positivos en monitorización.

C.2– Validación post-despliegue

Tras cada despliegue se recomienda, como mínimo, comprobar endpoints de salud, realizar login con usuario de prueba, ejecutar operación de lectura y escritura en flujo académico, verificar subida y descarga de material y probar el correo transaccional (recuperación de contraseña o notificación) mediante Brevo. Estas comprobaciones validan la operativa esencial en producción.

Declaración de uso de IA generativa

En este TFG se ha utilizado IA generativa como herramienta de apoyo en tareas acotadas y verificables, con el objetivo de mejorar claridad documental, revisar estructura y acelerar actividades de bajo riesgo. La IA no ha sustituido el criterio técnico ni ha generado decisiones de diseño o implementación sin validación humana.

D.1– Alcance y criterios de uso

El uso de IA se limitó a apoyo de redacción y reformulación de texto técnico, revisión de estructura y coherencia de secciones, generación de listados de verificación y casos borde, microcopy y textos de ayuda para interfaz, y aclaraciones puntuales de sintaxis o formatos (por ejemplo, JSON o $\text{\LaTeX}$). Estas tareas se consideraron de bajo riesgo y siempre se validaron manualmente.

Se excluyeron explícitamente usos como decisiones arquitectónicas finales sin justificación propia, código productivo incorporado sin revisión manual, resultados empíricos o métricas no verificadas y contenido no contrastado con el repositorio y la evidencia de pruebas. Esta restricción garantiza responsabilidad técnica.

D.2– Proceso de control y verificación

Cada propuesta generada por IA se trató como borrador. El proceso seguido consistió en formular la consulta indicando el contexto del proyecto, evaluar si la respuesta era coherente con requisitos y código, ajustar manualmente terminología y rigor académico, contrastar con artefactos del repositorio (memoria, código, tests) e integrar la versión final ya revisada. Este flujo evita incorporaciones automáticas sin validación.

D.3– Ejemplos concretos de consultas y respuestas

Los siguientes ejemplos resumen consultas reales y la respuesta resumida que se utilizó como punto de partida. En todos los casos se realizó revisión manual antes de incorporar el contenido a la memoria o al producto.

Ejemplo 1. Redacción académica de objetivos. *Consulta:* *Reescribe este párrafo de objetivos para un tono académico, evitando repeticiones y manteniendo el significado.* **Respuesta resumida:** propuesta de redacción más concisa con estructura en dos frases y vocabulario técnico coherente. **Aplicación:** se ajustaron términos a la nomenclatura del ERS y se incorporó solo tras validar la trazabilidad con requisitos.

Ejemplo 2. Estructura de sección comparativa. *Consulta:* Sugiere un esquema para un análisis de competidores en una memoria de TFG de software. **Respuesta resumida:** se propuso dividir por tipologías de competidor, incluir criterios de comparación y cerrar con diferenciadores propios. **Aplicación:** el esquema se adaptó al contenido real del proyecto y a las evidencias técnicas implementadas.

Ejemplo 3. Validación de estructuras ZIP. *Consulta:* Propón ejemplos de reglas para validar estructura interna de un ZIP con archivos obligatorios y carpetas esperadas. **Respuesta resumida:** lista de casos con archivos obligatorios, extensiones permitidas y modos estricto/mínimo requerido. **Aplicación:** se contrastó con la lógica del servicio de validación y se redactó la explicación en la memoria con terminología consistente.

Ejemplo 4. Textos de ayuda en interfaz de entrega. *Consulta:* Sugiere mensajes breves y claros para indicar que una entrega requiere un ZIP con nombre y estructura concretos. **Respuesta resumida:** propuestas de microcopy con tono informativo y referencias explícitas a nombre esperado, tamaño y estructura. **Aplicación:** los textos se ajustaron manualmente para alinearlos con el vocabulario usado en la UI del proyecto.

Ejemplo 5. Casos borde para pruebas. *Consulta:* Enumera casos borde para pruebas de subida de ZIP en un sistema de entregables. **Respuesta resumida:** listado de escenarios (nombre incorrecto, estructura incompleta, archivos extra, ZIP corrupto, tamaño excedido). **Aplicación:** se utilizó como checklist de verificación y se descartaron casos no aplicables al alcance real.

Este anexo se incluye para garantizar transparencia metodológica y trazabilidad del proceso de elaboración de la memoria.

Documentación complementaria disponible

Como soporte adicional al contenido principal de la memoria, el repositorio oficial del proyecto incorpora documentos técnicos de operación, seguridad, testing y despliegue que permiten ampliar detalle en revisiones futuras.

El código fuente del proyecto se encuentra alojado en GitHub y está disponible en el siguiente enlace: <https://github.com/Mancalrod/TFG>.

Dentro del repositorio, se puede consultar la documentación técnica complementaria en la ruta `docs/`, donde se encuentran esquemas, configuraciones de despliegue y otros artefactos de diseño adicionales no incluidos explícitamente en esta memoria.

Caso de uso de ejemplo

Para ilustrar de forma unificada la funcionalidad de la plataforma, se describe a continuación el recorrido completo (flujo de extremo a extremo) de una práctica a través de los tres perfiles del sistema: Administrador, Profesor y Estudiante. Este recorrido es representativo de la operativa del TFG y coincide con el hilo conductor de la demostración en vídeo del proyecto.

F.1– 1. Administrador: Configuración inicial

El administrador accede a la plataforma y crea un curso denominado “Ingeniería de Software”. Seguidamente, da de alta a dos usuarios: un **profesor** y un **estudiante**, y crea un grupo llamado “Grupo A”. Finalmente, asocia al profesor como docente del curso y vincula al estudiante al “Grupo A”. Con esto, la base académica queda lista.

F.2– 2. Profesor: Creación de la actividad y el entregable

El profesor inicia sesión y visualiza el curso “Ingeniería de Software” en su panel. Entra en él y crea una nueva **Actividad** (ej: “Práctica 1”), asignándola al “Grupo A” para que sólo sus estudiantes la vean. Dentro de la actividad, define un **Entregable**, configurando la fecha límite y exigiendo un formato específico (por ejemplo, un archivo ZIP con un documento PDF obligatorio en su interior).

F.3– 3. Estudiante: Revisión y entrega

El estudiante inicia sesión y, en su dashboard, observa que tiene una nueva actividad pendiente en el curso “Ingeniería de Software”. Accede a los detalles y comprueba los requisitos del entregable (fecha y formato ZIP). El estudiante prepara su archivo en el ordenador local, accede al formulario de entrega, sube su ZIP y pulsa enviar. El sistema valida en tiempo real la extensión y estructura del ZIP y, tras procesarlo correctamente, le confirma que la entrega ha sido registrada.

F.4– 4. Profesor: Corrección y retroalimentación

Más tarde, el profesor accede a la sección de **Evaluaciones** o al detalle del entregable y observa la entrega pendiente del estudiante. Abre la entrega, donde puede previsualizar el contenido o descargarlo. Tras revisarlo, le asigna una calificación de “8.5/10” y redacta un comentario detallado con sugerencias de mejora. Al guardar la evaluación, el profesor publica las notas.

F.5– 5. Estudiante: Consulta de resultados

El estudiante vuelve a iniciar sesión. En su vista de calificaciones (y en el detalle de la propia entrega) puede ver ahora su nota de 8.5 y leer la retroalimentación proporcionada por el profesor, cerrando así el ciclo completo de la práctica de forma trazable y transparente.

Referencias

- Brevo. (2026). *Brevo: Email marketing and more*. Descargado de <https://www.brevo.com/> (Fecha de consulta: 10 de mayo de 2026)
- Consortium, A. B. (2026). *Moscow method*. Descargado de <https://www.agilebusiness.org/page/ProjectFramework10MoSCoWPrioritisation> (Fecha de consulta: 18 de mayo de 2026)
- Contributors, V. (2026). *Vitest documentation*. Descargado de <https://vitest.dev/> (Fecha de consulta: 8 de abril de 2026)
- de Sevilla, U. (2026). *Proteus: Herramienta de modelado de requisitos*. Descargado de <https://github.com/proteus-University-of-Seville/proteus-core> (Fecha de consulta: 10 de mayo de 2026)
- EclEmma. (2026). *Jacoco: Java code coverage library*. Descargado de <https://www.eclemma.org/jacoco/> (Fecha de consulta: 10 de mayo de 2026)
- Foundation, O. (2021). *Owasp top 10: The ten most critical web application security risks*. Descargado de <https://owasp.org/www-project-top-ten/> (Fecha de consulta: 8 de abril de 2026)
- Foundation, O. (2026). *Node.js*. Descargado de <https://nodejs.org/> (Fecha de consulta: 10 de mayo de 2026)
- Foundation, T. A. S. (2026). *Apache maven*. Descargado de <https://maven.apache.org/> (Fecha de consulta: 10 de mayo de 2026)
- GitHub. (2026). *Github actions: Automate your workflow from idea to production*. Descargado de <https://github.com/features/actions> (Fecha de consulta: 10 de mayo de 2026)
- Group, P. G. D. (2026). *Postgresql: The world's most advanced open source relational database*. Descargado de <https://www.postgresql.org/> (Fecha de consulta: 10 de mayo de 2026)
- IETF. (2012). *Rfc 6749: The oauth 2.0 authorization framework*. Descargado de <https://datatracker.ietf.org/doc/html/rfc6749> (Fecha de consulta: 10 de mayo de 2026)
- Inc., D. (2026). *Docker: Accelerated container application development*. Descargado de <https://www.docker.com/> (Fecha de consulta: 10 de mayo de 2026)
- Meta. (2026). *React documentation*. Descargado de <https://react.dev/> (Fecha de consulta: 8 de abril de 2026)
- Microsoft. (2026a). *Playwright documentation*. Descargado de <https://playwright.dev/> (Fecha de consulta: 8 de abril de 2026)
- Microsoft. (2026b). *Typescript: Javascript with syntax for types*. Descargado de <https://www.typescriptlang.org/> (Fecha de consulta: 10 de mayo de 2026)
- Networks, R. (2026). *Render: Cloud hosting for developers*. Descargado de <https://render.com/> (Fecha de consulta: 10 de mayo de 2026)
- PITest. (2026). *Pit: State of the art mutation testing systems for the jvm*. Descargado de <https://pitest.org/> (Fecha de consulta: 10 de mayo de 2026)

- Redgate. (2026). *Flyway documentation*. Descargado de <https://documentation.red-gate.com/flyway> (Fecha de consulta: 8 de abril de 2026)
- SonarSource. (2026). *Sonarcloud: Clean code as a service*. Descargado de <https://sonarcloud.io/> (Fecha de consulta: 10 de mayo de 2026)
- Team, S. M. (2026). *Stryker mutator*. Descargado de <https://stryker-mutator.io/> (Fecha de consulta: 10 de mayo de 2026)
- VMware. (2026). *Spring boot reference documentation*. Descargado de <https://docs.spring.io/spring-boot/documentation.html> (Fecha de consulta: 8 de abril de 2026)
- You, E. (2026). *Vite: Next generation frontend tooling*. Descargado de <https://vitejs.dev/> (Fecha de consulta: 10 de mayo de 2026)